竞赛算法笔记

算法、数据结构与数学速查手册

基础技巧与工程　数学与数论　数据结构　图论与树
字符串　离线算法　计算几何与扫描线　动态规划与优化

2026 年 9 月

目录

阅读约定 5

第一章 基础技巧与工程 6

1.1 速记与小工具 6
1.2 差分约束 6
1.3 `bitset` 按 64 位分块读取 6
1.4 编译、运行与对拍脚本 7
1.5 向量上的 max-plus 卷积 7
1.6 位运算性质与 Montgomery 乘法片段 8
1.7 二进制前缀区间分解 8
1.8 数学速记 9
1.9 子集枚举与随机数 10
1.10 无向图重定向 10
1.11 曼哈顿距离与切比雪夫距离 11
1.12 位运算内建函数 11
1.13 Tarjan 点双与圆方树片段 11
1.14 组合数与逆元预处理 12
1.15 布尔矩阵乘法辅助生成器 12
1.16 日期与星期 13
1.17 手写哈希表 14
1.18 Python 速查 16

第二章 数学与数论 18

2.1 组合恒等式 18
2.2 拉格朗日插值 19
2.3 二进制 GCD 23
2.4 扩展中国剩余定理（EXCRT） 23
2.5 值域 GCD 预处理 24
2.6 线性递推求逆元 25
2.7 大整数质因数分解 25
2.8 组合数预处理 (卢卡斯定理) 27
2.9 去除素因子后的阶乘模 28
2.10 约数函数 29
2.11 离散对数（BSGS / EXBSGS） 30
2.12 欧拉函数 31
2.13 莫比乌斯函数 33
2.14 矩阵快速幂 33
2.15 快速傅里叶变换（FFT） 34
2.16 Min_25 筛 35
2.17 大范围素数计数 37
2.18 生成 10^9 内素数 37
2.19 位运算卷积（FWT） 38

2.20 多项式与 NTT 40
2.21 自适应辛普森积分 45
2.22 行列式 45
2.23 高斯消元 46
2.24 GCD 卷积 47
2.25 Trygub numbers 48

第三章 数据结构 50

3.1 ST 表 50
3.2 二维前缀和与差分 50
3.3 分块 51
3.4 ZKW 线段树 51
3.5 懒标记线段树 52
3.6 线段树合并 54
3.7 线段树 55
3.8 李超树 57
3.9 树状数组（Fenwick） 58
3.10 线性基 58
3.11 主席树（可持久化线段树） 60
3.12 并查集 62
3.13 带权并查集 63
3.14 可撤销并查集 63
3.15 Link-Cut Tree（LCT） 64
3.16 笛卡尔树 65
3.17 猫树 66
3.18 区间加与区间和 67
3.19 珂朵莉树（ODT） 69
3.20 状态压缩 RMQ 70

第四章 图论与树 72

4.1 树链之外的 dfn 区间（至多 5 段） 72
4.2 虚树 73
4.3 树上最近公共祖先（LCA） 73
4.4 链式前向星 77
4.5 强连通分量（SCC） 78
4.6 2-sat 80
4.7 线段树优化建图 85
4.8 树链剖分 87
4.9 最大流与最小割 88
4.10 最小费用最大流 90
4.11 欧拉路径与回路 92
4.12 最大团 93

第五章 字符串 95

5.1 Manacher 95
5.2 字符串哈希 95
5.3 单哈希（自然溢出） 96
5.4 单哈希（取模） 97
5.5 字典序最大后缀（双指针） 97
5.6 后缀数组 98

5.7 Z 函数 100
5.8 KMP 100
5.9 AC 自动机 101
5.10 后缀自动机 103
5.11 广义后缀自动机求多个串不同字串模板 106

第六章 离线算法 107
6.1 莫队算法 107
6.2 回滚莫队（只增） 108
6.3 带修莫队 109
6.4 树上莫队 111

第七章 计算几何与扫描线 115
7.1 矩形并面积（整数网格） 115
7.2 二维矩形加与矩形查询 116
7.3 计算几何基础 118

第八章 动态规划与优化 125
8.1 背包与方案恢复 125

阅读约定

> 这是一份以 C++ 模板为主的竞赛速查笔记。公式使用 LaTeX；代码保留原始思路，使用前请按题目下标、边界和模数核对。PDF 按 A4 版式排版，目录条目和书签均可跳转到对应章节。

- 模板默认使用 C++17；`Info`、`Tag`、模数和数组范围需要按题目自行替换。
- 复杂度说明是该算法的典型复杂度，不代表每个片段都已经封装成可直接提交的完整程序。
- 标记为 “片段” 或 “实验记录” 的内容保留了原始上下文，使用前应先补齐变量和边界条件。

第 1 章

基础技巧与工程

> 零散但高频的竞赛技巧、脚本和语言工具。

1.1 速记与小工具

用途：将原来混在一起的零散公式、底层技巧、脚本和辅助代码按用途拆开。

1.2 差分约束

用途：把不等式约束转成图上的边，再用最短路或最长路判断可行性并求边界。

对于 $x_i - x_j \le a$，从 j 向 i 连权值为 a 的边；最短路给出各点的最大可行值。对于 $x_i - x_j \ge a$，同样从 j 向 i 连边，改用最长路视角求最小值。图不连通时应补超级源点。

1.3 bitset 按 64 位分块读取

用途： t1 是位集合时，把其底层存储按 unsigned long long 分块读取，便于批量进行位运算；这是竞赛中常见的实现相关优化。

```
// t1 为 bitset（或按机器字存储的自定义 bitset）
ull *vx = reinterpret_cast<ull *>(&t1);
for (int i = 0; i < WORDS; i++) {
  // 常见实现下，vx[i] 表示 bitset 的第 64 * i ~ 64 * i + 63 位
}
```

WORDS 通常取 (N + 63) / 64。标准库 std::bitset 不保证内部布局，只有在确认编译器实现，或使用自定义的 vector<ull> / array<ull, K> 存储时才能这样做；否则应改用可移植的逐位接口。

例如，假设采用“低下标对应低位”的常见布局：

数组元素	对应的 bitset 下标	取第 k 位
vx[0]	0 到 63	vx[0] >> k（0 \le k < 64）
vx[1]	64 到 127	vx[1] >> (k - 64)（64 \le k < 128）
vx[2]	128 到 191	vx[2] >> (k - 128)（128 \le k < 192）

统一写法是：

```
int word = k >> 6;       // k / 64
int offset = k & 63;     // k % 64
bool on = (vx[word] >> offset) & 1;
```

比如 t1[0] = t1[3] = t1[64] = 1 时，常见布局下 vx[0] 的低 4 位为 1001，vx[1] 的最低位为 1；数值上就是 vx[0] = 9、vx[1] = 1。位编号和字节序仍应以所用实现为准。

1.4 编译、运行与对拍脚本

用途：用 Bash 快速编译、运行、查看代码，并对拍 good.cpp（正解）与 bad.cpp（待测程序）。

1.4.1 .bashrc 中的快捷命令

```
r() {
    (($# == 1)) || { echo "usage: r <file>" >&2; return 2; }
    local src="${1%.cpp}.cpp" exe="${1%.cpp}"
    g++ -std=c++17 -O2 -pipe "$src" -o "$exe" && "./$exe"
}

p() {
    (($# == 1)) || { echo "usage: p <file>" >&2; return 2; }
    cat "${1%.cpp}.cpp"
}

s() {
    (($# == 1)) || { echo "usage: s <file>" >&2; return 2; }
    submit "${1%.cpp}.cpp"       # 换成比赛环境提供的提交命令
}
```

保存后执行 source ~/.bashrc，即可使用 r a、p b、s b。其中 r 编译并运行，p 将源码输出到终端；s 只是提交命令的包装，ICPC 并不存在统一的官方 s 命令，具体命令由比赛系统决定。

1.4.2 Linux 对拍脚本

将下面内容保存为 stress.sh，并与 good.cpp、bad.cpp、make.cpp 放在同一目录：

```
#!/usr/bin/env bash
g++ -std=c++17 -O2 -pipe good.cpp -o .good || exit 1
g++ -std=c++17 -O2 -pipe bad.cpp  -o .bad  || exit 1
g++ -std=c++17 -O2 -pipe make.cpp -o .make || exit 1

for ((tc = 1; ; tc++)); do
    ./.make > .in
    ./.good < .in > .good.out
    ./.bad  < .in > .bad.out
    if ! diff -Z -B .good.out .bad.out >/dev/null; then
        echo "WA on test $tc"
        cp .in failed.in
        diff -u .good.out .bad.out
        exit 1
    fi
    ((tc % 100 == 0)) && echo "AC $tc"
done
```

执行 chmod +x stress.sh 后运行 ./stress.sh；出现错误时，输入会保存在 failed.in。

1.5 向量上的 max-plus 卷积

用途：对已经按差分排序的序列做 max-plus 合并；常见于凸函数或斜率优化。

```
std::vector<i64> operator*(const std::vector<i64> &a,
                           const std::vector<i64> &b) {
  if (a.empty() || b.empty()) {
    return {};
  }
  std::vector<i64> c(a.size() + b.size() - 1);
```

```
  int i = 0, j = 0;
  c[0] = a[0] + b[0];
  while (i + j + 1 < c.size()) {
    if (i + 1 < a.size() &&
        (j + 1 == b.size() || a[i + 1] - a[i] > b[j + 1] - b[j])) {
      i++;
    } else {
      j++;
    }
    c[i + j] = a[i] + b[j];
  }
  return c;
}
```

1.6　位运算性质与 Montgomery 乘法片段

用途：记录 Lucas 奇偶性、异或恒等式和基于高位近似的模乘技巧。

```
/*
  a^b=a+b-2*(a&b);
  // 当 n&m!=m 时 C(n,m) 为偶数。
  // 当 n&m==m 时 C(n,m) 为奇数
*/
typedef long long ll;
typedef __int128_t i128;
i128 _base = 1;
inline ll mol(ll x) { return x - mod * (_base * x >> 64); }

// in main
_base = (_base << 64) / mod;
```

1.7　二进制前缀区间分解

用途：将闭区间 $[0, h]$ 拆成 $O(\log(h+1))$ 个二进制对齐块。每个块都能写成 `prefix 00...0` 到 `prefix 11...1`，适合数位 DP、Trie 和按二进制前缀处理区间。

返回的每个二元组 `(left, suffix_bits)` 表示

$$[\,\texttt{left},\ \texttt{left} + 2^{\texttt{suffix_bits}} - 1\,].$$

其中 `left` 的低 `suffix_bits` 位全为 0。`left` 是完整的块左端点；若需要去掉末尾零位后的紧凑前缀，应再取 `left >> suffix_bits`。

```
using u64 = uint64_t;
auto bitdec = [](u64 h) {
  // h + 1 不能在无符号整数中溢出。
  assert(h != numeric_limits<u64>::max());
  vector<pair<u64, int>> blocks;
  int suffix_bits = 0;
  for (u64 x = h + 1; x; x >>= 1, ++suffix_bits) {
    if (x & 1) {
      blocks.emplace_back((x - 1) << suffix_bits, suffix_bits);
    }
  }
  return blocks;
};
```

例如 $h=12$ 时，$h+1=13=(1101)_2$。函数依次返回 `(12,0)`、`(8,2)`、`(0,3)`，对应 $[12,12]$、$[8,11]$、$[0,7]$。返回顺序是从右到左；若需要按左端点递增排列，对结果执行 `reverse`。块数恰好为 `popcount(h + 1)`，时间复杂度为 $O(\log(h+1))$。

1.8 数学速记

用途：便于在推式或构造题中快速调用的公式。

1.8.1 异或前缀

定义 $F(0)=0$，且 $F(n)=1\oplus 2\oplus\cdots\oplus n$。则

$$F(n)=\begin{cases} n, & n \bmod 4=0,\\ 1, & n \bmod 4=1,\\ n+1, & n \bmod 4=2,\\ 0, & n \bmod 4=3. \end{cases}$$

因此 $1\le l\le r$ 时，区间异或为 $l\oplus(l+1)\oplus\cdots\oplus r=F(r)\oplus F(l-1)$。

1.8.2 Wilson 定理

对整数 $p>1$，p 为质数当且仅当

$$(p-1)!\equiv -1 \pmod p.$$

若 m 为合数且 $m\neq 4$，则 $(m-1)!\equiv 0 \pmod m$；例外 $m=4$ 时，$3!\equiv 2 \pmod 4$。

1.8.3 常用幂和

$$\sum_{i=1}^{n} i^2=\frac{n(n+1)(2n+1)}{6},\qquad \sum_{i=1}^{n} i^3=\left(\frac{n(n+1)}{2}\right)^2.$$

1.8.4 错位排列

D_n 表示把 n 个不同元素放回 n 个带编号位置，并且没有元素回到原位置的排列数。初值和递推为

$$D_0=1,\qquad D_1=0,\qquad D_n=(n-1)(D_{n-1}+D_{n-2})\quad (n\ge 2).$$

1.8.5 Dilworth 在序列划分中的形式

把一个序列划分成尽量少的**非递增子序列**，最少条数等于原序列的 **最长严格上升子序列**长度。严格性必须配对：若要求每条子序列严格下降，对应的则是最长非下降子序列长度。

第二类斯特林数满足：

$$S(n,m)=S(n-1,m-1)+mS(n-1,m),\qquad S(0,0)=1.$$

```
int S[101][101] = {{1}};
for (int n = 1; n <= 100; ++n)
  for (int k = 1; k <= n; ++k)
    S[n][k] = S[n - 1][k - 1] + k * S[n - 1][k];
```

1.9　子集枚举与随机数

用途：枚举一个掩码的所有非空子集，并生成 64 位随机数。

```
int x = 13;
for (int s = x; s; s = x & (s - 1)) {
  cout << s << "\n";
}

// 枚举 n 位中恰好有 k 个 1 的集合: Gosper's Hack
uint64_t x = (1ULL << k) - 1;
uint64_t limit = 1ULL << n;
while (x < limit) {
    // use x
    uint64_t low = x & -x;
    uint64_t next = x + low;
    x = next | (((x ^ next) / low) >> 2);
}
```

对于 $x = 13 = (1101)_2$，循环按从大到小的顺序枚举其 7 个非空子集。

`mt19937_64 rnd(time(0))` 用于生成 64 位伪随机数。

1.10　无向图重定向

用途：不断删除低度点并保留剩余方向，常用于控制重定向后图的最大出度。

```
int ed = (2 * m + n - 1) / n;
vector<vector<int>> rg(n + 1);
queue<int> q;
for (int i = 1; i <= n; i++) {
  if (d[i] <= ed) {
    q.push(i);
  }
}
while (q.size()) {
  int u = q.front();
  q.pop();
  if (vis[u])
    continue;
  vis[u] = 1;
  for (auto it : g[u]) {
    if (vis[it])
      continue;
    rg[u].push_back(it);
    if (--d[it] <= ed) {
      q.push(it);
    }
  }
}
```

上面的片段按当前度数剥离顶点，并在删除顶点时确定与未删除邻点之间的方向。若题目给定阈值，应直接替换 ed；使用前还要确认题目条件保证所有顶点最终都会入队。

另一类问题是给无向图定向，使最长有向路径尽可能短。其最优值与图的最小染色数有关：按颜色编号从小到大定向后，任意有向路径经过的颜色严格递增，所以边数至多为 $\chi(G)-1$。这与本节的低度剥离模板不是同一个结论。

若只需一个固定的度数方向，也常把边从度数大的端点指向度数小的端点；度数相同时再按编号打破平局。具体出度界仍取决于题目条件。

1.11 曼哈顿距离与切比雪夫距离

用途：通过坐标变换互换两类距离，方便把几何问题转成前缀、扫描或极值问题。

$$(x,y)\mapsto\left(\frac{x+y}{2},\frac{x-y}{2}\right)$$

可把原坐标系的切比雪夫距离转成新坐标系的曼哈顿距离。

$$(x,y)\mapsto(x+y,x-y)$$

可把原坐标系的曼哈顿距离转成新坐标系的切比雪夫距离。

在该变换下，四种轴向移动分别变为：

$$(x,y+d)\mapsto(x+d,y-d),\quad (x,y-d)\mapsto(x-d,y+d),$$

$$(x+d,y)\mapsto(x+d,y+d),\quad (x-d,y)\mapsto(x-d,y-d).$$

1.12 位运算内建函数

用途：获取 1 的个数、最低位、最低位位置和奇偶校验。

```
int bitcnt(int x) { return __builtin_popcountll(x); }
int lowbit(int x) { return x & -x; }
int lowbitpos(int x) { return __builtin_ffsll(x); }
int bitmod(int x) { return __builtin_parityll(x); }
```

1.13 Tarjan 点双与圆方树片段

用途：通过 low-link 值提取点双连通分量，并把分量节点接入辅助树。

```
void Tarjan(int u) {
  low[u] = dfn[u] = ++dfc;
  stk[++tp] = u;
  ++num;
  for (int v : G[u]) {
    if (!dfn[v]) {
      Tarjan(v);
      low[u] = std::min(low[u], low[v]);
      if (low[v] == dfn[u]) {
        wgh[++cnt] = 0;
        for (int x = 0; x != v; --tp) {
          x = stk[tp];
          T[cnt].push_back(x);
          T[x].push_back(cnt);
          ++wgh[cnt];
        }
```

```
        T[cnt].push_back(u);
        T[u].push_back(cnt);
        ++wgh[cnt];
      }
    } else
      low[u] = std::min(low[u], dfn[v]);
  }
}
```

1.14　组合数与逆元预处理

用途：用 Pascal 递推预处理组合数，用线性递推预处理模逆元。

```
for (int i = 0; i <= n; i++) {
  c[i][0] = 1;
  c[i][i] = 1;
  for (int j = 1; j <= n; j++) {
    c[i][j] = c[i - 1][j] + c[i - 1][j - 1];
    c[i][j] %= mod;
  }
}

inv[1] = 1;
for (int i = 2; i <= n; i++) {
  inv[i] = mod - (mod / i * inv[mod % i]) % mod;
}
```

1.15　布尔矩阵乘法辅助生成器

用途：对布尔矩阵做传递闭包，并自动生成矩阵乘法所需的变量和转移表达式。

```
void show(auto f) {
  int n = f.size();
  for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
        f[i][j] |= f[i][k] & f[k][j];
      }
    }
  }
  string all = "int ";
  for (int i = 0; i < n; i++) {
    all += "x0" + to_string(i) + ",";
  }
  all.pop_back();
  all += ";";
  cout << all << "\n";
  cout << "************************************************\n";
  string sum = "int ";
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      if (f[i][j]) {
        sum += "x" + to_string(i) + to_string(j) + "=0,";
      }
    }
  }
  sum.pop_back();
  sum += ";";
  cout << sum << "\n";
```

```
  cout << "****************************************\n";
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {

      string ss = "u.x" + to_string(i) + to_string(j) + "=";
      int flag = 0;
      for (int k = 0; k < n; k++) {
        if (f[i][k] && f[k][j]) {
          flag = 1;
          ss += "a.x" + to_string(i) + to_string(k) + "*" + "b.x" +
                to_string(k) + to_string(j) + "+";
        }
      }
      if (ss.size() && flag) {
        ss.pop_back();
        ss += ";";
        cout << ss << " ";
        cout << "\n";
      }
    }
  }
  cout << "********************************\n";
  for (int j = 0; j < n; j++) {
    string s;
    s = "u.x0" + to_string(j) + '=';
    int flag = 0;
    for (int i = 0; i < n; i++) {
      if (f[i][j]) {
        flag = 1;
        s += "a.x0" + to_string(i) + "*" + "b.x" + to_string(i) + to_string(j) +
             "+";
      }
    }
    if (flag) {
      s.pop_back();
      s += ";";
      cout << s << "\n";
    }
  }
}
```

1.16 日期与星期

用途：判断闰年、月份天数，并把日期转换为星期。

```
const int d[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
bool isLeap(int y) { return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0); }
int daysInMonth(int y, int m) { return d[m - 1] + (isLeap(y) && m == 2); }

int getDay(int y, int m, int d) {
  int ans = 0;
  for (int i = 1970; i < y; i++) {
    ans += 365 + isLeap(i);
  }
  for (int i = 1; i < m; i++) {
    ans += daysInMonth(y, i);
  }
  ans += d;
  return (ans + 2) % 7 + 1;
}
```

1.17　手写哈希表

用途：用数组和链式冲突实现低常数哈希表，并支持遍历版本。

```
/*
htb<int,int,(int)1e7+1>mp;
mt19937_64 rng(time(0));
void slove()
{
    int n = 1e7;
    int ans = 0;
    for (int i = 1; i <= n; i++)
    {
        int x = rng();
        ans+=mp[x]++;
    }
    cout << ans << "\n";
} // 牛客测试 900ms 左右

  htb<int,int,(int)4e6>mp;
void slove()
{
    int n = 2e6;
    int ans = 0;
    for (int i = 1; i <= n; i++)
    {
        int x = rng();
        ans+=mp[x]++;
    }
    cout << ans << "\n";
} // 牛客测试 150ms 左右
*/

template <typename T, typename V, int N> struct htb {
  int h[N], to[N], idx = 0;
  int sz = 0;
  T k[N];
  V w[N];
  static int hashx(unsigned long long v) {
    v ^= v << 13;
    v ^= v >> 7;
    return (v ^ (v << 17)) % N;
  }
  bool count(T x) {
    for (int i = h[hashx(x)]; i; i = to[i]) {
      if (k[i] == x)
        return true;
    }
    return false;
  }
  V &operator[](T x) {
    int &p = h[hashx(x)];
    for (int i = p; i; i = to[i]) {
      if (k[i] == x)
        return w[i];
    }
    k[++idx] = x;
    to[idx] = p;
    w[p = idx] = 0;
    sz++;
    return w[idx];
  }
```

```
  bool erase(T x) {
    int prev = 0;
    for (int i = h[hashx(x)]; i; i = to[i]) {
      if (k[i] == x) {
        if (prev == 0)
          h[hashx(x)] = to[i];
        else
          to[prev] = to[i];
        sz--;
        return true;
      }
      prev = i;
      i = to[i];
    }
    return false;
  }
  void clear() {
    while (idx)
      h[hashx(k[idx--])] = 0;
    sz = idx = 0;
  }
  int size() { return sz; }
};

template <typename T, typename V, int N> struct htb {
  int h[N], to[N], idx = 0;
  int sz = 0;
  T k[N];
  V w[N];
  int head = 0, tail = 0;
  int prev[N], next[N];
  static int hashx(unsigned long long v) {
    v ^= v << 13;
    v ^= v >> 7;
    return (v ^ (v << 17)) % N;
  }
  bool count(T x) {
    for (int i = h[hashx(x)]; i; i = to[i]) {
      if (k[i] == x)
        return true;
    }
    return false;
  }

  V &operator[](T x) {
    int &p = h[hashx(x)];
    for (int i = p; i; i = to[i]) {
      if (k[i] == x)
        return w[i];
    }
    k[++idx] = x;
    to[idx] = p;
    w[p = idx] = 0;
    sz++;

    prev[idx] = tail;
    next[idx] = 0;
    if (tail)
      next[tail] = idx;
    else
      head = idx;
    tail = idx;
    return w[idx];
  }
```

```
  bool erase(T x) {
    int prev_node = 0;
    int hash_val = hashx(x);
    for (int i = h[hash_val]; i; i = to[i]) {
      if (k[i] == x) {

        if (prev_node == 0)
          h[hash_val] = to[i];
        else
          to[prev_node] = to[i];

        if (prev[i])
          next[prev[i]] = next[i];
        else
          head = next[i];

        if (next[i])
          prev[next[i]] = prev[i];
        else
          tail = prev[i];
        sz--;
        return true;
      }
      prev_node = i;
    }
    return false;
  }
  void clear() {
    while (idx)
      h[hashx(k[idx--])] = 0;
    sz = idx = 0;
    head = tail = 0;
  }
  int size() { return sz; }
  std::vector<std::pair<T, V>> to_vector() {
    std::vector<std::pair<T, V>> res;
    res.reserve(sz);
    for (int i = head; i != 0; i = next[i]) {
      res.emplace_back(k[i], w[i]);
    }
    return res;
  }
};
```

1.18　Python 速查

用途： Python 竞赛常用导入、组合排列、字典和高精度小数操作。

```
import itertools
from itertools import combinations,permutations
from collections import defaultdict
from more_itertools  import distinct_permutations,distinct_combinations
from collections import deque
from heapq import heappush,heappop
import sys
from more_itertools.more import distinct_permutations
from decimal import Decimal,getcontext
input=lambda :sys.stdin.readline().strip("\n\r")
arr=[1,1,2,3]
for  x in distinct_combinations(arr,2):
    print(x)
```

```
print("#########")
for x in combinations(arr,2):
    print(x)

print("#########")
for x in distinct_permutations(arr):
    print(x)

print("#########")
for x in permutations(arr):
    print(x)
dic=dict()
dic['2']=1
print(dic.get('2',None))
print(dic.get('1',None))
getcontext().prec = 450
a=Decimal(2)
b=Decimal(3)
print(a/b)
print(str(a/b))
#ord 函数可以将一个字符转换成其对应的 ASCII 码值，而 chr 函数则是将一个数字（ASCII 码值）转换成对应的字符。
print(ord('a'))
```

第 2 章

数学与数论

> 组合、数论、代数、变换和数值计算模板。

2.1 组合恒等式

用途： 常用组合恒等式、第二类斯特林数和二项式求和公式。

2.1.1 第二类斯特林数

用途： 计算把 n 个不同元素放入 m 个非空相同盒子的方案数。

$$S(n,m)=\frac{1}{m!}\sum_{k=0}^{m}(-1)^k\binom{m}{k}(m-k)^n.$$

递推式为：

$$S(n,m)=S(n-1,m-1)+mS(n-1,m).$$

2.1.2 二项式求和

用途： 将部分二项式和转成递推，避免每次重新求和。

设 $f_i=\sum_{j=0}^{A}\binom{i}{j}$，则

$$f_i=2f_{i-1}-\binom{i-1}{A}.$$

2.1.3 常用恒等式

用途： 组合计数、前缀化简和二项式展开中的直接结论。

1. $\sum_{i=q}^{n}\binom{i}{q}=\binom{n+1}{q+1}$。
2. $(-1)^m\binom{n-1}{m}=\sum_{i=0}^{m}(-1)^i\binom{n}{i}$。
3. $\sum_{i=0}^{n}\binom{n}{i}i=n2^{n-1}$。
4. $\sum_{i=0}^{n}\binom{n}{i}i^2=n(n+1)2^{n-2}$。
5. $(x+y)^n=\sum_{i=0}^{n}\binom{n}{i}x^iy^{n-i}$。

2.2 拉格朗日插值

用途： 已知若干个横坐标互不相同的点，求经过这些点的唯一低次多项式，或只求该多项式在某个位置的值。给定 m 个点时，插值多项式的次数至多为 $m-1$。

2.2.1 基本公式

给定 m 个点

$$(x_0, y_0), (x_1, y_1), \dots, (x_{m-1}, y_{m-1}),$$

其中任意 $i \neq j$ 都有 $x_i \neq x_j$。定义第 i 个拉格朗日基函数

$$L_i(x) = \prod_{\substack{0 \le j < m \\ j \neq i}} \frac{x - x_j}{x_i - x_j}.$$

它满足

$$L_i(x_i) = 1, \qquad L_i(x_j) = 0 \quad (j \neq i).$$

因此插值多项式为

$$P(x) = \sum_{i=0}^{m-1} y_i L_i(x) = \sum_{i=0}^{m-1} y_i \prod_{\substack{0 \le j < m \\ j \neq i}} \frac{x - x_j}{x_i - x_j}.$$

需要特别注意：分子是 $x - x_j$，分母是 $x_i - x_j$，二者不能颠倒。

2.2.2 正确性与唯一性

把 $x = x_q$ 代入上式。除第 q 项外，每一项的分子都包含 $x_q - x_q = 0$；第 q 项的基函数值为 1，所以 $P(x_q) = y_q$。

另一方面，假设有两个次数至多为 $m-1$ 的多项式都经过这 m 个点，那么它们的差在 $x_0, x_1, \dots, x_{m-1}$ 处都有根。一个非零的 $m-1$ 次多项式不可能拥有 m 个不同的根，因此二者之差只能是零多项式。这同时证明了插值多项式的唯一性。

2.2.3 三点插值示例

已知三个点 $(1,3), (2,7), (3,13)$，则

$$P(x) = 3\frac{(x-2)(x-3)}{(1-2)(1-3)} + 7\frac{(x-1)(x-3)}{(2-1)(2-3)} + 13\frac{(x-1)(x-2)}{(3-1)(3-2)}.$$

化简得到

$$P(x) = x^2 + x + 1,$$

例如 $P(4) = 21$。实际做题时通常不需要展开系数，直接代入插值公式即可。

2.2.4 任意节点：单次查询 $O(m^2)$

直接按照定义计算每一个 $L_i(k)$，即可在 $O(m^2)$ 次模乘内求出 $P(k)$。在模质数 MOD 下，除法改为乘法逆元，并且必须保证所有 $x_i - x_j$ 在模 MOD 意义下非零。

```
using i64 = long long;

i64 mod_pow(i64 a, i64 e, i64 mod) {
  i64 r = 1 % mod;
  while (e) {
    if (e & 1) r = (__int128)r * a % mod;
    a = (__int128)a * a % mod;
    e >>= 1;
  }
  return r;
}

// MOD 为质数，且 x[i] 在模 MOD 意义下两两不同。
i64 lagrange_direct(const vector<i64> &x,
                    const vector<i64> &y,
                    i64 k, i64 MOD) {
  auto norm = [&](i64 v) {
    v %= MOD;
    if (v < 0) v += MOD;
    return v;
  };

  const int m = (int)x.size();
  k = norm(k);
  for (int i = 0; i < m; ++i) {
    if (k == norm(x[i])) return norm(y[i]);
  }

  i64 ans = 0;
  for (int i = 0; i < m; ++i) {
    i64 numerator = 1;
    i64 denominator = 1;
    for (int j = 0; j < m; ++j) {
      if (i == j) continue;
      numerator = (__int128)numerator * norm(k - norm(x[j])) % MOD;
      denominator = (__int128)denominator
                    * norm(norm(x[i]) - norm(x[j])) % MOD;
    }
    i64 term = (__int128)norm(y[i]) * numerator % MOD;
    term = (__int128)term * mod_pow(denominator, MOD - 2, MOD) % MOD;
    ans += term;
    if (ans >= MOD) ans -= MOD;
  }
  return ans;
}
```

上面的实现还对每个分母单独进行一次快速幂，严格复杂度为 $O(m^2 + m \log \mathrm{MOD})$。如果使用批量求逆，可以把求逆部分降到 $O(m + \log \mathrm{MOD})$。

2.2.5 重心形式：预处理后快速查询

定义重心权重

$$w_i = \frac{1}{\prod_{j \neq i} (x_i - x_j)}.$$

先用 $O(m^2)$ 预处理所有 w_i。当查询点 k 不等于任何 x_i 时，第二重心公式为

$$P(k)=\frac{\sum_{i=0}^{m-1}\frac{w_i y_i}{k-x_i}}{\sum_{i=0}^{m-1}\frac{w_i}{k-x_i}}.$$

预处理之后，每次查询只需 $O(m)$ 次数域运算。若 $k=x_q$，必须直接返回 y_q，不能代入含有零分母的重心公式。在浮点数场景中，第二重心形式通常也比直接展开拉格朗日多项式更加稳定。

如果动态加入新节点 x_m，不仅要计算新权重，原有权重也要更新：

$$w_i \leftarrow \frac{w_i}{x_i-x_m}\quad (0\le i<m),\qquad w_m=\frac{1}{\prod_{j=0}^{m-1}(x_m-x_j)}.$$

因此一次插入的代价是 $O(m)$，不能只计算新节点的权重而保持旧权重不变。

2.2.6 连续节点：单次查询 $O(n)$

竞赛中常见的特殊情况是已知

$$P(0)=y_0,P(1)=y_1,\dots,P(n)=y_n.$$

此时横坐标连续，并且分母可以化成阶乘：

$$\prod_{\substack{0\le j\le n\\ j\ne i}}(i-j)=i!(-1)^{n-i}(n-i)!.$$

对于查询点 k，定义前缀积与后缀积

$$\mathrm{pre}[q]=\prod_{j=0}^{q-1}(k-j),\qquad \mathrm{suf}[q]=\prod_{j=q}^{n}(k-j).$$

那么排除因子 $k-i$ 后的分子就是 $\mathrm{pre}[i]\,\mathrm{suf}[i+1]$，从而

$$P(k)=\sum_{i=0}^{n} y_i\,\mathrm{pre}[i]\,\mathrm{suf}[i+1]\frac{(-1)^{n-i}}{i!(n-i)!}.$$

预处理阶乘、逆阶乘、前缀积和后缀积后，时间和空间复杂度均为 $O(n)$。

```cpp
// MOD 为质数，n < MOD，y[i] = P(i)，0 <= i <= n。
long long lagrange_consecutive(const vector<long long> &y,
                               long long k,
                               long long MOD) {
  const int n = (int)y.size() - 1;
  auto norm = [&](long long v) {
    v %= MOD;
    if (v < 0) v += MOD;
    return v;
  };

  k = norm(k);
  // 节点位于 0..n；在有限域中只需比较归一化后的 k。
```

```
  if (k <= n) return norm(y[(int)k]);

  vector<long long> fac(n + 1), ifac(n + 1);
  fac[0] = 1;
  for (int i = 1; i <= n; ++i)
    fac[i] = (__int128)fac[i - 1] * i % MOD;

  ifac[n] = mod_pow(fac[n], MOD - 2, MOD);
  for (int i = n; i >= 1; --i)
    ifac[i - 1] = (__int128)ifac[i] * i % MOD;

  vector<long long> pre(n + 2, 1), suf(n + 2, 1);
  for (int i = 0; i <= n; ++i)
    pre[i + 1] = (__int128)pre[i] * norm(k - i) % MOD;
  for (int i = n; i >= 0; --i)
    suf[i] = (__int128)suf[i + 1] * norm(k - i) % MOD;

  long long ans = 0;
  for (int i = 0; i <= n; ++i) {
    long long term = norm(y[i]);
    term = (__int128)term * pre[i] % MOD;
    term = (__int128)term * suf[i + 1] % MOD;
    term = (__int128)term * ifac[i] % MOD;
    term = (__int128)term * ifac[n - i] % MOD;
    if ((n - i) & 1) term = (MOD - term) % MOD;
    ans += term;
    if (ans >= MOD) ans -= MOD;
  }
  return ans;
}
```

2.2.7 经典应用：高次幂前缀和

固定非负整数 d，定义

$$S_d(N) = \sum_{i=1}^{N} i^d.$$

由

$$S_d(N) - S_d(N-1) = N^d$$

可知 $S_d(N)$ 是关于 N 的次数至多为 $d+1$ 的多项式，即 Faulhaber 多项式。因此只需计算它在 $0, 1, \dots, d+1$ 处的值，就可以通过连续节点插值求出极大 N 处的值。准备样本后，单次插值复杂度为 $O(d)$。

更一般地，判断一道题能否使用插值时，可以寻找以下结构：

1. 目标量关于某个变量是次数有上界的多项式。
2. 能够低成本计算足够多个不同位置上的函数值。
3. 差分会降低次数：若 $\Delta F(n) = F(n) - F(n-1)$ 是 d 次多项式，则 $F(n)$ 的次数至多为 $d+1$。

2.2.8 易错点

- m 个横坐标不同的点确定唯一的至多 $m-1$ 次多项式，不是 m 次。
- 模运算中要求横坐标在模 `MOD` 意义下两两不同，而不只是在整数意义下不同。
- 查询点等于某个节点时直接返回已知值，避免制造零分母。
- 连续节点分母的符号是 $(-1)^{n-i}$；漏掉这个符号会使约一半项出错。
- 使用费马小定理求逆元要求 `MOD` 为质数且分母非零；合数模数不能直接使用 `a^(MOD-2)`。

- 插值只能恢复次数上界范围内的多项式。套用模板前，必须先证明目标量确实是多项式并确定需要多少个采样点。

2.3 二进制 GCD

用途：只使用移位、减法和计数尾零计算 gcd，避免除法。

```
int gcd(int a, int b) {
  if (b == 0)
    return abs(a);
  if (a == 0)
    return abs(b);
  int az = __builtin_ctzll(a);
  int bz = __builtin_ctzll(b);
  int z = std::min(az, bz);
  a >>= az, b >>= bz;
  while (a != b) {
    int diff = b - a;
    az = __builtin_ctzll(diff);
    b = b < a ? b : a;
    a = abs(diff) >> az;
  }
  return a << z;
}
```

2.4 扩展中国剩余定理（EXCRT）

用途：求解模数不互质时的扩展中国剩余定理，并返回最小非负解。

```
// 要求 c%gcd(a,b)==0 才成立
int exgcd(int a, int b, int &x, int &y) {
  if (!b) {
    x = 1, y = 0;
    return a;
  }
  int g = exgcd(b, a % b, y, x);
  y -= a / b * x;
  return g;
}
int excrt(const vector<int> &r, const vector<int> &m) {
  // 对于所有 i, j，都满足 (r_i - r_j) % gcd (m_i, m_j) == 0 才成立
  int n = r.size();
  int r0 = 0, m0 = 1;
  for (int i = 0; i < n; i++) {
    int m1 = m[i];
    int r1 = r[i] % m1;
    if (r1 < 0) {
      r1 += m1;
    }
    if (m0 < m1) {
      swap(r0, r1);
      swap(m0, m1);
    }
    if (!(m0 % m1)) {
      if (r0 % m1 != r1) {
        return -1;
      }
      continue;
```

```
    }
    int x, y;
    int g = exgcd(m0, m1, x, y);
    if ((r1 - r0) % g) {
      return -1;
    }
    int u1 = m1 / g;
    r0 += (r1 - r0) / g % u1 * x % u1 * m0;
    m0 *= u1;
    if (r0 < 0) {
      r0 += m0;
    }
  }
  // 通解 r0+k*m0
  // m0 所有模数最小公倍数
  return r0;
}
```

2.5 值域 GCD 预处理

用途：预处理小质因子和小范围 gcd，处理大量值域 gcd 查询。

```
struct Rangegcd {
  vector<int> prime, minp;
  vector<bool> book; // 这里面的实际存的的 bit 类型
  void initprime(int n) {
    book.resize(n + 10);
    minp.resize(n + 10);
    for (int i = 2; i <= n; i++) {
      if (!book[i]) {
        prime.push_back(i);
        minp[i] = i;
      }
      for (auto &p : prime) {
        if (1ll * i * p > n)
          break;
        book[i * p] = 1;
        minp[i * p] = p;
        if (i % p == 0)
          break;
      }
    }
  }
  vector<vector<int>> g;
  vector<array<int, 3>> ff;
  int v;
  void init(int n) {
    v = sqrt(n);
    v++;
    initprime(n);
    ff.resize(n + 1);
    g.resize(v + 1);
    for (int i = 0; i <= v; i++)
      g[i].resize(v + 1);
    for (int i = 0; i <= v; i++)
      g[0][i] = g[i][0] = i;
    for (int i = 1; i <= v; i++) {
      for (int j = 1; j <= i; j++) {
        g[i][j] = g[j][i] = g[i - j][j];
      }
    }
```

```
    for (int i = 1; i <= n; i++) {
      if (!book[i]) {
        ff[i][0] = ff[i][1] = 1;
        ff[i][2] = i;
      } else {
        int p = minp[i];
        for (int j = 0; j <= 2; j++)
          ff[i][j] = ff[i / p][j];
        ff[i][0] *= p;
        if (ff[i][0] > ff[i][1])
          swap(ff[i][0], ff[i][1]);
        if (ff[i][1] > ff[i][2])
          swap(ff[i][1], ff[i][2]);
      }
    }
  }
  int mgg(int x, int &y) {
    int d = (x > v) ? (y % x == 0 ? x : 1) : g[x][y % x];
    y /= d;
    return d;
  }
  int gcd(int x, int y) {
    return mgg(ff[x][0], y) * mgg(ff[x][1], y) * mgg(ff[x][2], y);
  }
} rangegcd;
```

2.6 线性递推求逆元

```
   for(int i=2;i<=n;i++)
       inv[i]=(ll)p-(p/i)*inv[p%i]%p,printf("%d\n",inv[i]);
```

2.7 大整数质因数分解

用途：使用快速模乘、Miller–Rabin 与 Pollard–Rho 分解大整数。

```
// n=1e5  值域 1e9  牛客耗时 700+ms
// n=1e5  值域 1e10  牛客耗时 800+ms
// n=1e4 值域 1e18  牛客耗时  260+ms
// n=5e4  值域 1e12  牛客耗时 500+ms
mt19937_64 rng(time(0));
struct Factor {
  using f64 = long double;
  // mt19937_64 rng(time(0));
  f64 iv;
  int m;
  void setmod(int mod) {
    m = mod;
    iv = (f64)1 / m;
  }
  int gcd(int a, int b) {
    if (b == 0)
      return abs(a);
    if (a == 0)
      return abs(b);
    int az = __builtin_ctzll(a);
    int bz = __builtin_ctzll(b);
    int z = std::min(az, bz);
    a >>= az, b >>= bz;
```

```
  while (a != b) {
    int diff = b - a;
    az = __builtin_ctzll(diff);
    b = b < a ? b : a;
    a = abs(diff) >> az;
  }
  return a << z;
}
int mul(int a, int b) {
  int r = a * b - (int)(a * iv * b - 0.5) * m;
  return r < m ? r : r - m;
}
int add(int a, int b) { return a += b, a >= m ? a - m : a; }
int qpow(int a, int b) {
  int ans = 1;
  while (b) {
    if (b & 1)
      ans = mul(ans, a);
    a = mul(a, a);
    b >>= 1;
  }
  return ans;
}
bool isprime(int n) {
  if (n <= 2 || (!(n & 1)))
    return n == 2;
  int t = __builtin_ctzll(n - 1);
  int u = (n - 1) >> t;
  setmod(n);
  for (int i = 0; i < 8; i++) {
    int v = qpow(rng() % (n - 2) + 2, u);
    if (v == 1 || v == n - 1)
      continue;
    for (int j = 1; j <= t; j++) {
      v = mul(v, v);
      if (v == n - 1)
        break;
      if (j == t)
        return false;
    }
  }
  return true;
}
int rho(int n) {
  if (!(n & 1))
    return 2;
  if (n % 3 == 0)
    return 3;
  if (n % 5 == 0)
    return 5;
  if (n % 7 == 0)
    return 7;
  int s = 0, t = 0, c = rng() % (n - 1) + 1;
  for (int i = 1;; i <<= 1) {
    int v = 1;
    s = t;
    for (int j = 1; j <= i; j++) {
      t = mul(t, t);
      t = add(t, c);
      v = mul(v, abs(t - s));
      if (j == i || ((j & 127) == 0)) {
        int d = gcd(n, v);
        if (d > 1)
          return d;
```

```
        }
      }
    }
    return 0;
  }
  vector<int> getprime(int n) {
    vector<int> ans;
    auto f = [&](auto f, int n) {
      if (n == 1)
        return;
      if (isprime(n))
        return ans.push_back(n), void();
      int m = rho(n);
      f(f, m);
      f(f, n / m);
    };
    f(f, n);
    sort(begin(ans), end(ans));
    return ans;
  }
} factor;

// n=1e5  值域 1e9  牛客耗时 700+ms
// n=1e5  值域 1e10  牛客耗时 800+ms
// n=1e4 值域 1e18  牛客耗时  260+ms
// n=5e4  值域 1e12  牛客耗时 500+ms
```

2.8 组合数预处理 (卢卡斯定理)

用途：预处理组合数、逆元或阶乘逆元，服务于模意义下的组合计算。

```
struct Comb {
  vector<int> f, fac, inv;
  int ksm(int a, int b) {
    int ans = 1;
    a %= mod;
    while (b) {
      if (b & 1)
        ans = ans * a % mod;
      b >>= 1;
      a = a * a % mod;
    }
    return ans;
  }
  void initC(int n) {
    f.resize(n + 10);
    fac.resize(n + 10);
    f[0] = fac[0] = 1;
    for (int i = 1; i <= n; i++) {
      f[i] = f[i - 1] * i;
    }
    fac[n] = ksm(f[n], mod - 2);
    for (int i = n - 1; i >= 1; i--) {
      fac[i] = fac[i + 1] * (i + 1) % mod;
    }
  }
  void initinv(int n) {
    inv.resize(n + 10);
    inv[1] = 1;
    for (int i = 2; i <= n; i++) {
      inv[i] = mod - (mod / i * inv[mod % i]) % mod;
```

```
    }
  }
  int C(int n, int m) {
    assert(n >= 0);
    if (n < m)
      return 0;
    return f[n] * fac[m] % mod * fac[n - m] % mod;
  }
  int Lucas(int n, int m) {
    if (m == 0)
      return 1; // mod*logp(m)*log2(mod)  mod 为素数 1e5 以内大致
    return (C(n % mod, m % mod) * Lucas(n / mod, m / mod)) % mod;
  }

} comb;
```

2.9　去除素因子后的阶乘模

用途： 计算从 $n!$ 中完全移除素因子 p 后的单位部分。令

$$e = \nu_p(n!) = \sum_{j\ge 1} \left\lfloor \frac{n}{p^j} \right\rfloor, \qquad U_p(n!) = \frac{n!}{p^e}.$$

下面第一个重载返回 $U_p(n!) \bmod p^a$。要求 p 为素数、pa = p^a、$a \ge 1$ 且 $n \ge 0$。

```
int factmod(long long n, int p, int pa) {
  // f[i] 是 1..i 中所有非 p 倍数之积，模 pa。
  vector<int> f(pa);
  f[0] = 1;
  for (int i = 1; i < pa; ++i) {
    f[i] = (i % p == 0)
               ? f[i - 1]
               : (long long)f[i - 1] * i % pa;
  }

  // 一个完整块的单位乘积：奇素数幂以及模 2、4 时为 -1；
  // 模 2^a 且 a >= 3 时为 +1。
  const bool negate_full_block = (p != 2 || pa <= 4);
  int res = 1;
  while (n > 1) {
    if (negate_full_block && ((n / pa) & 1)) {
      res = pa - res;
    }
    res = (long long)res * f[n % pa] % pa;
    n /= p;
  }
  return res;
}
```

只需要模 p 时，可以使用更短的特化版本；其中整块乘积由 Wilson 定理化为 $-1 \pmod p$，$p = 2$ 时同样成立。

```
int factmod(long long n, int p) {
  vector<int> f(p);
  f[0] = 1;
  for (int i = 1; i < p; ++i) {
    f[i] = (long long)f[i - 1] * i % p;
  }
```

```
  int res = 1;
  while (n > 1) {
    if ((n / p) & 1) {
      res = p - res;
    }
    res = (long long)res * f[n % p] % p;
    n /= p;
  }
  return res;
}
```

每次令 $n \leftarrow \lfloor n/p \rfloor$，是在继续处理原阶乘中含有 $p, p^2, \ldots$ 的项剥去一层 p 后留下的单位部分。三个参数的版本预处理时间、空间分别为 $O(p^a)$、$O(p^a)$，单次递推为 $O(\log_p n)$；模 p 版本的预处理降为 $O(p)$。同一模数有多次查询时，应把数组 `f` 缓存起来。

该函数只返回单位部分，并不返回指数 e。计算组合数模素数幂时，还需要单独用 Legendre 公式求指数并与单位部分合并。若 `p` 不是素数，或 `pa` 不是 `p` 的正整数幂，这两个模板都不适用。

2.10 约数函数

用途：由质因数分解得到约数个数、约数和，并给出线性筛预处理。

若

$$N = \prod_{i=1}^{k} p_i^{a_i},$$

则约数个数为

$$d(N) = \prod_{i=1}^{k} (a_i + 1),$$

约数和为

$$\sigma(N) = \prod_{i=1}^{k} \left(1 + p_i + p_i^2 + \cdots + p_i^{a_i}\right).$$

```
// 求 (1-n) 每一个数约数的个数
const int N = 1e7;
int d[N];
int book[N];
int arr[N];
int num[N];
int cnt;
void f() {
  d[1] = 1;
  for (int i = 2; i <= 1e6; i++) {
    if (!book[i]) {
      arr[++cnt] = i, d[i] = 2, num[i] = 1;
    }
    for (int j = 1; j <= cnt && 1ll * i * arr[j] <= 1e6; j++) {
      book[i * arr[j]] = 1;
      if (i % arr[j] == 0) {
        num[i * arr[j]] = num[i] + 1;
        d[i * arr[j]] = d[i] / (num[i] + 1) * (num[i * arr[j]] + 1);
        break;
      }
      num[i * arr[j]] = 1;
      d[i * arr[j]] = d[i] * d[arr[j]];
    }
  }
}
```

```
// 求 (1 - n) 每一个数所有约数的和 const int N = 1e5 + 5;
bool book[N];
int arr[N];
long long sp[N], sum[N];
int cnt;
void f() {
  cnt = 0;
  sp[1] = 1;
  sum[1] = 1;
  for (int i = 2; i < N; ++i) {
    if (!book[i]) {
      arr[cnt++] = i;
      sp[i] = i + 1;
      sum[i] = i + 1;
    }
    for (int j = 0; j < cnt && i * arr[j] < N; ++j) {
      book[i * arr[j]] = 1;
      if (!(i % arr[j])) {
        sp[i * arr[j]] = sp[i] * arr[j] + 1;
        sum[i * arr[j]] = sum[i] / sp[i] * sp[i * arr[j]];
        break;
      }
      sum[i * arr[j]] = sum[i] * sum[arr[j]];
      sp[i * arr[j]] = 1 + arr[j];
    }
  }
}
```

2.11　离散对数（BSGS / EXBSGS）

用途： BSGS 求离散对数；EXBSGS 进一步处理 gcd 不为 1 的情况。

```
auto getord = [&](int x) {
  int ans = p - 1;
  // d 存放 p - 1 的质因子，p 为质数。
  for (auto it : d) {
    while (ans % it == 0 && ksm(x % p, ans / it, p) == 1)
      ans /= it;
  }
  return ans;
};
```

BSGS 的目标是给定 a, b, p，求最小非负整数 x，满足

$$a^x \equiv b \pmod{p}.$$

通常要求 $\gcd(a, p) = 1$；EXBSGS 放宽这一条件，并在约分过程中检查无解情况。

```
int exgcd(int a, int b, int &x, int &y) {
  if (!b)
    return x = 1, y = 0, a;
  int d = exgcd(b, a % b, y, x);
  y -= a / b * x;
  return d;
}
int inv(int v, int p) {
  int x, y;
  exgcd(v, p, x, y);
  return (x % p + p) % p;
```

```
}
int bsgs(int a, int p, int b) {
  a %= p, b %= p;
  if (b == 1 || p == 1)
    return 0;
  map<int, int> mp;
  int B = ceil(sqrt(p));
  ll s = 1;
  for (int i = 0; i < B; i++, s = s * a % p) {
    if (s == b)
      return i;
    mp[s * b % p] = i;
  }
  ll t = s;
  for (int i = 1; i <= B + 1; i++, s = s * t % p)
    if (mp.count(s))
      return i * B - mp[s];
  return -1;
}
int exbsgs(int a, int p, int b) {
  a %= p, b %= p;
  if (b == 1 || p == 1)
    return 0;
  int cnt = 0;
  int s = 1;
  while (1) {
    int d = __gcd(a, p);
    if (d == 1)
      break;
    if (b % d)
      return -1;
    b /= d, p /= d;
    cnt++, s = s * (a / d) % p;
    if (s == b)
      return cnt;
  }
  b = inv(s, p) * b % p;
  ll ret = bsgs(a, p, b);
  return ret == -1 ? -1 : ret + cnt;
}
```

2.12 欧拉函数

用途： 计算 φ(n) 及其积性、求和等常用性质。

若 $p_1, p_2, \ldots, p_k$ 是 n 的不同质因子，则

$$\varphi(n) = n \prod_{i=1}^{k} \left(1 - \frac{1}{p_i}\right).$$

$\varphi(n)$ 表示 $1 \le x \le n$ 且 $\gcd(x, n) = 1$ 的整数个数，并且

$$\sum_{d|n} \varphi(d) = n.$$

例如，数列 a 的 gcd 总和可以写成

$$\sum_{i=1}^{n} \sum_{j=1}^{n} \gcd(a_i, a_j) = \sum_{i=1}^{n} \sum_{j=1}^{n} \sum_{d|a_i,\ d|a_j} \varphi(d).$$

2.12.1 常用性质

1. 若 p 为质数，则 $\varphi(p) = p - 1$。
2. 若 p 为质数，则 $\varphi(p^k) = p^k - p^{k-1}$。
3. 当 $n > 2$ 时，与 n 互质的数可以按 x 与 $n - x$ 成对出现，因此 $\varphi(n)$ 为偶数。
4. 当 $n > 1$ 时，所有不超过 n 且与 n 互质的数之和为 $\varphi(n)n/2$。

Euler 定理：若 $\gcd(a, m) = 1$，则

$$a^{\varphi(m)} \equiv 1 \pmod{m}.$$

在扩展欧拉降幂中，常用如下分情况表达式：

$$a^c \equiv \begin{cases} a^{c \bmod \varphi(m)}, & \gcd(a,m) = 1, \\ a^{c \bmod \varphi(m)} + \varphi(m), & \gcd(a,m) \neq 1,\ c < \varphi(m), \\ a^{(c \bmod \varphi(m)) + \varphi(m)}, & \gcd(a,m) \neq 1,\ c \geq \varphi(m). \end{cases}$$

```cpp
// 质数筛求欧拉函数
const int N = 1e7;
int arr[N];
int book[N];
int phi[N];
int cnt;
phi[1] = 1;
for (int i = 2; i <= 1e6; i++) {
  if (!book[i])
    arr[++cnt] = i, phi[i] = i - 1;
  for (int j = 1; j <= cnt && 1ll * i * arr[j] <= 1e6; j++) {
    book[i * arr[j]] = 1;
    if (i % arr[j] == 0) {
      phi[i * arr[j]] = arr[j] * phi[i];
      break;
    }
    phi[i * arr[j]] = phi[i] * phi[arr[j]];
  }
}
int phi(int n) {
  int res = n;
  for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) {
      while (n % i == 0) {
        n /= i;
      }
      res = res / i * (i - 1);
    }
  }
  if (n > 1) {
    res = res / n * (n - 1);
  }
  return res;
}
```

2.13 莫比乌斯函数

用途：线性筛计算 μ(n)，用于莫比乌斯反演和整除卷积。

```
int arr[N];
int book[N];
int mu[N];
void f() {
  int cnt = 0;
  mu[1] = 1;
  for (int i = 2; i <= 1e6; i++) {
    if (!book[i])
      arr[++cnt] = i, mu[i] = -1;
    for (int j = 1; j <= cnt && i * arr[j] <= 1e6; j++) {
      book[i * arr[j]] = 1;
      if (i % arr[j] == 0) {
        mu[i * arr[j]] = 0;
        break;
      }
      mu[i * arr[j]] = -mu[i];
    }
  }
}
```

2.14 矩阵快速幂

用途：用矩阵乘法和快速幂处理线性递推及有限状态转移。

2.14.1 例一：斐波那契数列

$$f(n) = f(n-1) + f(n-2).$$

令 $F(0) = 0, F(1) = 1$，则

$$\begin{bmatrix} F(n+1) & F(n) \end{bmatrix} = \begin{bmatrix} F(1) & F(0) \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n.$$

2.14.2 例二：带多项式项的递推

若

$$f(n) = f(n-1) + 2f(n-2) + n^3,$$

可把状态扩展为

$$[f(n), f(n-1), n^3, n^2, n, 1]^T,$$

再用一个固定矩阵完成一次状态转移。

```
const int M = 101;
array<array<int, M>, M> temp;
struct mat {
  array<array<int, M>, M> a;
  mat(int v) {
    for (int i = 0; i < M; i++)
      a[i].fill(v);
  }
  mat() {
```

```
    for (int i = 0; i < M; i++)
      a[i].fill(0);
    for (int i = 0; i < M; i++)
      a[i][i] = 1;
  }
};
mat mul(mat a, mat b) {
  mat temp(0);
  for (int k = 0; k < M; k++) {
    for (int i = 0; i < M; i++) {
      for (int j = 0; j < M; j++) {
        temp.a[i][j] += a.a[i][k] * b.a[k][j] % mod;
        temp.a[i][j] %= mod;
      }
    }
  }
  return temp;
}
mat ksm(mat a, int b) {
  mat ans;
  while (b) {
    if (b & 1)
      ans = mul(ans, a);
    b >>= 1;
    a = mul(a, a);
  }
  return ans;
}
```

2.15　快速傅里叶变换（FFT）

用途： 快速计算多项式卷积，复杂度 O(N log N)。

```
vector<int> FFT(vector<int> a1, vector<int> b1) {
#define comp complex<double>
  const double PI = acos(-1);
  vector<comp> a(a1.size()), b(b1.size());
  for (int i = 0; i < a.size(); i++) {
    a[i] = a1[i];
  }
  for (int i = 0; i < b.size(); i++) {
    b[i] = b1[i];
  }
  int s = a.size() + b.size() - 1;
  int lim = 1;
  int l = 0;
  while (lim < s + 1)
    lim *= 2, l++;
  vector<int> r(lim);
  a.resize(lim);
  b.resize(lim);
  for (int i = 0; i < lim; i++) {
    r[i] = (r[i >> 1] >> 1) | ((i & 1) << (l - 1));
  }
  auto fft = [&](vector<comp> &a, int type) {
    for (int i = 0; i < lim; i++)
      if (i < r[i])
        swap(a[i], a[r[i]]);
    for (int i = 1; i < lim; i <<= 1) {
      comp x(cos(PI / i), type * sin(PI / i));
      for (int j = 0; j < lim; j += (i << 1)) {
```

```
        comp y(1, 0);
        for (int k = 0; k < i; k++, y *= x) {
          comp p = a[j + k], q = y * a[j + k + i];
          a[j + k] = p + q;
          a[j + k + i] = p - q;
        }
      }
    }
  };
  fft(a, 1);
  fft(b, 1);
  for (int i = 0; i < lim; i++)
    a[i] *= b[i];
  fft(a, -1);
  while (a.size() > s)
    a.pop_back();
  vector<int> ans;
  for (auto it : a) {
    ans.push_back((int)(0.5 + it.real() / lim));
  }
  return ans;
}
```

2.16 Min_25 筛

用途：对积性函数或素数函数做整除分块预处理，常用于大范围求和。

```
int ksm(int a, int b) {
  int ans = 1;
  while (b) {
    if (b & 1)
      ans = ans * a % mod;
    b >>= 1;
    a = a * a % mod;
  }
  return ans;
}
struct Min_25 {
  vector<int> id1, id2, book, arr, sum, w;
  vector<int> g1, g2;
  vector<int> sum1, sum2;
  int sqrn = 0, cnt = 0, m = 0, n = 0;
  int getid(int x) {
    if (x <= sqrn)
      return id1[x];
    else {
      return id2[n / x];
    }
  }
  int f(int x) {
    x %= mod;
    return (x * x % mod - x) % mod;
  }
  void init(int n) {
    sqrn = sqrt(n + 0.5);
    this->n = n;
    book.resize(sqrn + 10);
    arr.resize(sqrn + 10);
    id1.resize(2 * sqrn + 10);
    id2.resize(2 * sqrn + 10);
    w.resize(2 * sqrn + 10);
```

```
  g1.resize(2 * sqrn + 10);
  g2.resize(2 * sqrn + 10);
  sum1.resize(2 * sqrn + 10);
  sum2.resize(2 * sqrn + 10);
  for (int i = 2; i <= sqrn; i++) {
    if (!book[i])
      arr[++cnt] = i;
    for (int j = 1; j <= cnt && i * arr[j] <= sqrn; j++) {
      book[i * arr[j]] = 1;
      if (i % arr[j] == 0)
        break;
    }
  }
  arr[cnt + 1] = 1e9;
  cnt = 1;
  while (arr[cnt] * arr[cnt] <= n)
    cnt++;
  cnt--;
  int r = 1;
  for (int l = 1; l <= n; l = r + 1) {
    r = n / (n / l);
    w[++m] = n / l;
    if (n / l <= sqrn) {
      id1[n / l] = m;
    } else {
      id2[n / (n / l)] = m;
    }
  }
  auto initmg = [&]() {
    int inv6 = ksm(6, mod - 2);
    int inv2 = ksm(2, mod - 2);
    // 注意 g 要忽略 1
    for (int i = 1; i <= m; i++) {
      g1[i] = w[i] % mod * (w[i] % mod + 1) % mod * (2 * w[i] % mod + 1) %
                  mod * inv6 % mod -
              1;
      g2[i] = w[i] % mod * (w[i] % mod + 1) % mod * inv2 % mod - 1;
    }
    for (int i = 1; i <= cnt; i++) {
      sum1[i] = sum1[i - 1] + arr[i] * arr[i] % mod;
      sum2[i] = sum2[i - 1] + arr[i] % mod;
      sum1[i] %= mod;
      sum2[i] %= mod;
    }
  };
  initmg();
  // sum g 记得预处理 sum 和 g
  for (int i = 1; i <= cnt; i++) { // 枚举质数
    for (int j = 1; j <= m && arr[i] * arr[i] <= w[j]; j++) {
      int x = getid(w[j] / arr[i]);
      g1[j] =
          (g1[j] - arr[i] * arr[i] % mod * (g1[x] - sum1[i - 1]) % mod) % mod;
      g2[j] = (g2[j] - arr[i] * (g2[x] - sum2[i - 1]) % mod) % mod;
    }
  }
}
int S(int n, int j) {
  int ans = g1[getid(n)] - sum1[j - 1] - g2[getid(n)] + sum2[j - 1];
  ans %= mod;
  for (int i = j; arr[i] * arr[i] <= n; i++) {
    for (int t = arr[i]; t * arr[i] <= n; t = t * arr[i]) {
      ans += S(n / t, i + 1) * f(t) % mod + f(t * arr[i]);
      ans %= mod;
    }
```

```
    }
    if (ans < 0)
      ans += mod;
    return ans;
  };
} min25;
```

2.17 大范围素数计数

用途：用整除分块和筛法在大范围内统计素数个数。

```
ll f[N], g[N], n;
void init() {
  ll i, j, m;
  for (m = 1; m * m <= n; ++m)
    f[m] = n / m - 1;
  for (i = 1; i <= m; ++i)
    g[i] = i - 1;
  for (i = 2; i <= m; ++i) {
    if (g[i] == g[i - 1])
      continue;
    for (j = 1; j <= min(m - 1, n / i / i); ++j) {
      if (i * j < m)
        f[j] -= f[i * j] - g[i - 1];
      else
        f[j] -= g[n / i / j] - g[i - 1];
    }
    for (j = m; j >= i * i; --j)
      g[j] -= g[j / i] - g[i - 1];
  }
  // f[i] 表示 [2,n/i] 范围内的素数个数注意 f 只有 sqrt(n) 的长度
  // 所以 f[1] 表示 n 以内的素数个数
}
```

2.18 生成 10^9 内素数

用途：分块筛或位图筛生成 10^9 内的全部素数。

```
void init() {
  using ull = unsigned long long;
  int n = 1e9;
  // 1e9 内共 50847534 个质数; 1000ms 左右
  const int sz = 510510;
  vector<int> prime(50847534 + 3);
  vector<int> book(sz + 10);
  vector<ull> bit(sz / 64 + 1);
  int cnt = 0;
  auto set = [&](auto &bit, int pos) {
    pos--;
    bit[pos >> 6] |= (1ull << (pos & 63));
  };
  for (int i = 2; i <= sz; i++) {
    if (!book[i]) {
      prime[++cnt] = i;
      if (cnt <= 7) {
        set(bit, i);
      }
    }
    for (int j = 1; j <= cnt && i * prime[j] <= sz; j++) {
```

```
      book[i * prime[j]] = 1;
      if (j <= 7) {
        set(bit, i * prime[j]);
      }
      if (i % prime[j] == 0)
        break;
    }
  }
  for (int i = 2; i <= (n + sz - 1) / sz; i++) {
    int l = (i - 1) * sz + 1;
    int r = min(i * sz, n);
    vector<ull> tbit = bit;
    for (int j = 8; j <= cnt && prime[j] * prime[j] <= r; j++) {
      int t1 = l / prime[j] * prime[j];
      int t2 = prime[j] << 1;
      if (!(t1 & 1))
        t1 += t2 >> 1;
      while (t1 <= r) {
        if (t1 >= l)
          set(tbit, t1 - (l - 1));
        t1 += t2;
      }
    }
    for (int j = 0; j <= sz / 64; j++) {
      ull t1 = ~tbit[j];
      while (t1) {
        int t2 = __builtin_ctzll(t1);
        if (l + (j << 6) + t2 > r)
          break;
        prime[++cnt] = l + (j << 6) + t2;
        t1 -= t1 & -t1;
      }
    }
  }
}
```

2.19　位运算卷积（FWT）

用途：当下标是位掩码时，计算

$$c[k] = \sum_{i \text{ op } j = k} a[i]b[j],$$

其中 op 可以是按位 OR、XOR 或 AND。它是普通卷积在位运算半群上的对应物，常用于统计子集合并、异或和或状态转移。

三个函数的流程完全相同：把长度补成 2 的幂，正变换，逐点相乘，再做逆变换。`fwt_or`、`fwt_xor`、`fwt_and` 分别对应 `|`、`^`、`&`；XOR 的逆变换每层乘 `inv2`。时间复杂度为 $O(L \log L)$，空间复杂度为 $O(L)$，其中 L 是补齐后的最小二次幂长度。

例：若 `a[1]=1`、`b[2]=1`，则 OR 卷积在下标 `1|2=3` 处增加 1；XOR 卷积在下标 `1^2=3` 处增加 1；AND 卷积在下标 `1&2=0` 处增加 1。调用方式例如：`auto c = fwt_xor(a, b);`，其中 `c[k]` 就是所有 `i ^ j == k` 的乘积和。这里数组下标从 0 开始，长度不必预先是 2 的幂，函数会自动补零。模数必须能求 2 的逆元（当前 998244353 满足），并且输入下标应覆盖所有需要的位数。

```
#define int long long
const int mod = 998244353;
int inv2 = 499122177;
vector<int> fwt_or(vector<int> a, vector<int> b) {
  int lim = 1;
  while (lim < max(a.size(), b.size()))
```

```
    lim *= 2;
  int n = lim;
  while (a.size() < lim)
    a.push_back(0);
  while (b.size() < lim)
    b.push_back(0);
  auto fwtor = [&](vector<int> &a, int op) {
    for (int i = 1; i < n; i <<= 1) {
      for (int p = i << 1, j = 0; j < n; j += p) {
        for (int k = 0; k < i; k++) {
          a[i + j + k] += a[j + k] * op % mod;
          a[i + j + k] %= mod;
        }
      }
    }
  };
  fwtor(a, 1);
  fwtor(b, 1);
  for (int i = 0; i < n; i++) {
    a[i] = a[i] * b[i] % mod;
  }
  fwtor(a, -1);
  for (auto &it : a) {
    if (it < 0)
      it += mod;
  }
  return a;
}
vector<int> fwt_xor(vector<int> a, vector<int> b) {
  int lim = 1;
  while (lim < max(a.size(), b.size()))
    lim *= 2;
  int n = lim;
  while (a.size() < lim)
    a.push_back(0);
  while (b.size() < lim)
    b.push_back(0);
  auto fwtxor = [&](vector<int> &a, int op) {
    for (int i = 1; i < n; i <<= 1) {
      for (int p = i << 1, j = 0; j < n; j += p) {
        for (int k = 0; k < i; k++) {
          int X = a[j + k], Y = a[i + j + k];
          a[j + k] = (X + Y) % mod;
          a[i + j + k] = (X + mod - Y) % mod;
          (a[j + k] *= op) %= mod, (a[i + j + k] *= op) %= mod;
        }
      }
    }
  };
  fwtxor(a, 1);
  fwtxor(b, 1);
  for (int i = 0; i < n; i++) {
    a[i] = a[i] * b[i] % mod;
  }
  fwtxor(a, inv2);
  for (auto &it : a) {
    if (it < 0)
      it += mod;
  }
  return a;
}
vector<int> fwt_and(vector<int> a, vector<int> b) {
  int lim = 1;
  while (lim < max(a.size(), b.size()))
```

```
    lim *= 2;
  int n = lim;
  while (a.size() < lim)
    a.push_back(0);
  while (b.size() < lim)
    b.push_back(0);
  auto fwtand = [&](vector<int> &a, int op) {
    for (int i = 1; i < n; i <<= 1) {
      for (int p = i << 1, j = 0; j < n; j += p) {
        for (int k = 0; k < i; k++) {
          (a[j + k] += a[i + j + k] * op % mod + mod) %= mod;
        }
      }
    }
  };
  fwtand(a, 1);
  fwtand(b, 1);
  for (int i = 0; i < n; i++) {
    a[i] = a[i] * b[i] % mod;
  }
  fwtand(a, -1);
  for (auto &it : a) {
    if (it < 0)
      it += mod;
  }
  return a;
}
```

2.20　多项式与 NTT

用途： NTT/FFT 风格的多项式乘法及相关运算模板。

```
const int P = 998244353;
std::vector<int> rev, roots{0, 1};
int ksm(int a, int b) {
  int res = 1;
  for (; b; b >>= 1, a = 1ll * a * a % P)
    if (b & 1)
      res = 1ll * res * a % P;
  return res;
}
void dft(std::vector<int> &a) {
  int n = a.size();
  if (rev.size() != n) {
    int k = __builtin_ctz(n) - 1;
    rev.resize(n);
    for (int i = 0; i < n; ++i)
      rev[i] = rev[i >> 1] >> 1 | (i & 1) << k;
  }
  for (int i = 0; i < n; ++i)
    if (rev[i] < i)
      std::swap(a[i], a[rev[i]]);
  if (roots.size() < n) {
    int k = __builtin_ctz(roots.size());
    roots.resize(n);
    while ((1 << k) < n) {
      int e = ksm(3, (P - 1) >> (k + 1));
      for (int i = 1 << (k - 1); i < (1 << k); ++i) {
        roots[2 * i] = roots[i];
        roots[2 * i + 1] = 1ll * roots[i] * e % P;
      }
```

```
      ++k;
    }
  }
  for (int k = 1; k < n; k *= 2) {
    for (int i = 0; i < n; i += 2 * k) {
      for (int j = 0; j < k; ++j) {
        int u = a[i + j];
        int v = 1ll * a[i + j + k] * roots[k + j] % P;
        int x = u + v;
        if (x >= P)
          x -= P;
        a[i + j] = x;
        x = u - v;
        if (x < 0)
          x += P;
        a[i + j + k] = x;
      }
    }
  }
}
void idft(std::vector<int> &a) {
  int n = a.size();
  std::reverse(a.begin() + 1, a.end());
  dft(a);
  int inv = ksm(n, P - 2);
  for (int i = 0; i < n; ++i)
    a[i] = 1ll * a[i] * inv % P;
}
struct Poly {
  std::vector<int> a;
  Poly() {}
  Poly(int a0) {
    if (a0)
      a = {a0};
  }
  Poly(const std::vector<int> a1) : a(a1) {
    while (!a.empty() && !a.back())
      a.pop_back();
  }
  int size() const { return a.size(); }
  int operator[](int idx) const {
    if (idx < 0 || idx >= size())
      return 0;
    return a[idx];
  }
  Poly mulxk(int k) const {
    auto b = a;
    b.insert(b.begin(), k, 0);
    return Poly(b);
  }
  Poly Pxk(int k) const {
    k = std::min(k, size());
    return Poly(std::vector<int>(a.begin(), a.begin() + k));
  }
  Poly divxk(int k) const {
    if (size() <= k)
      return Poly();
    return Poly(std::vector<int>(a.begin() + k, a.end()));
  }
  friend Poly operator+(const Poly a, const Poly &b) {
    std::vector<int> res(std::max(a.size(), b.size()));
    for (int i = 0; i < res.size(); ++i) {
      res[i] = a[i] + b[i];
      if (res[i] >= P)
```

```
      res[i] -= P;
    }
    return Poly(res);
  }
  friend Poly operator-(const Poly a, const Poly &b) {
    std::vector<int> res(std::max(a.size(), b.size()));
    for (int i = 0; i < res.size(); ++i) {
      res[i] = a[i] - b[i];
      if (res[i] < 0)
        res[i] += P;
    }
    return Poly(res);
  }
  friend Poly operator*(Poly a, Poly b) {
    int sz = 1, tot = a.size() + b.size() - 1;
    while (sz < tot)
      sz *= 2;
    a.a.resize(sz);
    b.a.resize(sz);
    dft(a.a);
    dft(b.a);
    for (int i = 0; i < sz; ++i)
      a.a[i] = 1ll * a[i] * b[i] % P;
    idft(a.a);
    return Poly(a.a);
  }
  Poly &operator+=(Poly b) { return (*this) = (*this) + b; }
  Poly &operator-=(Poly b) { return (*this) = (*this) - b; }
  Poly &operator*=(Poly b) { return (*this) = (*this) * b; }
  Poly deriv() const {
    if (a.empty())
      return Poly();
    std::vector<int> res(size() - 1);
    for (int i = 0; i < size() - 1; ++i)
      res[i] = 1ll * (i + 1) * a[i + 1] % P;
    return Poly(res);
  }
  Poly integr() const {
    if (a.empty())
      return Poly();
    std::vector<int> res(size() + 1);
    for (int i = 0; i < size(); ++i)
      res[i + 1] = 1ll * a[i] * ksm(i + 1, P - 2) % P;
    return Poly(res);
  }
  Poly inv(int m) const {
    Poly x(ksm(a[0], P - 2));
    int k = 1;
    while (k < m) {
      k *= 2;
      x = (x * (2 - Pxk(k) * x)).Pxk(k);
    }
    return x.Pxk(m);
  }
  Poly log(int m) const { return (deriv() * inv(m)).integr().Pxk(m); }
  Poly exp(int m) const {
    Poly x(1);
    int k = 1;
    while (k < m) {
      k *= 2;
      x = (x * (1 - x.log(k) + Pxk(k))).Pxk(k);
    }
    return x.Pxk(m);
  }
```

```
  Poly pow(int k, int m) const {
    int i = 0;
    while (i < size() && a[i] == 0) {
      i++;
    }
    if (i == size() || 1ll * i * k >= m) {
      return Poly(vector<int>(m));
    }
    int v = a[i];
    auto f = divxk(i) * ksm(v, P - 2);
    // ksm(v,k) 这个 k 需要欧拉降幂
    return (f.log(m - i * k) * k).exp(m - i * k).mulxk(i * k) * ksm(v, k);
  }
  Poly sqrt(int m) const {
    Poly x(1);
    int k = 1;
    while (k < m) {
      k *= 2;
      x = (x + (Pxk(k) * x.inv(k)).Pxk(k)) * ((P + 1) / 2);
    }
    return x.Pxk(m);
  }
  Poly mulT(Poly b) const {
    if (b.size() == 0)
      return Poly();
    int n = b.size();
    std::reverse(b.a.begin(), b.a.end());
    return ((*this) * b).divxk(n - 1);
  }
  std::vector<int> eval(std::vector<int> x) const {
    if (size() == 0)
      return std::vector<int>(x.size(), 0);
    const int n = std::max((int)x.size(), (int)size());
    std::vector<Poly> q(4 * n);
    std::vector<int> ans(x.size());
    x.resize(n);
    std::function<void(int, int, int)> build = [&](int p, int l, int r) {
      if (r - l == 1) {
        q[p] = std::vector<int>{1, (P - x[l]) % P};
      } else {
        int m = (l + r) / 2;
        build(2 * p, l, m);
        build(2 * p + 1, m, r);
        q[p] = q[2 * p] * q[2 * p + 1];
      }
    };
    build(1, 0, n);
    std::function<void(int, int, int, const Poly &)> work =
        [&](int p, int l, int r, const Poly &num) {
          if (r - l == 1) {
            if (l < (int)ans.size())
              ans[l] = num[0];
          } else {
            int m = (l + r) / 2;
            work(2 * p, l, m, num.mulT(q[2 * p + 1]).Pxk(m - l));
            work(2 * p + 1, m, r, num.mulT(q[2 * p]).Pxk(r - m));
          }
        };
    work(1, 0, n, mulT(q[1].inv(n)));
    return ans;
  }
};
// 寻找原根 （一个模数可能有多个原根）
// 998244353   原根有 3
```

```
int find() {
  int i = 2;
  int k = __builtin_ctz(P - 1);
  while (true) {
    if (power(i, (P - 1) / 2) != 1) {
      break;
    }
    i += 1;
  }
  return power(i, (P - 1) >> k);
}
```

2.20.1 轻量 NTT 卷积

用途： 用 NTT 在模 998244353 下快速完成多项式卷积。

```
vector<int> NTT(vector<int> a, vector<int> b) {
  int s = a.size() + b.size() - 1;
  int len = 1;
  while (len < a.size() + b.size())
    len *= 2;
  a.resize(len);
  b.resize(len);
  const int mod = 998244353;
  int n = a.size();
  int m = b.size();
  const int lg = log2(n);
  auto ksm = [&](int a, int b) {
    int ans = 1;
    while (b) {
      if (b & 1)
        ans = ans * a % mod;
      b >>= 1;
      a = a * a % mod;
    }
    return ans;
  };
  auto ntt = [&](vector<int> &a, int f) {
    for (int i = 0; i < n; i++) {
      int ans = 0;
      int cnt = 0;
      for (int j = lg - 1; j >= 0; j--) {
        if (i >> j & 1)
          ans += (1 << cnt);
        cnt++;
      }
      if (i <= ans)
        swap(a[i], a[ans]);
    }
    for (int len = 2; len <= n; len *= 2) {
      int gn = ksm(3, (mod - 1) / len);
      if (f)
        gn = ksm(3, mod - 1 - (mod - 1) / len);
      for (int i = 0; i < n; i += len) {
        int w = 1;
        for (int j = i; j < i + len / 2; j++) {
          int y = w * a[j + len / 2] % mod;
          int x = a[j];
          a[j] = (x + y) > mod ? x + y - mod : x + y;
          a[j + len / 2] = (x - y + mod) > mod ? x - y : x - y + mod;
          w = w * gn % mod;
        }
```

```
      }
    }
  };
  ntt(a, 0);
  ntt(b, 0);
  for (int i = 0; i < n; i++)
    a[i] = a[i] * b[i] % mod;
  ntt(a, 1);
  int inv = ksm(n, mod - 2);
  for (int i = 0; i < n; i++)
    a[i] = a[i] * inv % mod;
  while (a.size() > s)
    a.pop_back();
  return a;
}
```

2.21 自适应辛普森积分

用途：自适应划分区间，用数值积分计算定积分。

```
const double Pi = std::acos(-1.0);
constexpr double EPS = 1e-9;
double f(double x) { return (c * x + d) / (a * x + b); }
double simpson(double l, double r) {
  return (f(l) + 4 * f((l + r) / 2) + f(r)) * (r - l) / 6;
}
double integral(double l, double r, double eps, double st) {
  double mid = (l + r) / 2;
  double sl = simpson(l, mid);
  double sr = simpson(mid, r);
  if (std::abs(sl + sr - st) <= 15 * eps)
    return sl + sr + (sl + sr - st) / 15;
  return integral(l, mid, eps / 2, sl) + integral(mid, r, eps / 2, sr);
}
double integral(double l, double r) {
  return integral(l, r, EPS, simpson(l, r));
}
```

2.22 行列式

用途：用消元或模意义运算计算矩阵行列式。

```
// 任意模数
// n^2*log2+n^3
int n;
cin >> n;
int mod;
cin >> mod;
vector<vector<int>> a(n + 10, vector<int>(n + 10));
for (int i = 1; i <= n; i++) {
  for (int j = 1; j <= n; j++) {
    cin >> a[i][j];
  }
}
int ans = 1;
int w = 1;
for (int i = 1; i <= n; i++) {
  for (int j = i + 1; j <= n; j++) {
    while (a[i][i]) {
```

```
      int div = a[j][i] / a[i][i];
      for (int k = i; k <= n; k++) {
        a[j][k] -= 1ll * div * a[i][k] % mod;
        a[j][k] %= mod;
      }
      swap(a[i], a[j]);
      w = -w;
    }
    swap(a[i], a[j]);
    w = -w;
  }
}
for (int i = 1; i <= n; i++)
  ans = ans * a[i][i] % mod;
ans = w * ans;
if (ans < 0)
  ans += mod;
cout << ans << "\n";
```

2.23　高斯消元

用途：求解线性方程组、秩和自由元。

```
void slove() {
  int n;
  cin >> n;
  vector<vector<double>> a(n + 10, vector<double>(n + 10));
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n + 1; j++) {
      cin >> a[i][j];
    }
  }
  auto print = [&](auto a) {
    for (int i = 1; i <= n; i++) {
      for (int j = 1; j <= n + 1; j++) {
        cout << a[i][j] << " ";
      }
      cout << "\n";
    }
    cout << "\n";
  };
  int r = 0;
  for (int c = 1; c <= n; c++) {
    double mx = -1e9;
    int p = -1;
    for (int i = r + 1; i <= n; i++) {
      if (abs(a[i][c]) > mx) {
        mx = abs(a[i][c]);
        p = i;
      }
    }
    if (p == -1 || (abs(mx) < eps)) {
      continue;
    }
    swap(a[r + 1], a[p]);
    for (int j = n + 1; j >= c; j--) {
      a[r + 1][j] /= a[r + 1][c];
    }
    for (int i = 1; i <= n; i++) {
      if (i == r + 1 || abs(a[i][c]) < eps)
        continue;
```

```
      for (int j = n + 1; j >= c; j--) {
        a[i][j] -= a[r + 1][j] * a[i][c];
      }
    }
    r++;
  }
  cout << fixed << setprecision(2);
  if (r == n) {
    for (int i = 1; i <= n; i++) {
      cout << "x" << to_string(i) << "=" << a[i][n + 1] << "\n";
    }
    return;
  }
  for (int i = r + 1; i <= n; i++) {
    if (abs(a[i][n + 1]) > eps) {
      cout << "-1\n";
      return;
    }
  }
  cout << "0\n";
}
```

2.24 GCD 卷积

用途：计算

$$c[k] = \sum_{\gcd(i,j)=k} a[i]b[j],$$

也就是按下标的最大公约数分组的卷积。直接枚举 (i,j) 是 $O(n^2)$；这里先做约数和变换 $A[d]=\sum_{d|i} a[i]$、$B[d]=\sum_{d|i} b[i]$，逐点相乘后用莫比乌斯反演恢复 c。`init` 用线性筛求 μ，`gc` 完成三次变换。**复杂度：**设 $L=\min(|a|,|b|)-1$，时间 $O(L\log L)$，空间 $O(L)$。

例如只有下标 1、2 且 `a[1]=a[2]=b[1]=b[2]=1` 时，四对下标中有三对的 gcd 为 1、一对的 gcd 为 2，所以结果是 `c[1]=3,c[2]=1`。使用时先 `Poly p; p.init(L);`（`L` 为需要的最大下标），再调用 `auto c = p.gc(a, b);`；数组按 1 下标存放，`c[k]` 就是 gcd 恰为 k 的贡献。gc 内部的 `f(...,1)` 是“求所有倍数之和”，`f(...,-1)` 是乘上 `\mu(j)` 的莫比乌斯逆变换。

```
struct Poly
{

    vector<int> mu, arr, book;
    int cnt = 0;
    void init(int n)
    {
        mu.resize(n + 10);
        arr.resize(n + 10);
        book.resize(n + 10);
        book[1] = 1;
        mu[1] = 1;
        for (int i = 2; i <= n; i++)
        {
            if (!book[i])
            {
                arr[++cnt] = i;
                mu[i] = -1;
            }
            for (int j = 1; i * arr[j] <= n; j++)
            {
                book[i * arr[j]] = 1;
                if (i % arr[j] == 0)
```

```
                {
                    break;
                }
                mu[i * arr[j]] = mu[i] * mu[arr[j]];
            }
        }
    }
    vector<int> gc(vector<int> a, vector<int> b)
    {
        int n = a.size() - 1;
        int m = b.size() - 1;
        int len = min(n, m);
        auto f = [&](vector<int> &a, int flag)
        {
            vector<int> b(len + 1);
            for (int i = 1; i <= len; i++)
            {
                for (int j = 1; i * j <= len; j++)
                {
                    int t = a[i * j];
                    if (flag == -1)
                        t = t * mu[j];
                    b[i] = ((b[i] + t) % mod + mod) % mod;
                }
            }
            a = b;
        };
        vector<int> c(len + 1);
        f(a, 1);
        f(b, 1);
        for (int i = 1; i <= len; i++)
            c[i] = a[i] * b[i] % mod;
        f(c, -1);
        return c;
    }
};
```

2.25 Trygub numbers

用途：在固定进制 B 下，稀疏维护一个可以非常大的整数，支持给某一位加上一个带符号的系数，并查询任意位。`mp[p]` 只保存非零的局部位值，进位或借位沿着更高位传播，因此不需要开一个和整数大小相同的数组。“Trygub number” 不是标准库名称，这个模板只适合题目明确要求按位加减、且整数位数可能远大于数组范围的场景；普通整数运算不需要使用它。**复杂度：** `getdig` 为 $O(\log K)$；`add` 为 $O(t \log K)$，其中 t 是本次实际传播经过的位数、K 是 `map` 中的非零项数。连续进位很多时，单次操作可能变慢。

例如 $B = 10$ 时：`add(1, 0)` 表示加 1×10^0；再执行 `add(9, 0)`，第 0 位变成 0 并向第 1 位进 1，此时 `getdig(0)=0`、`getdig(1)=1`。执行 `add(-1, 0)` 后表示数字 9，查询第 0 位得到 9。位下标从 0 开始。负的 `mp` 值表示尚未显式展开的借位，`getdig` 会检查前一个非零项并补上这一步。

```
const int B = 10;
struct TrygubNumber {
  map<int, int> mp;
  void add(int a, int b) {
    while (1) {
      if (!a)
        break;
      auto &it = mp[b];
      it += a, a = it / B, it -= a * B;
      if (!it)
        mp.erase(b);
```

```
      b++;
    }
  }
  int getdig(int p) {
    int ans = mp.count(p) ? mp[p] : 0;
    auto it = mp.lower_bound(p);
    if (it != mp.begin() && prev(it)->second < 0)
      --ans;
    return (ans % B + B) % B;
  }
} s1;
// s1.add(a, b): 加上 a * B^b; s1.getdig(p): 询问第 p 位
```

第 3 章

数据结构

> 维护序列、集合、区间和动态树结构的模板。

3.1 ST 表

用途： 静态数组上的幂次区间查询；适合最值、按位与、按位或和 gcd 等幂等运算。

```
// 求区间最大值，求区间最小值，求区间
// 求区间按位与、按位或和 gcd；下面给出区间最大值示例。
template <class T> struct rmq {
  int n;
  vector<T> a;
  vector<array<T, 21>> f;
  function<T(T, T)> func;
  rmq(){};
  vector<int> len;
  void work(vector<T> a, function<T(T, T)> func_) {
    this->a = a;
    this->func = func_;
    n = a.size();
    f.assign(n + 1, {});
    len.resize(n + 1);
    len[1] = 0;
    for (int i = 2; i <= n; i++)
      len[i] = len[i / 2] + 1;
    for (int i = 1; i <= n; i++)
      f[i][0] = a[i - 1];
    const int lg = log2(n);
    for (int j = 1; j <= lg; j++) {
      for (int i = 1; i + (1 << (j)) - 1 <= n; i++) {
        f[i][j] = func(f[i][j - 1], f[i + (1 << (j - 1))][j - 1]);
      }
    }
    return;
  }
  T ask(int l, int r) {
    return func(f[l][len[r - l + 1]],
                f[r - (1 << len[r - l + 1]) + 1][len[r - l + 1]]);
  }
};
```

3.2 二维前缀和与差分

用途： 用前缀和 O(1) 查询矩形和；差分形式可支持矩形加。

```
for (int i = 1; i <= n; i++)
  for (int j = 1; j <= m; j++)
    sum[i][j] = arr[i][j] + sum[i - 1][j] + sum[i][j - 1] - sum[i - 1][j - 1];
auto get = [&](int x1, int y1, int x2, int y2) {
```

```
  // 右下角 x2  y2 左上角 x1 y1
  return sum[x2][y2] - sum[x1 - 1][y2] - sum[x2][y1 - 1] + sum[x1 - 1][y1 - 1];
};

arr[x1][y1] += c;
arr[x1][y2 + 1] -= c;
arr[x2 + 1][y1] -= c;
arr[x2 + 1][y2 + 1] += c;
```

3.3 分块

用途：平方分解维护序列，适合低复杂度的区间修改与查询。

```
int n;
cin >> n;
vector<int> le(n + 10);
vector<int> ri(n + 10);
vector<int> nowbk(n + 10);
int bk = sqrt(n);
for (int i = 1; i <= n; i++) {
  int cnt = (i - 1) / bk + 1;
  le[i] = (cnt - 1) * bk + 1; //  当前下标所在块的左边界下标
  ri[i] = min(n, cnt * bk);   //  当前下标所在快的右边界下标
  nowbk[i] = cnt;
}
auto get = [&](int st, int ed) {
  if (nowbk[st] == nowbk[ed]) {
    for (int i = st; i <= ed; i++) {
    }
    return;
  }
  for (int i = st; i <= ri[st]; i++) {
  }

  for (int i = nowbk[st] + 1; i <= nowbk[ed] - 1; i++) {
    int stpos = (i - 1) * bk + 1;
    int edpos = min(n, i * bk); // 当前块的区间左边界下标和区间右边界下标
  }
  for (int i = le[ed]; i <= ed; i++) {
  }
  return;
};
```

3.4 ZKW 线段树

用途：非递归地支持单点赋值和闭区间查询；`ask(l,r)` 保留左右合并顺序，并避免空 `Info` 参与第一次合并。

```
template <class Info> struct ZKW {
  int n, s = 1;
  vector<Info> t;
  ZKW(int n) : n(n) {
    while (s < n)
      s <<= 1;
    t.resize(s << 1);
  }
  void build(vector<Info> a) {
    for (int i = 1; i <= n; i++)
      t[s + i - 1] = a[i];
    for (int i = s - 1; i; i--)
```

```
      t[i] = t[i << 1] + t[i << 1 | 1];
  }
  void modify(int p, Info x) {
    for (t[p += s - 1] = x; p >>= 1;)
      t[p] = t[p << 1] + t[p << 1 | 1];
  }
  Info ask(int l, int r) {
    Info x{}, y{};
    bool hx = 0, hy = 0;
    for (l += s - 1, r += s; l < r; l >>= 1, r >>= 1) {
      if (l % 2) {
        if (!hx)
          x = t[l++];
        else
          x = x + t[l++];
        hx = 1;
      }
      if (r % 2) {
        --r;
        if (!hy)
          y = t[r];
        else
          y = t[r] + y;
        hy = 1;
      }
    }
    if (!hx)
      return y;
    if (!hy)
      return x;
    return x + y;
  }
};

struct Info {};

Info operator+(Info ls, Info rs) {
  Info u = Info();
  return u;
}
```

3.5　懒标记线段树

用途：支持区间修改与区间查询，通过懒标记把复杂度保持在 O(log N)。

```
template <class Info, class Tag> struct segmenttree {
  int n;
  vector<Info> info;
  vector<Tag> tag;
  void init(int n) {
    this->n = n;
    tag.resize(4 * n + 10);
    info.resize(4 * n + 10);
  }
  segmenttree(int n) { init(n); }
  segmenttree() {}
  void up(int i) { info[i] = info[2 * i] + info[2 * i + 1]; }
  void apply(int i, const Tag &v) {
    info[i].apply(v);
    tag[i].apply(v);
  }
```

```
  void down(int i) {
    apply(2 * i, tag[i]);
    apply(2 * i + 1, tag[i]);
    tag[i] = Tag();
  }
  void modify(int i, int pos, int st, int ed, const Info &v) {
    if (st == ed) {
      info[i] = v;
      return;
    }
    int mid = (st + ed) / 2;
    down(i);
    if (pos <= mid)
      modify(i * 2, pos, st, mid, v);
    if (pos > mid)
      modify(i * 2 + 1, pos, mid + 1, ed, v);
    up(i);
  }
  void modify(int pos, const Info &v) { modify(1, pos, 1, n, v); }
  Info ask(int i, int st, int ed, int l, int r) {
    if (st >= l && ed <= r) {
      return info[i];
    }
    int mid = (st + ed) / 2;
    Info u1 = Info();
    Info u2 = Info();

    down(i);
    int f1 = 0;
    int f2 = 0;
    if (l <= mid)
      u1 = ask(i * 2, st, mid, l, r), f1 = 1;
    if (r > mid)
      u2 = ask(i * 2 + 1, mid + 1, ed, l, r), f2 = 1;
    if (f1 == f2)
      return u1 + u2;
    if (f1 == 1)
      return u1;
    return u2;
  }
  Info ask(int l, int r) { return ask(1, 1, n, l, r); }
  void rangemodify(int i, int l, int r, int st, int ed, const Tag &v) {
    if (st >= l && ed <= r) {
      apply(i, v);
      return;
    }
    int mid = (st + ed) / 2;
    down(i);
    if (l <= mid)
      rangemodify(i * 2, l, r, st, mid, v);
    if (r > mid)
      rangemodify(i * 2 + 1, l, r, mid + 1, ed, v);
    up(i);
  }
  void rangemodify(int l, int r, const Tag &v) {
    rangemodify(1, l, r, 1, n, v);
  }
};
struct Tag {
  void apply(Tag t) {}
};
struct Info {
  void apply(Tag t) {}
};
```

```
Info operator+(Info ls, Info rs) {
  Info u;
  return u;
}
```

3.6　线段树合并

用途：合并两棵稀疏线段树，常用于树上子树信息合并。

```
const int N = 7e6;
int ls[N], rs[N], val[N], k[N];
void slove() {
  int n, m;
  cin >> n >> m;
  vector<vector<int>> g(n + 10);
  HLD hld(n);
  for (int i = 1; i < n; i++) {
    int x, y;
    cin >> x >> y;
    g[x].push_back(y);
    g[y].push_back(x);
    hld.add(x, y);
    hld.add(y, x);
  }
  hld.work(1);
  int tot = 0;
  auto up = [&](int x) {
    val[x] = max(val[ls[x]], val[rs[x]]);
    if (val[ls[x]] > val[rs[x]]) {
      k[x] = k[ls[x]];
    } else {
      k[x] = k[rs[x]];
      if (val[ls[x]] == val[rs[x]]) {
        k[x] = min(k[rs[x]], k[ls[x]]);
      }
    }
  };
  auto insert = [&](auto insert, int now, int l, int r, int pos, int f) -> int {
    if (!now)
      now = ++tot;
    if (l == r) {
      val[now] += f;
      k[now] = l;
      return now;
    }
    int mid = (l + r) / 2;
    if (pos <= mid)
      ls[now] = insert(insert, ls[now], l, mid, pos, f);
    else
      rs[now] = insert(insert, rs[now], mid + 1, r, pos, f);
    up(now);
    return now;
  };
  vector<int> rt(n + 10);
  for (int i = 1; i <= m; i++) {
    int x, y;
    cin >> x >> y;
    int z;
    cin >> z;
    int lc = hld.lca(x, y);
    rt[hld.fa[lc]] = insert(insert, rt[hld.fa[lc]], 1, 1e5, z, -1);
```

```
    rt[lc] = insert(insert, rt[lc], 1, 1e5, z, -1);
    rt[x] = insert(insert, rt[x], 1, 1e5, z, 1);
    rt[y] = insert(insert, rt[y], 1, 1e5, z, 1);
  }
  auto merge = [&](auto merge, int x, int y, int l, int r) -> int {
    if (x == 0 || y == 0)
      return x + y;
    if (l == r) {
      val[x] += val[y];
      k[x] = l;
      return x;
    }
    int mid = (l + r) / 2;
    ls[x] = merge(merge, ls[x], ls[y], l, mid);
    rs[x] = merge(merge, rs[x], rs[y], mid + 1, r);
    up(x);
    return x;
  };
  vector<int> ans(n + 10);
  auto dfs = [&](auto dfs, int u, int f) -> void {
    for (auto it : g[u]) {
      if (it == f)
        continue;
      dfs(dfs, it, u);
      rt[u] = merge(merge, rt[u], rt[it], 1, 1e5);
    }
    ans[u] = k[rt[u]];
  };
  dfs(dfs, 1, 0);
  for (int i = 1; i <= n; i++) {
    cout << ans[i] << "\n";
  }
}
signed main() {
  ios::sync_with_stdio(0);
  cin.tie(0);
  cout.tie(0);
  slove();
  return 0;
}
```

3.7 线段树

用途：通用的区间信息维护模板，支持单点或区间操作。

```
template <class Info> struct segmenttree {
  vector<Info> info;
  int n;
  void init(int n) {
    this->n = n;
    info.resize(4 * n + 10);
  }
  segmenttree(int n) {
    this->n = n;
    info.resize(4 * n + 10);
  }
  segmenttree() {}
  void modify(int i, int st, int ed, int pos, const Info &u) {
    if (st == ed) {
      info[i] = u;
      return;
```

```
    }
    int mid = (st + ed) / 2;
    if (pos <= mid)
      modify(i * 2, st, mid, pos, u);
    if (pos > mid)
      modify(i * 2 + 1, mid + 1, ed, pos, u);
    info[i] = info[i * 2] + info[i * 2 + 1];
  }
  Info ask(int i, int st, int ed, int l, int r) {
    if (st >= l && ed <= r) {
      return info[i];
    }
    int mid = (st + ed) / 2;
    int f1 = 0;
    int f2 = 0;
    Info u1, u2;
    if (l <= mid)
      u1 = ask(i * 2, st, mid, l, r), f1 = 1;
    if (r > mid)
      u2 = ask(i * 2 + 1, mid + 1, ed, l, r), f2 = 1;
    if (f1 == f2)
      return u1 + u2;
    if (f1 == 1)
      return u1;
    return u2;
  }
  template <class F>
  int findFirst(int i, int st, int ed, int l, int r, F &&pred) {
    if (st > r || ed < l) {
      return -1;
    }
    if (st >= l && ed <= r && !pred(info[i])) {
      return -1;
    }
    if (st == ed) {
      return st;
    }
    int mid = (st + ed) / 2;
    int res = findFirst(2 * i, st, mid, l, r, pred);
    if (res == -1) {
      res = findFirst(2 * i + 1, mid + 1, ed, l, r, pred);
    }
    return res;
  }

  template <class F>
  int findLast(int i, int st, int ed, int l, int r, F &&pred) {
    if (st > r || ed < l) {
      return -1;
    }
    if (st >= l && ed <= r && !pred(info[i])) {
      return -1;
    }
    if (st == ed)
      return st;
    int mid = (st + ed) / 2;
    int res = findLast(2 * i + 1, mid + 1, ed, l, r, pred);
    if (res == -1) {
      res = findLast(2 * i, st, mid, l, r, pred);
    }
    return res;
  }
  Info ask(int l, int r) { return ask(1, 1, n, l, r); }
  void modify(int pos, Info u) { modify(1, 1, n, pos, u); }
```

```
  template <class F> int findFirst(int l, int r, F &&pred) {
    return findFirst(1, 1, n, l, r,
                     pred); // 从左向右找第一个满足条件的位置；找不到返回-1
  }
  template <class F> int findLast(int l, int r, F &&pred) {
    return findLast(1, 1, n, l, r,
                    pred); // 从右向左找第一个满足条件的位置；找不到返回-1
  }
  /*
    Info u=Info();
    int pos = seg.findFirst(1, n, [&](auto x)
                               {
                                if((u+x).cnt-1>m){
                                   return 1;
                                }
                                u=u+x;
                                return 0; });
  */
};
struct Info {};
Info operator+(Info ls, Info rs) {
  Info u = Info();
  return u;
}
```

3.8 李超树

用途：维护直线或线段的最值，支持插入与单点最值查询。

```
const int N = 2e6;
double k[N], b[N];
int d[N * 4], tmp[N];
const int inf = 1e18;
double f(int x, int id) { return k[id] * tmp[x] + b[id]; }
double ask(int i, int l, int r, int x) {
  if (l == r)
    return f(x, d[i]);
  int mid = (l + r) / 2;
  return max(f(x, d[i]),
             x <= mid ? ask(i * 2, l, mid, x) : ask(i * 2 + 1, mid + 1, r, x));
}
void update(int i, int l, int r, int id) {
  if (l == r) {
    if (f(l, id) > f(l, d[i]))
      d[i] = id;
    return;
  }
  int mid = (l + r) / 2;
  if (f(mid, id) > f(mid, d[i]))
    swap(id, d[i]);
  if (f(l, id) > f(l, d[i]))
    update(i * 2, l, mid, id);
  else if (f(r, id) > f(r, d[i]))
    update(i * 2 + 1, mid + 1, r, id);
}
void modify(int i, int l, int r, int st, int ed, int id) {
  if (st >= l && ed <= r) {
    update(i, st, ed, id);
    return;
  }
  int mid = (st + ed) / 2;
```

```
  if (l <= mid)
    modify(i * 2, l, r, st, mid, id);
  if (r > mid)
    modify(i * 2 + 1, l, r, mid + 1, ed, id);
}
```

3.9　树状数组（Fenwick）

用途：以极小常数支持单点修改、前缀查询及其扩展。

```
struct fenwick {
  using i64 = long long;
  vector<i64> c;
  int n;
  int lg2;
  fenwick(int n) {
    c.resize(n + 1);
    lg2 = log2(n) + 1;
    this->n = n;
  }
  void clear(int i) {
    while (i <= n) {
      c[i] = 0;
      i += i & -i;
    }
  }
  void add(int i, i64 x) {
    while (i <= n) {
      c[i] += x;
      i += i & -i;
    }
  }
  i64 sum1(int i) {
    i64 sum = 0;
    while (i) {
      sum += c[i];
      i -= i & -i;
    }
    return sum;
  }
  i64 getsum(int l, int r) { return sum1(r) - sum1(l - 1); }
  int kth(i64 k) { // 求第 K 小的数是多少// 如 1 3 3 4 第二小的数是 3, 第三小数是 3
    i64 now = 0;
    i64 sum = 0;
    for (int i = lg2; i >= 0; i--) {
      if (now + (1 << i) <= n && sum + c[now + (1 << i)] < k) {
        sum += c[now + (1 << i)];
        now += (1 << i);
      }
    }
    return now + 1;
  }
};
```

3.10　线性基

用途：维护向量空间基，处理异或最大值、表示性和第 k 小异或值。

```
struct xorbase {
  vector<int> base;
  int iszero = 0;
  int wai = 0;
  xorbase() { base.resize(70); }
  int insert(int x) {
    for (int i = 62; i >= 0; i--) {
      if (x >> i & 1) {
        if (base[i])
          x ^= base[i];
        else {
          base[i] = x;
          return 1ll;
        }
      }
    }
    iszero = 1;
    wai++;
    return 0ll;
  }
  int kth(int k) // 求的是去重后的第 k 小 这里我们不算空集所以可能异或不出来 0
  {
    k -= iszero;
    if (k == 0)
      return 0;
    int ans = 0;
    int cnt = 0;
    for (int i = 0; i <= 60; i++)
      if (base[i])
        cnt++;
    if ((1ll << cnt) - 1 < k)
      return -1; // 总的异或种类小于 k;
    for (int i = 60; i >= 0; i--) {
      if (base[i]) {
        if ((k >> (--cnt) & 1) ^ (ans >> i & 1)) {
          ans ^= base[i];
        }
      }
    }
    return ans;
  }
  int getrk(int x) {
    // 去重后的排名是多少从小到大
    int cnt = 0;
    int ans = 0;
    for (int i = 0; i <= 60; i++) {
      if (base[i]) {
        if (x >> i & 1)
          ans += (1ll << cnt);
        cnt++;
      }
    }
    ans += iszero;
    return ans;
  }
};

struct Ji {
  int p[31], pos[31];
  void insert(int j, int x) {
    for (int i = 30, val = x; i >= 0; --i)
      if (val & (1ll << i)) {
        if (!p[i]) {
```

```
          p[i] = val, pos[i] = j;
          return;
        }
        if (pos[i] < j) {
          swap(pos[i], j);
          swap(val, p[i]);
        }
        val ^= p[i];
      }
  }
  int qry(int l) {
    int ans = 0;
    for (int i = 30; i >= 0; --i)
      if (p[i] && pos[i] >= l)
        ans = max(ans, ans ^ p[i]);
    return ans;
  }
};
```

3.11　主席树（可持久化线段树）

用途：可持久化线段树，保存多个版本并支持区间第 k 小等查询。

```
struct Ptr {
  vector<int> ls, rs, sum, root;
  int idx = 0, size = 0, n = 0, range = 0;
  void init(vector<int> a, int size, int range) { // 动态开点
    this->n = a.size();
    this->range = range;
    this->size = size;

    root.resize(n + 1);
    ls.resize(size + 5);
    rs.resize(size + 5);
    sum.resize(size + 5);

    a.insert(a.begin(), 0);
    for (int i = 1; i <= n; i++)
      root[i] = insert(root[i - 1], 1, range, a[i]);
  }
  Ptr(vector<int> a, int size, int range) {
    // 动态开点
    init(a, size, range);
  }
  int insert(int pre, int l, int r, int pos) {
    int rt = ++idx;
    ls[rt] = ls[pre];
    rs[rt] = rs[pre];
    sum[rt] = sum[pre];
    sum[rt]++;
    if (l == r)
      return rt;
    int mid = (l + r) / 2;
    if (pos <= mid)
      ls[rt] = insert(ls[pre], l, mid, pos);
    else
      rs[rt] = insert(rs[pre], mid + 1, r, pos);
    return rt;
  };
  int kth(int pre, int now, int l, int r, int k) {
    if (l == r) {
```

```
      return l;
    }
    int mid = (l + r) / 2;
    int ans = sum[ls[now]] - sum[ls[pre]];
    if (k <= ans)
      return kth(ls[pre], ls[now], l, mid, k);
    else
      return kth(rs[pre], rs[now], mid + 1, r, k - ans);
  };
  int kth(int l, int r, int k) {
    return kth(root[l - 1], root[r], 1, range, k);
  }
  int rangecnt(int pre, int now, int st, int ed, int l, int r) {
    if (!sum[now])
      return 0;
    if (st >= l && ed <= r) {
      return sum[now] - sum[pre];
    }
    int ans = 0;
    int mid = (st + ed) / 2;
    if (l <= mid)
      ans += rangecnt(ls[pre], ls[now], st, mid, l, r);
    if (r > mid)
      ans += rangecnt(rs[pre], rs[now], mid + 1, ed, l, r);
    return ans;
  }
  int rangecnt(int l, int r, int v1, int v2) {
    return rangecnt(root[l - 1], root[r], 1, range, v1, v2);
  }
};
// 注意叶子节点没有左右子区间
struct ChairmanTree {
    struct Node { int l, r, sum; };

    int L, R;
    std::vector<Node> t{{0, 0, 0}};
    ChairmanTree(int l, int r) : L(l), R(r) {}
    int ins(int p, int l, int r, int x) {
        int q = (int)t.size(); t.push_back(t[p]); ++t[q].sum;
        if (l == r) return q;
        int m = (int)((long long)l + ((long long)r - l) / 2);
        if (x <= m) t[q].l = ins(t[p].l, l, m, x);
        else        t[q].r = ins(t[p].r, m + 1, r, x);
        return q;
    }
// 注意叶子节点没有左右子区间
    int kth(int u, int v, int l, int r, int k) {
        if (l == r) return l;
        int m = (l+r)/2;
        int cnt = t[t[v].l].sum - t[t[u].l].sum;
        if (k <= cnt) return kth(t[u].l, t[v].l, l, m, k);
        return kth(t[u].r, t[v].r, m + 1, r, k - cnt);
    }
    // p=0 表示从空树开始；返回新根，旧根不变。
    int insert(int p, int x) { return ins(p, L, R, x); }
    // 查询 root_v-root_u 中第 k 小，k 从 1 开始。
    int ask(int root_u, int root_v, int k)  {
        return kth(root_u, root_v, L, R, k);
    }
};
```

3.12　并查集

用途：维护动态连通块，支持近似 O(1) 的合并与代表元查询。

```
struct DSU {
  vector<int> f, siz;
  DSU() {}
  DSU(int n) { init(n); }
  void init(int n) {
    f.resize(n + 1);
    iota(f.begin() + 1, f.begin() + 1 + n, 1);
    siz.assign(n + 1, 1);
  }
  int find(int x) {
    while (x != f[x]) {
      x = f[x] = f[f[x]];
    }
    return x;
  }
  bool same(int x, int y) { return find(x) == find(y); }

  bool merge(int x, int y) {
    x = find(x);
    y = find(y);
    if (x == y) {
      return false;
    }
    siz[x] += siz[y];
    f[y] = x;
    return true;
  }
  int size(int x) { return siz[find(x)]; }
};

struct DSU {
  vector<int> f, fv;
  int find(int x) {
    if (f[x] == x)
      return x;
    int xx = find(f[x]);
    fv[x] ^= fv[f[x]];
    f[x] = xx;
    return xx;
  }
  DSU(int n) {
    f.resize(n + 10);
    fv.resize(n + 10);
    for (int i = 1; i <= n; i++)
      f[i] = i;
  }
  int merge(int x, int y, int w) {
    int u = x;
    int v = y;
    x = find(x);
    y = find(y);
    if (x == y) {
      if ((fv[u] ^ fv[v] ^ w) == 0)
        return 1;
      else
        return -1;
    }
    f[x] = y;
```

```
    fv[x] = fv[u] ^ fv[v] ^ w;
    return 2;
  }
};
```

3.13 带权并查集

用途：在并查集连通关系上维护势能差、距离或模约束。

```
int find(int x) {
  if (f[x] == x)
    return x;
  int now = f[x];
  f[x] = find(f[x]);
  val[x] += val[now];
  return f[x];
};
void slove() {
  for (int i = 1; i <= n; i++)
    f[i] = i, val[i] = 0;
  for (int i = 1; i < n; i++) {
    int a, b, c;
    cin >> a >> b;
    int x = find(a);
    int y = find(b);
    // 他们之间的权值为 1
    val[y] += val[a] + 1;
    f[y] = x;
  }
}
```

3.14 可撤销并查集

用途：通过栈记录修改，配合分治或回滚处理动态连通性。

```
struct DSU {
  int n;
  vector<int> f, sz, st;
  DSU() {}
  DSU(int n) {
    f.resize(n + 1);
    sz.resize(n + 1, 1);
    for (int i = 1; i <= n; i++)
      f[i] = i;
  }
  int find(int x) { return f[x] == x ? x : find(f[x]); }
  void merge(int x, int y) {
    x = find(x), y = find(y);
    if (x == y) {
      st.push_back(-1);
      return;
    }
    if (sz[x] > sz[y])
      swap(x, y);
    st.push_back(x);
    f[x] = y;
    sz[y] += sz[x];
  }
  void del() {
```

```
    if (st.size()) {
      int x = st.back();
      if (x == -1) {
        st.pop_back();
        return;
      }
      sz[f[x]] -= sz[x];
      f[x] = x;
      st.pop_back();
    }
  }
};
```

3.15 Link-Cut Tree（LCT）

用途： Link-Cut Tree 维护动态森林中的路径信息和连边操作。

```
struct LCT {
  int ch[N][2], fa[N], tag[N];
  void clear(int x) { ch[x][0] = ch[x][1] = fa[x] = tag[x] = 0; }
  int getch(int x) { return ch[fa[x]][1] == x; }
  int isroot(int x) { return ch[fa[x]][0] != x && ch[fa[x]][1] != x; }

  void update(int x) {
    if (!isroot(x))
      update(fa[x]);
    pushdown(x);
  }
  void print(int x) {
    if (!x)
      return;
    pushdown(x);
    print(ch[x][0]);
    printf("%d ", x);
    print(ch[x][1]);
  }
  void rotate(int x) {
    int y = fa[x], z = fa[y], chx = getch(x), chy = getch(y);
    fa[x] = z;
    if (!isroot(y))
      ch[z][chy] = x;
    ch[y][chx] = ch[x][chx ^ 1];
    fa[ch[x][chx ^ 1]] = y;
    ch[x][chx ^ 1] = y;
    fa[y] = x;
    up(y);
    up(x);
    if (z)
      up(z);
  }
  void splay(int x) {
    update(x);
    up(x);
    for (int f = fa[x]; f = fa[x], !isroot(x); rotate(x))
      if (!isroot(f))
        rotate(getch(x) == getch(f) ? f : x);
  }
  void access(int x) {
    for (int f = 0; x; f = x, x = fa[x])
      splay(x), ch[x][1] = f, up(x);
  }
```

```
  void makeroot(int x) {
    access(x);
    splay(x);
    tag[x] ^= 1;
    swap(ch[x][0], ch[x][1]);
  }
  int find(int x) {
    access(x);
    splay(x);
    while (ch[x][0])
      x = ch[x][0];
    splay(x);
    return x;
  }
  void link(int x, int y) {
    makeroot(x);
    fa[x] = y;
  }
  void cut(int x, int y) {
    makeroot(x);
    access(y);
    splay(y);
    ch[y][0] = fa[x] = 0;
    up(y);
  }
  void split(int x, int y) {
    makeroot(x);
    access(y);
    splay(y);
  }
#define ls ch[x][0]
#define rs ch[x][1]

  void pushdown(int x) {
    if (tag[x]) {
      if (ch[x][0]) {
        tag[ch[x][0]] ^= 1, swap(ch[ch[x][0]][0], ch[ch[x][0]][1]);
      }
      if (ch[x][1]) {
        tag[ch[x][1]] ^= 1, swap(ch[ch[x][1]][0], ch[ch[x][1]][1]);
      }
      tag[x] = 0;
    }
  }
  void up(int x) {
    if (!x)
      return;
  }
#undef ls
#undef rs
} st;
```

3.16 笛卡尔树

用途：按键值和优先级同时满足二叉搜索树、堆性质的笛卡尔树。

```
vector<pair<int, int>> getcart(vector<int> a) + {
  int n = a.size();
  vector<pair<int, int>> ans(n + 10);
  a.insert(a.begin(), 0);
  vector<int> st;
```

```
  for (int i = 1; i <= n; i++) {
    int last = 0;
    while (st.size() && a[st.back()] > a[i]) {
      last = st.back();
      st.pop_back();
    }
    if (st.size()) {
      ans[st.back()].second = i;
    }
    if (last) {
      ans[i].first = last;
    }
    st.push_back(i);
  }
  ans[0].first = ans[0].second = st[0]; // 根节点 现在为最小值为根节点
  return ans;
}
```

3.17 猫树

用途： 静态区间查询结构。每个线段树节点只预处理“左半段后缀”和“右半段前缀”，查询时找到 `l,r` 所在的最低公共节点，答案就是两段各取一次并合并。它适合只查询、不修改的数组：预处理 $O(n\log n)$，查询 $O(1)$，空间 $O(n\log n)$。下面是区间求和版本，`pre[d][i]` 表示第 d 层中从该分界右侧开始的前缀和，`suf[d][i]` 表示左侧到该位置的后缀和。

例如，令数组为 `[1,2,3,4,5,6,7,8]`，查询区间 `[2,6]`。两端第一次分开的节点是根。预处理得到根层的 `suf[0][2]=2+3+4`、`pre[0][6]=5+6`，所以 `ask(2,6)=9+11=20`。

调用时先 `CatTree tr(a);`，再用 `tr.ask(l,r);`；构造函数中的数组按 1 下标传入。若维护最小值，把补齐元素改为 `INF`，并将代码中的 `+` 换成 `min`。对于非交换运算，请保证调用时 `l<=r` 并按原顺序合并；当前代码中的自动 `swap` 适用于加法、最小值等交换运算。

```
vector<int> a = {0, 1, 2, 3, 4, 5, 6, 7, 8}; // a[0] 占位
CatTree tr(a);
cout << tr.ask(2, 6) << "\n"; // 20
```

```
struct CatTree {
  int n, s, lg;
  vector<int> a, pos;
  vector<vector<int>> pre, suf;

  CatTree(vector<int> b) {
    n = b.size() - 1;
    s = 1;
    while (s < n)
      s <<= 1;
    lg = __lg(s);
    a.assign(s + 1, 0);
    for (int i = 1; i <= n; i++)
      a[i] = b[i];
    pos.resize(s + 1);
    for (int i = 1; i <= s; i++)
      pos[i] = s + i - 1;
    pre.assign(lg, vector<int>(s + 1));
    suf.assign(lg, vector<int>(s + 1));
    for (int d = 0; d < lg; d++) {
      int len = 1 << (lg - d), half = len >> 1;
      for (int l = 1; l <= s; l += len) {
        int m = l + half;
```

```
        suf[d][m - 1] = a[m - 1];
        for (int i = m - 2; i >= l; i--)
          suf[d][i] = a[i] + suf[d][i + 1];
        pre[d][m] = a[m];
        for (int i = m + 1; i < l + len; i++)
          pre[d][i] = pre[d][i - 1] + a[i];
      }
    }
  }

  int ask(int l, int r) {
    if (l > r)
      swap(l, r);
    if (l == r)
      return a[l];
    int d = __lg(pos[l] ^ pos[r]);
    d = lg - 1 - d;
    return suf[d][l] + pre[d][r];
  }
};
```

3.18 区间加与区间和

用途：用两个树状数组或分块实现区间加与区间和。

```
struct fenwick {
  vector<int> c1, c2;
  int n;
  void init(int n) {}
  fenwick(int n) {
    this->n = n;
    c1.resize(n + 10);
    c2.resize(n + 10);
  }
  void add(int x, int d) {
    int k = x;
    while (x <= n) {
      c1[x] += d;
      c2[x] += d * k;
      x += x & -x;
    }
  }
  void rangeadd(int l, int r, int d) {
    add(l, d);
    add(r + 1, -d);
  }
  int gets(int x) {
    int ans = 0;
    int d = x;
    while (x) {
      ans += c1[x] * (d + 1) - c2[x];
      x -= x & -x;
    }
    return ans;
  }
  int getsum(int l, int r) {
    if (l > r)
      return 0;
    return gets(r) - gets(l - 1);
  }
};
```

```
// 分块区间加分
struct blk {
  int B;
  int S;
  int n;
  vector<int> d1, d2, b1, b2;
  blk(int n, int B) {
    this->n = n;
    this->B = B;
    S = (1 << B);
    d1.resize(n + 10);
    d2.resize(n + 10);
    b1.resize(n + 10);
    b2.resize(n + 10);
  }
  void add(int p, int v) {
    if (p > n)
      return;
    d1[p] += v, d2[p] += v * p;
    int bid = p >> B;
    b1[bid] += v, b2[bid] += v * p;
  }
  void rangeadd(int l, int r, int v) {
    add(l, v);
    add(r + 1, -v);
  }
  int gets(int p) {
    if (p <= 0)
      return 0;
    int ans1 = 0, ans2 = 0;
    int q = p;
    while ((q + 1) & (S - 1)) {
      ans1 += d1[q];
      ans2 += d2[q];
      --q;
    }
    if (q >= 0) {
      int bid = q >> B;
      while (bid >= 0) {
        ans1 += b1[bid];
        ans2 += b2[bid];
        --bid;
      }
    }
    return (p + 1) * ans1 - ans2;
  }
  int getsum(int l, int r) {
    if (l > r)
      return 0;
    return gets(r) - gets(l - 1);
  }
};
struct blk {
  int n, B, S;
  vector<int> v, vb;
  blk(int n, int B) {
    this->n = n;
    this->B = B;
    S = (1 << B);
    v.resize(n + 10);
    vb.resize(n + 10);
  }
  void add(int p, int x) {
    v[p] += x;
```

```
    vb[p >> B] += x;
  }
  int gets(int p) {
    int ans = 0;
    while ((p + 1) & (S - 1))
      ans += v[p--];
    if (p < 0)
      return ans;
    p >>= B;
    while (p >= 0)
      ans += vb[p--];
    return ans;
  }
  int getsum(int l, int r) {
    if (l > r)
      return 0;
    return gets(r) - gets(l - 1);
  }
};
```

3.19 珂朵莉树（ODT）

用途：维护连续同值区间，适用于操作次数较少且数据随机的题目。

```
// 注意一定要插入 node(l,r,v) 这种格式不要用 {l,r,v} 会出错
// 注意一定要插入 node(l,r,v) 这种格式不要用 {l,r,v} 会出错
// 注意一定要插入 node(l,r,v) 这种格式不要用 {l,r,v} 会出错
// 注意一定要插入 node(l,r,v) 这种格式不要用 {l,r,v} 会出错
struct node {

  int l, r;
  mutable int v;
  node(int l = 0, int r = 0, int v = 0) {
    this->l = l;
    this->r = r;
    this->v = v;
  }
  bool operator<(struct node a) const { return l < a.l; }
};
set<node> s;
// 注意一定要插入 node(l,r,v) 这种格式不要用 {l,r,v} 会出错
// 注意一定要插入 node(l,r,v) 这种格式不要用 {l,r,v} 会出错
auto split(int pos) {
  auto it = s.lower_bound(node(pos, 0, 0));
  if (it != s.end() && (it->l) == pos)
    return it;
  it--;
  int l = it->l;
  int r = it->r;
  int v = it->v;
  s.erase(it);
  s.insert(node(l, pos - 1, v));
  return s.insert(node(pos, r, v)).first;
}
// 注意一定要插入 node(l,r,v) 这种格式不要用 {l,r,v} 会出错
// 注意一定要插入 node(l,r,v) 这种格式不要用 {l,r,v} 会出错
void change(int l, int r, int x) {
  auto itr = split(r + 1);
  auto itl = split(l);
  // 先 split(r+1) 后 split(l) 防止 RE
  //  因为是 split(r+1) 所以我们可能会存一个 n+1 的不法状态但是不影响
```

```
  for (auto it = itl; it != itr; it++) {
  }
  s.erase(itl, itr);
  s.insert(node(l, r, x));
}
```

3.20　状态压缩 RMQ

用途：用位掩码压缩小范围状态，快速回答静态 RMQ。

```
O(n)
template <class T, class Cmp = std::less<T>> struct RMQ {
  const Cmp cmp = Cmp();
  static constexpr unsigned B = 64;
  using u64 = unsigned long long;
  int n;
  std::vector<std::vector<T>> a;
  std::vector<T> pre, suf, ini;
  std::vector<u64> stk;
  RMQ() {}
  RMQ(const std::vector<T> &v) { init(v); }
  void init(const std::vector<T> &v) {
    n = v.size();
    pre = suf = ini = v;
    stk.resize(n);
    if (!n) {
      return;
    }
    const int M = (n - 1) / B + 1;
    const int lg = std::__lg(M);
    a.assign(lg + 1, std::vector<T>(M));
    for (int i = 0; i < M; i++) {
      a[0][i] = v[i * B];
      for (int j = 1; j < B && i * B + j < n; j++) {
        a[0][i] = std::min(a[0][i], v[i * B + j], cmp);
      }
    }
    for (int i = 1; i < n; i++) {
      if (i % B) {
        pre[i] = std::min(pre[i], pre[i - 1], cmp);
      }
    }
    for (int i = n - 2; i >= 0; i--) {
      if (i % B != B - 1) {
        suf[i] = std::min(suf[i], suf[i + 1], cmp);
      }
    }
    for (int j = 0; j < lg; j++) {
      for (int i = 0; i + (2 << j) <= M; i++) {
        a[j + 1][i] = std::min(a[j][i], a[j][i + (1 << j)], cmp);
      }
    }
    for (int i = 0; i < M; i++) {
      const int l = i * B;
      const int r = std::min(1U * n, l + B);
      u64 s = 0;
      for (int j = l; j < r; j++) {
        while (s && cmp(v[j], v[std::__lg(s) + l])) {
          s ^= 1ULL << std::__lg(s);
        }
        s |= 1ULL << (j - l);
```

```
        stk[j] = s;
      }
    }
  }
  T ask(int l, int r) {
    assert(l <= r);
    l--;
    if (l / B != (r - 1) / B) {
      T ans = std::min(suf[l], pre[r - 1], cmp);
      l = l / B + 1;
      r = r / B;
      if (l < r) {
        int k = std::__lg(r - l);
        ans = std::min({ans, a[k][l], a[k][r - (1 << k)]}, cmp);
      }
      return ans;
    } else {
      int x = B * (l / B);
      return ini[__builtin_ctzll(stk[r - 1] >> (l - x)) + l];
    }
  }
};
RMQ<int, less<int>> q(st);
RMQ<int, greater<int>> q(st);
```

第 4 章

图论与树

> 图上的连通性、路径、流和树上算法。

4.1 树链之外的 dfn 区间（至多 5 段）

用途：借助正向、反向两套 DFS 序，把不在指定树链上的节点表示成至多五个连续区间，再合并相邻或相交区间。

```
// dfn1 按正序遍历儿子，dfn2 按倒序遍历儿子。
vector<pair<int,int>> query(int x, int y) {
    vector<pair<int,int>> res;
    auto put = [&](int l, int r) {
        if (l <= r) res.emplace_back(l, r);
    };
    if (dfn1[x] > dfn1[y])
        swap(x, y);
    int w = lca(x, y);
    if (x == y) {
        put(1, dfn1[x] - 1);
        put(dfn1[x] + 1, n);
    }
    else if (w == x) {
        put(1, dfn1[x] - 1);
        put(dfn1[y] + 1, n);
        put(dfn2[y] + 1, n);
    }
    else {
        int z = jump(y, dep[y] - dep[w] - 1);
        put(1, dfn1[w] - 1);
        put(dfn2[x] + 1, n);
        put(dfn1[y] + 1, n);
        put(dfn1[x] + 1, dfn1[z] - 1);
        put(dfn2[y] + 1, end2[z]);
    }
    return res;
}
vector<pair<int,int>> mergeSeg(vector<pair<int,int>> a) {
    sort(a.begin(), a.end());
    vector<pair<int,int>> res;
    for (auto [l, r] : a) {
        if (l > r) continue;
        if (res.empty() || l > res.back().second + 1)
            res.push_back({l, r});
        else
            res.back().second = max(res.back().second, r);
    }
    return res;
}
```

4.2 虚树

用途：只保留关键点及其必要 LCA，压缩树上状态转移规模。

```
auto bvt = [&](vector<int> key) {
  sort(key.begin(), key.end(), [&](int x, int y) { return dfn[x] < dfn[y]; });
  vector<int> a;
  for (int i = 0; i < (int)key.size() - 1; ++i) {
    a.push_back(key[i]);
    a.push_back(LCA(key[i], key[i + 1]).first);
  }
  a.push_back(key.back());
  sort(a.begin(), a.end(), [&](int x, int y) { return dfn[x] < dfn[y]; });
  a.erase(unique(a.begin(), a.end()), a.end());
  vector<pair<int, int>> ask;
  for (int i = 0; i < (int)a.size() - 1; i++) {
    int lc = LCA(a[i], a[i + 1]).first;
    ask.push_back({lc, a[i + 1]});
  }
  return ask;
};
```

4.3 树上最近公共祖先（LCA）

用途：给出倍增、重链剖分和 Euler 序 RMQ 等 LCA 实现。

```
ll LCA(ll u, ll v) {
  if (dis[u] < dis[v])
    swap(u, v);
  per(i, 18, 0) if (dis[fa[u][i]] >= dis[v]) u = fa[u][i];
  if (u == v)
    return u;
  per(i, 18, 0) if (fa[u][i] != fa[v][i]) u = fa[u][i], v = fa[v][i];
  return fa[u][0];
}

struct HLD {
  int n;
  vector<int> sz, top, de, fa, id, seq, son, dfn;
  vector<vector<int>> g;
  int cur = 0;
  int cnt = 0;
  HLD(int n) {
    this->n = n;
    sz.resize(n + 1);
    top.resize(n + 1);
    fa.resize(n + 1);
    de.resize(n + 1);
    dfn.resize(n + 1);
    id.resize(n + 1);
    seq.resize(n + 1);
    son.resize(n + 1);
    g.assign(n + 1, {});
  }
  void add(int x, int y) { g[x].push_back(y); };
  void dfs1(int u, int f) {
    sz[u] = 1;
    int maxx = 0;
    fa[u] = f;
    dfn[u] = ++cnt;
```

```
    de[u] = de[f] + 1;
    for (auto it : g[u]) {
      if (it == f)
        continue;
      dfs1(it, u);
      sz[u] += sz[it];
      if (sz[it] > maxx) {
        son[u] = it;
        maxx = sz[it];
      }
    }
  }
  void dfs2(int u, int f, int tp) {
    id[u] = ++cur;
    top[u] = tp;
    seq[cur] = u;
    if (son[u])
      dfs2(son[u], u, tp);
    for (auto it : g[u]) {
      if (it == f || it == son[u])
        continue;
      dfs2(it, u, it);
    }
  };
  void work(int root) {
    dfs1(root, 0);
    dfs2(root, 0, root);
  }
  int lca(int x, int y) {
    while (top[x] != top[y]) {
      if (de[top[x]] < de[top[y]])
        swap(x, y);
      x = fa[top[x]];
    }
    return de[x] < de[y] ? x : y;
  }
  int dis(int x, int y) { return de[x] + de[y] - 2 * de[lca(x, y)]; }
  bool isson(int x, int y) {
    return dfn[x] >= dfn[y] && dfn[y] + sz[y] - 1 >= dfn[x];
  }
  int jump(int u, int k) {
    if (de[u] <= k)
      return -1;
    int d = de[u] - k;
    while (de[top[u]] > d) {
      u = fa[top[u]];
    }
    return seq[id[u] - (de[u] - d)];
  }
};
struct HLD {
  int n;
  std::vector<int> siz, top, dep, parent, in, out, seq;
  std::vector<std::vector<int>> adj;
  int cur;

  HLD() {}
  HLD(int n) { init(n); }
  void init(int n) {
    this->n = n;
    siz.resize(n);
    top.resize(n);
    dep.resize(n);
    parent.resize(n);
```

```
  in.resize(n);
  out.resize(n);
  seq.resize(n);
  cur = 0;
  adj.assign(n, {});
}
void addEdge(int u, int v) {
  adj[u].push_back(v);
  adj[v].push_back(u);
}
void work(int root = 0) {
  top[root] = root;
  dep[root] = 0;
  parent[root] = -1;
  dfs1(root);
  dfs2(root);
}
void dfs1(int u) {
  if (parent[u] != -1) {
    adj[u].erase(std::find(adj[u].begin(), adj[u].end(), parent[u]));
  }

  siz[u] = 1;
  for (auto &v : adj[u]) {
    parent[v] = u;
    dep[v] = dep[u] + 1;
    dfs1(v);
    siz[u] += siz[v];
    if (siz[v] > siz[adj[u][0]]) {
      std::swap(v, adj[u][0]);
    }
  }
}
void dfs2(int u) {
  in[u] = cur++;
  seq[in[u]] = u;
  for (auto v : adj[u]) {
    top[v] = v == adj[u][0] ? top[u] : v;
    dfs2(v);
  }
  out[u] = cur;
}
int lca(int u, int v) {
  while (top[u] != top[v]) {
    if (dep[top[u]] > dep[top[v]]) {
      u = parent[top[u]];
    } else {
      v = parent[top[v]];
    }
  }
  return dep[u] < dep[v] ? u : v;
}

int dist(int u, int v) { return dep[u] + dep[v] - 2 * dep[lca(u, v)]; }

int jump(int u, int k) {
  if (dep[u] < k) {
    return -1;
  }

  int d = dep[u] - k;

  while (dep[top[u]] > d) {
    u = parent[top[u]];
```

```
    }

    return seq[in[u] - dep[u] + d];
  }

  bool isAncester(int u, int v) { return in[u] <= in[v] && in[v] < out[u]; }
  int rootedParent(int u, int v) {
    std::swap(u, v);
    if (u == v) {
      return u;
    }
    if (!isAncester(u, v)) {
      return parent[u];
    }
    auto it = std::upper_bound(adj[u].begin(), adj[u].end(), v,
                               [&](int x, int y) { return in[x] < in[y]; }) -
              1;
    return *it;
  }
  int rootedSize(int u, int v) {
    if (u == v) {
      return n;
    }
    if (!isAncester(v, u)) {
      return siz[v];
    }
    return n - siz[rootedParent(u, v)];
  }

  int rootedLca(int a, int b, int c) {
    return lca(a, b) ^ lca(b, c) ^ lca(c, a);
  }
};
// 注意 st 表 struct HLD {
vector<vector<int>> g;
vector<int> dfn, de, seq;
rmq<pair<int, int>> q;
int n = 0;
int cnt = 0;
HLD(int n) {
  this->n = n;
  g.resize(n + 10);
  dfn.resize(2 * n + 10);
  de.resize(n + 10);
  seq.resize(2 * n + 10);
}
void add(int x, int y) { g[x].push_back(y); }
void dfs(int u, int f, int d) {
  dfn[u] = ++cnt;
  seq[cnt] = u;
  de[u] = d;
  for (auto it : g[u]) {
    if (it == f)
      continue;
    dfs(it, u, d + 1);
    seq[++cnt] = u;
  }
}
void work(int rt) {
  dfs(rt, -1, 1);
  vector<pair<int, int>> a;
  for (int i = 1; i <= 2 * n - 1; i++) {
    a.push_back({de[seq[i]], seq[i]});
  }
```

```
  q.work(a, [&](auto x, auto y) { return min(x, y); });
}
int ask(int a, int b) {
  int l = dfn[a];
  int r = dfn[b];
  if (l > r)
    swap(l, r);
  return q.ask(l, r).second;
}
}
;
#include <bits/stdc++.h>
using namespace std;
constexpr int N = 5e5 + 5;
int n, m, R, dn, dfn[N], mi[19][N];
vector<int> e[N];
int get(int x, int y) { return dfn[x] < dfn[y] ? x : y; }
void dfs(int id, int f) {
  mi[0][dfn[id] = ++dn] = f;
  for (int it : e[id])
    if (it != f)
      dfs(it, id);
}
int lca(int u, int v) {
  if (u == v)
    return u;
  if ((u = dfn[u]) > (v = dfn[v]))
    swap(u, v);
  int d = __lg(v - u++);
  return get(mi[d][u], mi[d][v - (1 << d) + 1]);
}
int main() {
  scanf("%d %d %d", &n, &m, &R);
  for (int i = 2, u, v; i <= n; i++) {
    scanf("%d %d", &u, &v);
    e[u].push_back(v), e[v].push_back(u);
  }
  dfs(R, 0);
  for (int i = 1; i <= __lg(n); i++)
    for (int j = 1; j + (1 << i) - 1 <= n; j++)
      mi[i][j] = get(mi[i - 1][j], mi[i - 1][j + (1 << i - 1)]);
  for (int i = 1, u, v; i <= m; i++)
    scanf("%d %d", &u, &v), printf("%d\n", lca(u, v));
  return 0;
}
```

4.4 链式前向星

用途：用连续数组存储邻接表，适合边数大、追求常数的图算法。

```
const int N = 1e6;
int h[N], nex[N], v[N], w[N];
int idx;
int dis[N];
void add(int x, int y, int s) {
  v[++idx] = y;
  w[idx] = s;
  nex[idx] = h[x];
  h[x] = idx;
}
queue<int> q q.push(1);
```

```
dis[1] = 0;
while (q.size()) {
  int u = q.front();
  q.pop();
  for (int i = h[u]; i; i = nex[i]) {
    int j = v[i];
    if (dis[j] < dis[u] + w[i]) {
      dis[j] = dis[u] + w[i];
      q.push(j);
    }
  }
}
```

4.5 强连通分量（SCC）

用途： Tarjan 求强连通分量，并可进一步缩点为 DAG。

```
struct scc {
  vector<int> dfn, vis, low, st;
  vector<int> cnt;
  int n;
  int sz = 0;
  int idx = 0;
  vector<vector<int>> g;
  vector<int> id;
  scc() {}
  int size() { return sz; }
  scc(int n) {
    this->n = n;
    g.resize(n + 10);
    cnt.resize(n + 10);
    dfn.resize(n + 10);
    low.resize(n + 10);
    vis.resize(n + 10);
    id.resize(n + 10);
  }
  void add(int x, int y) { g[x].push_back(y); }
  void dfs(int u, int f) {
    if (dfn[u])
      return;
    dfn[u] = low[u] = ++idx;
    vis[u] = 1;
    st.push_back(u);
    for (auto it : g[u]) {
      // if(it==f)continue; 如果求无向图割边加上这一句
      if (!dfn[it]) {
        dfs(it, u);
        low[u] = min(low[u], low[it]);
      } else if (vis[it]) {
        low[u] = min(low[u], dfn[it]);
      }
    }
    if (dfn[u] == low[u]) {
      sz++;
      while (1) {
        int temp = st.back();
        st.pop_back();
        id[temp] = sz;
        cnt[sz]++;
        vis[temp] = 0;
        if (temp == u)
```

```
          break;
      }
    }
  }
  void work(int root) { dfs(root, -1); }
  vector<vector<int>> newg() {
    vector<vector<int>> rg(n + 10);
    map<pair<int, int>, int> mp;
    for (int i = 1; i <= n; i++) {
      for (auto it : g[i]) {
        if (id[it] == id[i])
          continue;
        if (mp.count({id[i], id[it]}))
          continue;
        rg[id[i]].push_back(id[it]);
        mp[{id[i], id[it]}] = 1;
      }
    }
    return rg;
  }
};
struct Tarjan { // 下面都是无向图中
  // incblock 去掉这个点增加的联通块个数。(如果他是孤立点联通快个数反而会减少)
  // (孤立点不算割点)
  vector<int> low, dfn, stk, incblock, h, v, nex, vis;
  vector<int> bri; //   是不是桥
  vector<int> cut; // 是不是割点
  int idx = 0, total = 0, top = 0;
  void init(int n, int m) {
    h.resize(n + 10, -1);
    v.resize(m + 10);
    nex.resize(m + 10, -1);
    bri.resize(m + 10);
    vis.resize(n + 10);
    low.resize(n + 10);
    dfn.resize(n + 10);
    stk.resize(n + 10);
    cut.resize(n + 10);
    incblock.resize(n + 10);
  }
  void add(int x, int y) {
    v[total] = y;
    nex[total] = h[x];
    h[x] = total++;
  }
  void work(int u, int fa) {
    low[u] = dfn[u] = ++idx;
    stk[top++] = u;
    vis[u] = 1;
    int son = 0;

    for (int i = h[u]; i != -1; i = nex[i]) {
      int j = v[i];
      if (j == fa)
        continue;
      if (!dfn[j]) {
        son++;
        work(j, u);
        if (low[u] > low[j])
          low[u] = low[j];
        if (low[j] > dfn[u]) {
          bri[i] = 1;
          bri[i ^ 1] = 1;
        }
```

```
        if (u != fa && low[j] >= dfn[u]) {
          cut[u] = 1;
          incblock[u]++;
        }
      } else if (low[u] > dfn[j]) {
        low[u] = dfn[j];
      }
    }
    if (u == fa && son > 1) {
      cut[u] = 1;
    }
    if (u == fa) {
      incblock[u] = son - 1;
    }
    vis[u] = false;
    top--;
  }
} tar;
// 圆方树 <=n 的是原点
pair<vector<vector<int>>, int> gettree(vector<vector<int>> &g, int n) {
  vector<int> dfn(n + 10), low(n + 10);
  int idx = 0;
  vector<int> st;
  vector<vector<int>> tg(2 * n + 10);
  int cnt = n;
  auto dfs = [&](auto dfs, int u) -> void {
    if (dfn[u])
      return;
    dfn[u] = low[u] = ++idx;
    st.push_back(u);
    for (auto it : g[u]) {
      if (!dfn[it]) {
        dfs(dfs, it);
        low[u] = min(low[u], low[it]);
        if (low[it] == dfn[u]) {
          ++cnt;
          int temp;
          do {
            temp = st.back();
            st.pop_back();
            tg[cnt].push_back(temp);
            tg[temp].push_back(cnt);
          } while (temp != it);
          tg[cnt].push_back(u);
          tg[u].push_back(cnt);
        }
      } else
        low[u] = min(low[u], dfn[it]);
    }
  };
  for (int i = 1; i <= n; i++)
    dfs(dfs, i);

  return {tg, cnt};
}
```

4.6　2-sat

```
 #include<bits/stdc++.h>
using namespace std;
mt19937_64 rng(time(0));
const int mod=1e9+7;
```

```
struct scc{
 vector<int>dfn,vis,low,st;
 vector<int>cnt;
 int n;
 int sz=0;
 int idx=0;
 vector<vector<int>>g;
 vector<int>id;
 scc(){
 }
 void init(int n){
 this->n=n;
 g.resize(n+10);
 cnt.resize(n+10);
 dfn.resize(n+10);
 low.resize(n+10);
 vis.resize(n+10);
 id.resize(n+10);
 }
 int size(){
 return sz;
 }
 scc(int n){
    init(n);
 }
 void add(int x,int y){
 g[x].push_back(y);
 }
 void dfs(int u,int f){
 if(dfn[u])return;
 dfn[u]=low[u]=++idx;
 vis[u]=1;
 st.push_back(u);
 for(auto it:g[u]){
 // if(it==f)continue; 如果求无向图割边加上这一句
 if(!dfn[it]){
 dfs(it,u);
 low[u]=min(low[u],low[it]);
 }
 else if(vis[it]) {
 low[u]=min(low[u],dfn[it]);
 }
 }
 if(dfn[u]==low[u]){
 sz++;
 while(1){
 int temp=st.back();st.pop_back();
 id[temp]=sz;
 cnt[sz]++;
 vis[temp]=0;
 if(temp==u)break;
 }
 }
 }
 void work(int root){
 dfs(root,-1);
 }
 vector<vector<int>>newg(){
 vector<vector<int>>rg(n+10);
 map<pair<int,int>,int>mp;
 for(int i=1;i<=n;i++){
 for(auto it:g[i]){
 if(id[it]==id[i])continue;
 if(mp.count({id[i],id[it]}))continue;
```

```
 rg[id[i]].push_back(id[it]);
 mp[{id[i],id[it]}]=1;
 }
 }
 return rg;
 }
};
vector<vector<int>>g(2e6);
int tr[(int)2e6][26];
void slove(){
    // memset(tr,0,sizeof(tr));
    // for(int i=0;i<1e6;i++)g[i].clear();
    // cerr<<"************************************\n";
    int n;cin>>n;
    int idx=0;
    int idx1=0;
    // cerr<<tr[0][0]<<"\n";
    auto insert=[&](string s,int ix){
        int p=0;
        for(auto it:s){
            // cerr<<s<<" "<<p<<" "<<tr[p][it-'a']<<" "<<idx1<<"\n";
            if(!tr[p][it-'a'])tr[p][it-'a']=++idx1;
            p=tr[p][it-'a'];
        }
        g[p].push_back(ix);
    };
    int all=0;
    for(int i=1;i<=n;i++){
        string s1,s2;
        cin>>s1>>s2;
        insert(s1,++idx);
        all+=s1.size();
        all+=s2.size();
        insert(s2,++idx);
    }
    all+=4*n;
    all+=10;
    scc scc(all*2+2);
auto neg = [&](int x) {
    return x <= all ? x + all : x - all;
};
auto addOr = [&](int a, int b) {
    scc.add(neg(a), b); // !a -> b
    scc.add(neg(b), a); // !b -> a
};
auto forbidState = [&](int x, bool xv, int y, bool yv) {
    int a = xv ? neg(x) : x;
    int b = yv ? neg(y) : y;
    addOr(a, b);
};
// 1. 只允许 00。
// x 必须不存在，并且 y 必须不存在。
auto only00 = [&](int x, int y) {
    forbidState(x, 0, y, 1); // 禁止 01：不允许只有 y 存在。
    forbidState(x, 1, y, 0); // 禁止 10：不允许只有 x 存在。
    forbidState(x, 1, y, 1); // 禁止 11：不允许两者都存在。
};
// 2. 只允许 01。
// x 必须不存在，并且 y 必须存在。
auto only01 = [&](int x, int y) {
    forbidState(x, 0, y, 0); // 禁止 00：y 不能不存在。
    forbidState(x, 1, y, 0); // 禁止 10：x 不能存在且 y 不存在。
    forbidState(x, 1, y, 1); // 禁止 11：x 不能存在。
};
```

```
// 3. 只允许 10。
// x 必须存在，并且 y 必须不存在。
auto only10 = [&](int x, int y) {
    forbidState(x, 0, y, 0); // 禁止 00: x 不能不存在。
    forbidState(x, 0, y, 1); // 禁止 01: x 不能不存在且 y 存在。
    forbidState(x, 1, y, 1); // 禁止 11: y 不能存在。
};

// 4. 只允许 11。
// x 必须存在，并且 y 必须存在。
auto only11 = [&](int x, int y) {
    forbidState(x, 0, y, 0); // 禁止 00: 不能两者都不存在。
    forbidState(x, 0, y, 1); // 禁止 01: x 不能不存在。
    forbidState(x, 1, y, 0); // 禁止 10: y 不能不存在。
};

// 5. 只允许 00、01。
// 无论 y 是否存在，x 都必须不存在。
auto allow00_01 = [&](int x, int y) {
    forbidState(x, 1, y, 0); // 禁止 10: x 存在、y 不存在。
    forbidState(x, 1, y, 1); // 禁止 11: x 存在、y 存在。
};

// 6. 只允许 00、10。
// 无论 x 是否存在，y 都必须不存在。
auto allow00_10 = [&](int x, int y) {
    forbidState(x, 0, y, 1); // 禁止 01: x 不存在、y 存在。
    forbidState(x, 1, y, 1); // 禁止 11: x 存在、y 存在。
};

// 7. 只允许 00、11。
// x、y 的存在状态必须相同。
// 两者要么都不存在，要么都存在。
auto allow00_11 = [&](int x, int y) {
    forbidState(x, 0, y, 1); // 禁止 01: 不允许 x 不存在但 y 存在。
    forbidState(x, 1, y, 0); // 禁止 10: 不允许 x 存在但 y 不存在。
};

// 8. 只允许 01、10。
// x、y 中恰好一个存在。
// 两者不能都不存在，也不能都存在。
auto allow01_10 = [&](int x, int y) {
    forbidState(x, 0, y, 0); // 禁止 00: 至少有一个必须存在。
    forbidState(x, 1, y, 1); // 禁止 11: 最多有一个可以存在。
};

// 9. 只允许 01、11。
// 无论 x 是否存在，y 都必须存在。
auto allow01_11 = [&](int x, int y) {
    forbidState(x, 0, y, 0); // 禁止 00: y 不允许不存在。
    forbidState(x, 1, y, 0); // 禁止 10: y 不允许不存在。
};

// 10. 只允许 10、11。
// 无论 y 是否存在，x 都必须存在。
auto allow10_11 = [&](int x, int y) {
    forbidState(x, 0, y, 0); // 禁止 00: x 不允许不存在。
    forbidState(x, 0, y, 1); // 禁止 01: x 不允许不存在。
};

// 11. 只允许 00、01、10。
// x、y 不能同时存在。
// 等价于：x 存在，则 y 必须不存在。
// 也等价于：y 存在，则 x 必须不存在。
```

```
auto allow00_01_10 = [&](int x, int y) {
    forbidState(x, 1, y, 1); // 禁止 11：不能同时存在。
};

// 12. 只允许 00、01、11。
// x 存在，则 y 一定存在。
// 等价于：y 不存在，则 x 一定不存在。
auto allow00_01_11 = [&](int x, int y) {
    forbidState(x, 1, y, 0); // 禁止 10：不允许 x 存在但 y 不存在。
};

// 13. 只允许 00、10、11。
// x 不存在，则 y 一定不存在。
// 等价于：y 存在，则 x 一定存在。
auto allow00_10_11 = [&](int x, int y) {
    forbidState(x, 0, y, 1); // 禁止 01：不允许 x 不存在但 y 存在。
};
// 14. 只允许 01、10、11。
// x、y 至少一个存在。
// 等价于：x 不存在，则 y 一定存在。
// 也等价于：y 不存在，则 x 一定存在。
auto allow01_10_11 = [&](int x, int y) {
    forbidState(x, 0, y, 0); // 禁止 00：不能同时不存在。
};
    for(int i=1;i<=n;i++){
        allow01_10(i*2-1,2*i);// 恰好存在一个
    }
    int ed=idx;
    auto dfs=[&](auto dfs,int u,int f)->void{
        for(int i=0;i<26;i++){
            if(tr[u][i]){
//                 cerr<<ed*2+5+u<<" "<<ed*2+5+tr[u][i]<<"\n";
                allow00_01_11(ed*2+5+u,ed*2+5+tr[u][i]); //x 存在那 y 一定存在
                dfs(dfs,tr[u][i],u);
            }
        }
        if(g[u].size()){
            int len=g[u].size();
            for(int i=1;i<len;i++){
                // pre=idx+i
                // add1(idx+i,idx+i+1);
                allow00_01_11(idx+i,idx+i+1); //x 存在那 y 一定存在

            }
            for(int i=1;i<=len;i++){
                allow00_01_11(g[u][i-1],idx+i);
                if(i>=2){
                    //x 存在则 y 一定不存在
                    allow00_01_10(g[u][i-1],idx+i-1);
                }
            }
            //idx+len
            allow00_01_11(idx+len,ed*2+5+u); //x 存在那 y 一定存在
               //x 存在则 y 一定不存在
                 if(f!=-1)
                allow00_01_10(idx+len,ed*2+5+f);
            idx+=len;
        }
    };dfs(dfs,0,-1);
    for(int i=1;i<=2*all;i++){
        scc.work(i);
    }
    vector<int>ans(all+2);
    for(int i=1;i<=all;i++){
```

```
        if(scc.id[i]==scc.id[i+all]){
            cout<<"NO\n";
    for(int i=0;i<=idx1;i++)for(int j=0;j<=25;j++)tr[i][j]=0;
    for(int i=0;i<=idx1;i++)g[i].clear();
            return;
        }
        if(scc.id[i]>scc.id[i+all]){
            ans[i]=0;
        }
        else ans[i]=1;
    }
    cout<<"YES\n";
    for(int i=2;i<=2*n;i+=2){
        if(ans[i-1]==1){
            cout<<"l";
        }
        else cout<<"r";
    }
    cout<<"\n";
    for(int i=0;i<=idx1;i++)for(int j=0;j<=25;j++)tr[i][j]=0;
    for(int i=0;i<=idx1;i++)g[i].clear();
}
signed main(){
    ios::sync_with_stdio(0);cin.tie(0);
    int t=1;
    cin>>t;
    while(t--)
    slove();
    return 0;
}
```

4.7 线段树优化建图

用途：用两棵有向线段树批量表示“点到区间”和“区间到点”的有向边，避免为一个区间逐点连边。out 树的边由父节点指向子节点，in 树的边由子节点指向父节点；每个原点 x 同时有 in[x] 和 out[x] 两个零代价副本，并在叶子处互连。

怎么连边：ask 把 [l,r] 分解成 $O(\log n)$ 个线段树节点并放进 all。addx_range(x,l,r,w) 对每个分解节点连 in[x] -> out[节点]，再沿 out 树向下就能到达区间内所有点；addrange_x(l,r,x,w) 连 in[节点] -> out[x]，任意区间内的点沿 in 树向上即可到达 x；addxy 则是普通的点到点边。

建图后照常跑最短路：边权非负用 Dijkstra，源点从 in[s]（或按题意使用对应副本）开始，终点读取 out[t] 的距离。若有负边，需改用能处理负边且无负环的算法。设原图有 K 条批量边，建图时间和边数为 $O(n + K \log n)$，结构点和空间为 O(n+K log n)；Dijkstra 的复杂度为 $O((V + E) \log V)$。

例如 $n = 8$ 时，给点 3 向 [2,6] 连权 5 的边，ask 可能分成 [2,2]、[3,4]、[5,6]，只需连 3 条边，树上的下行零边会覆盖 2 到 6；给 [2,6] 向点 7 连权 2 的边则反向使用 in 树即可。

在定义并完成 qbu 建图后，最短路部分就是普通 Dijkstra（下面假设所有边权非负）：

```
qbu g(n);
int source = 1, target = n;
g.addxy(1, 2, 3);
g.addx_range(3, 2, 6, 5);
g.addrange_x(2, 6, 7, 2);
vector<long long> dis(8 * n + 5, (long long)4e18);
priority_queue<pair<long long, int>, vector<pair<long long, int>>,
               greater<pair<long long, int>>>
    pq;
dis[g.in[source]] = 0;
pq.push({0, g.in[source]});
```

```
while (!pq.empty()) {
  auto [d, u] = pq.top();
  pq.pop();
  if (d != dis[u])
    continue;
  for (auto [x, y, w] : g.allg) { // 实战中应先按 x 建邻接表
    if (x == u && dis[y] > d + w)
      dis[y] = d + w, pq.push({dis[y], y});
  }
}
cout << dis[g.out[target]] << "\n";
```

上例为了直观直接扫描 `allg`；正式代码应把边整理成邻接表，否则 Dijkstra 会变慢。

```
struct qbu {
  int n;
  vector<int> in, out;
  int ad = 0;
  vector<array<int, 3>> allg;
  void add(int x, int y, int w) { allg.push_back({x, y, w}); }
  void bu(int i, int l, int r) {
    if (l == r) {
      in[l] = i + 4 * n;
      out[l] = i;
      add(i, i + ad, 0);
      add(i + ad, i, 0);
      return;
    }
    int mid = (l + r) / 2;
    bu(i * 2, l, mid);
    bu(i * 2 + 1, mid + 1, r);
    add(i, i * 2, 0);
    add(i, i * 2 + 1, 0);
    add(i * 2 + ad, i + ad, 0);
    add(i * 2 + 1 + ad, i + ad, 0);
  }
  qbu(int n) {
    this->n = n;
    in.resize(n + 10);
    out.resize(n + 10);
    ad = 4 * n;
    bu(1, 1, n);
  }
  void ask(int i, int l, int r, int st, int ed) {
    if (st >= l && ed <= r) {
      all.push_back(i);
      return;
    }
    int mid = (st + ed) / 2;
    if (l <= mid)
      ask(i * 2, l, r, st, mid);
    if (r > mid)
      ask(i * 2 + 1, l, r, mid + 1, ed);
  }
  vector<int> all;
  // x 向区间 [l,r] 连 边权为 w 的边。
  void addx_range(int x, int l, int r, int w) {
    ask(1, l, r, 1, n);
    for (auto it : all)
      add(in[x], it, w);
    all.clear();
  }
  /// x 点向 y 点连 w 的边
```

```
  void addxy(int x, int y, int w) { add(in[x], out[y], w); }
  // [l,r] 向 x 连边 w
  void addrange_x(int l, int r, int x, int w) {
    ask(1, l, r, 1, n);
    for (auto it : all) {
      add(it + ad, out[x], w);
    }
    all.clear();
  }
};
```

4.8 树链剖分

用途：把树上路径拆成 O(log N) 个连续区间，再配合线段树维护路径信息。

```
vector<int> fa(n + 10), sz(n + 10), top(n + 10), id(n + 10), de(n + 10),
    son(n + 10);
int idx = 0;
auto dfs1 = [&](auto dfs1, int u, int f) -> void {
  fa[u] = f;
  de[u] = de[f] + 1;
  sz[u] = 1;
  int maxx = 0;
  for (auto it : g[u]) {
    if (it == f)
      continue;
    dfs1(dfs1, it, u);
    sz[u] += sz[it];
    if (sz[it] > maxx) {
      maxx = sz[it];
      son[u] = it;
    }
  }
};
auto dfs2 = [&](auto dfs2, int u, int f) -> void {
  top[u] = f;
  id[u] = ++idx;
  if (son[u])
    dfs2(dfs2, son[u], f);
  for (auto it : g[u]) {
    if (it == fa[u] || it == son[u])
      continue;
    dfs2(dfs2, it, it);
  }
};
dfs1(dfs1, 1, 0);
dfs2(dfs2, 1, 1);
// 随便放的一个函数格式
auto add = [&](int x, int y, int k) {
  while (top[x] != top[y]) {
    if (de[top[x]] < de[top[y]])
      swap(x, y);
    seg.modify(id[top[x]], id[x], tag{k});
    x = fa[top[x]];
  }
  if (de[x] < de[y])
    swap(x, y);
  seg.modify(id[y], id[x], tag{k});
};
```

4.9　最大流与最小割

用途： Dinic 最大流模板，并附带最小割和若干闭合图、匹配结论。

```
// 链 ： D 中的一个子集 C 满足 C 是全序集 及 C 中所有元素都可以比较大小

// 反链 ： D 中的一个子集 B 满足 B 中任意非空子集都不是全序集
// 即所有元素之间都不可以比较大小

// 链覆盖 ： 若干个链的并集为 D ，且两两之间交集为 ∅

// 反链覆盖 ： 若干个反链的并集为 D ，且两两之间交集为 ∅

// 最长链 ： 所有链中元素个数最多的 (可以有多个最长链)

// 最长反链 ： 所有反链中元素个数最多的 (可以有多个最长反链

// 偏序集高度 ： 最长链的元素个数
// 偏序集宽度 ： 最长反链中的元素个数

// 最小链覆盖（使链最少）= 最长反链长度 = 偏序集宽度
// 最小反链覆盖 = 最长链长度 = 偏序集深度

// DAG 中，有如下的一些定义和性质：

// 链：一条链是一些点的集合，链上任意两个点 x，y，满足要么 x 能到达 y ，要么
// y 能到达 x 。

// 反链：一条反链是一些点的集合，链上任意两个点 x，y，满足 x 不能到达 y，且 y
// 也不能到达 x。

// 一个定理：最长反链长度 = 最小链覆盖（用最少的链覆盖所有顶点）

// 对偶定理：最长链长度 = 最小反链覆盖

// 那么我们要求出的就是这个有向无环图的最小链覆盖了。
// 最小链覆盖即路径可以相交的最小路径覆盖。

// 闭合图
// 首先，先了解什么是闭合图。闭合图一般指一个图中点的集合，从该集合中所有的点出发，能到达的点要求都必须在该点集中。也就是说，从该集合中出发，一定要回到该集合中，不能出到集合外。

// 最大权闭合子图，顾名思义，就是所有闭合子图中点权之和最大的那个，注意这里的权指的是点权，因为闭合图是对于点集而言的。

// 最大权闭合子图
// 了解完概念后，要知道如何求最大权闭合子图。大体的思路是借助最小割模型，让每一种闭合图通过变形后都能和一种割相对应，这样求最大权就是求最小割。

// 具体做法
// 构造一个新的流网络，建一个源点 s 和汇点 t，从 s 向原图中所有点权为正数的点建一条容量为点权的边，从点权为负数的点向 t 建一条容量为点权绝对值的边，原图中各点建的边都建成容量为正无穷的边。然后求从 s 到 t 的最小割，再用所有点权为正的权值之和减去最小割，就是我们要求的最大权值和了。

// 最大独立集：在N个点的图 G 中选出 m 个点，使这 m 个点两两之间没有边的点中，m 的最大值。
// 结论：二分图的最大点独立数 = 点的个数-最小点覆盖数（最大匹配）

// 最小边覆盖：
// 边覆盖的概念定义：
// 边覆盖是图的一个边子集，使该图上每一节点都与这个边子集中的一条边关联，只有含孤立点的图没有边覆盖，边覆盖也称为边覆盖集，图 G 的最小边覆盖就是指边数最少的覆盖，图 G 的最小边覆盖的边数称为 G 的边覆盖数。

// 普通图 的最小边覆盖好像也没有什么除了暴力好的解法，自己菜（逃
```

```
// 结论：
// 二分图的最小边覆盖数 = 图中的顶点数-（最小点覆盖数）该二分图的最大匹配数

// 最小点覆盖：
// 点覆盖的概念定义：
// 对于图 G=(V,E) 中的一个点覆盖是一个集合 S⊆V 使得每一条边至少有一个端点在 S 中。

// 最小点覆盖：就是中点的个数最少的 S 集合。
// 普通图的最小点覆盖数好像只能用搜索解，没有什么比较好的方法（可能我比较弱。。）所以在此只讨论二分图的最小点覆盖的求法

// 结论：
// 二分图的最小点覆盖数 = 该二分图的最大匹配数，具体证明的方法看大佬博客，里面还给出了如何求具体的最小覆盖的点是哪些点。

template <class T> struct MaxFlow {
  struct _Edge {
    int to;
    T cap;
    _Edge(int to, T cap) : to(to), cap(cap) {}
  };
  int n;
  std::vector<_Edge> e;
  std::vector<std::vector<int>> g;
  std::vector<int> cur, h;
  MaxFlow() {}
  MaxFlow(int n) { init(n); }
  void init(int n) {
    this->n = n;
    e.clear();
    g.assign(n, {});
    cur.resize(n);
    h.resize(n);
  }
  bool bfs(int s, int t) {
    h.assign(n, -1);
    std::queue<int> que;
    h[s] = 0;
    que.push(s);
    while (!que.empty()) {
      const int u = que.front();
      que.pop();
      for (int i : g[u]) {
        auto [v, c] = e[i];
        if (c > 0 && h[v] == -1) {
          h[v] = h[u] + 1;
          if (v == t) {
            return true;
          }
          que.push(v);
        }
      }
    }
    return false;
  }
  T dfs(int u, int t, T f) {
    if (u == t) {
      return f;
    }
    auto r = f;
    for (int &i = cur[u]; i < (int)g[u].size(); ++i) {
      const int j = g[u][i];
      auto [v, c] = e[j];
      if (c > 0 && h[v] == h[u] + 1) {
        auto a = dfs(v, t, std::min(r, c));
```

```
        e[j].cap -= a;
        e[j ^ 1].cap += a;
        r -= a;
        if (r == 0) {
          return f;
        }
      }
    }
    return f - r;
  }
  void addEdge(int u, int v, T c) {
    g[u].push_back(e.size());
    e.emplace_back(v, c);
    g[v].push_back(e.size());
    e.emplace_back(u, 0);
  }
  T flow(int s, int t) {
    T ans = 0;
    while (bfs(s, t)) {
      cur.assign(n, 0);
      ans += dfs(s, t, std::numeric_limits<T>::max());
    }
    return ans;
  }
  std::vector<bool> minCut() {
    std::vector<bool> c(n);
    for (int i = 0; i < n; i++) {
      c[i] = (h[i] != -1);
    }
    return c;
  }
  struct Edge {
    int from;
    int to;
    T cap;
    T flow;
  };
  std::vector<Edge> edges() {
    std::vector<Edge> a;
    for (int i = 0; i < e.size(); i += 2) {
      Edge x;
      x.from = e[i + 1].to;
      x.to = e[i].to;
      x.cap = e[i].cap + e[i + 1].cap;
      x.flow = e[i + 1].cap;
      a.push_back(x);
    }
    return a;
  }
};
```

4.10　最小费用最大流

用途：用势能优化的最短路增广，求最小费用最大流。

```
template <class T> struct MinCostFlow {
  struct _Edge {
    int to;
    T cap;
    T cost;
    _Edge(int to_, T cap_, T cost_) : to(to_), cap(cap_), cost(cost_) {}
```

```
  };
  int n;
  std::vector<_Edge> e;
  std::vector<std::vector<int>> g;
  std::vector<T> h, dis;
  std::vector<int> pre;
  bool dijkstra(int s, int t) {
    dis.assign(n, std::numeric_limits<T>::max());
    pre.assign(n, -1);
    std::priority_queue<std::pair<T, int>, std::vector<std::pair<T, int>>,
                        std::greater<std::pair<T, int>>>
        que;
    dis[s] = 0;
    que.emplace(0, s);
    while (!que.empty()) {
      T d = que.top().first;
      int u = que.top().second;
      que.pop();
      if (dis[u] != d) {
        continue;
      }
      for (int i : g[u]) {
        int v = e[i].to;
        T cap = e[i].cap;
        T cost = e[i].cost;
        if (cap > 0 && dis[v] > d + h[u] - h[v] + cost) {
          dis[v] = d + h[u] - h[v] + cost;
          pre[v] = i;
          que.emplace(dis[v], v);
        }
      }
    }
    return dis[t] != std::numeric_limits<T>::max();
  }
  MinCostFlow() {}
  MinCostFlow(int n_) { init(n_); }
  void init(int n_) {
    n = n_;
    e.clear();
    g.assign(n, {});
  }
  void addEdge(int u, int v, T cap, T cost) {
    g[u].push_back(e.size());
    e.emplace_back(v, cap, cost);
    g[v].push_back(e.size());
    e.emplace_back(u, 0, -cost);
  }
  std::pair<T, T> flow(int s, int t) {
    T flow = 0;
    T cost = 0;
    h.assign(n, 0);
    while (dijkstra(s, t)) {
      for (int i = 0; i < n; ++i) {
        h[i] += dis[i];
      }
      T aug = std::numeric_limits<int>::max();
      for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
        aug = std::min(aug, e[pre[i]].cap);
      }
      for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
        e[pre[i]].cap -= aug;
        e[pre[i] ^ 1].cap += aug;
      }
      flow += aug;
```

```
      cost += aug * h[t];
    }
    return std::make_pair(flow, cost);
  }
  struct Edge {
    int from;
    int to;
    T cap;
    T cost;
    T flow;
  };
  std::vector<Edge> edges() {
    std::vector<Edge> a;
    for (int i = 0; i < e.size(); i += 2) {
      Edge x;
      x.from = e[i + 1].to;
      x.to = e[i].to;
      x.cap = e[i].cap + e[i + 1].cap;
      x.cost = e[i].cost;
      x.flow = e[i + 1].cap;
      a.push_back(x);
    }
    return a;
  }
};
```

4.11　欧拉路径与回路

用途： 判断并构造图中的 Euler 路径或 Euler 回路。

```
// 欧拉路径

// 定义：从一个点出发，不重不漏的经过图中每一条边的一条路径（允许多次经过同一个点）。
// 判断方法

// 若为无向图，则需连通，且图中恰好存在两个点的度数是奇数，其他节点的度数为偶数，这两个度数为奇数的点就是起点与终点；或者所有点度数都是偶数。

// 若为有向图，则需连通，且图中恰好存在一个点入度比出度多一，一个点出度比入度多一，其他节点的出度等于入度；或者所有点入度等于出度。

// 欧拉回路

// 定义：起点和终点是一个点的欧拉路径。

// 若为无向图，则需连通，且所有点的度数都是偶数。

// 若为有向图，则需连通，且所有点的入度等于出度
vector<int> sta(n + 10);
// 如果想求字典序最小可以把边排序一下
auto dfs = [&](auto dfs, int u) -> void {
  for (; sta[u] < g[u].size();) {
    sta[u]++;
    // 这里是有向图，如果为无向图，当我们经过一条边之后，他的反向边我们也得标记不能走
    dfs(dfs, g[u][sta[u] - 1]);
  }
  ans.push_back(u);
};
dfs(dfs, rt);
reverse(ans.begin(), ans.end());
```

4.12 最大团

用途：在无向图中寻找顶点数最多的团，即集合内任意两点都有边。模板用分支限界搜索：`vis[0..num-1]` 是当前团，只有与其中每个点相邻的候选点才能继续加入；`cnt[i]` 给出从编号 `i` 往后的候选上界，若 `num + cnt[i] <= res` 就可以剪枝。`res` 是最大团大小，`group[1..res]` 保存一组最优顶点。输入是 n 及 n\times n 邻接矩阵（代码以 `n=0` 结束），调用 `maxClique()` 后读取 `res/group`。**复杂度：**邻接矩阵占 $O(n^2)$ 空间；搜索没有多项式保证，最坏时间可达 $O(n^2 2^n)$（候选枚举和与当前团逐点判边均计入），实际速度依赖分支限界剪枝。它适合点数较小的精确最大团问题，不能因为数组上限写成 1000 就认为能处理 1000 个点。

例：若边为 `1-2,1-3,2-3,3-4`，搜索会先得到 `{1,2,3}`，大小为 3；点 4 不能和 1、2 同时相邻，因此不会扩展成更大的团。对应的一个输入片段是：

```
4
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0
0
```

输出的最大团大小为 3。

```
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <ctime>
#include <iostream>
#include <map>
#include <queue>
#include <set>
#include <stack>
#include <string>
#include <utility>
#include <vector>
#define PI acos(-1.0)
#define E 1e-6
#define MOD 1000000007
#define INF 0x3f3f3f3f
#define N 1001
#define LL long long
using namespace std;
int n;
int G[N][N];
int cnt[N];                 // cnt[i] 为 >=i 的最大团点数
int group[N];               // 最大团的点
int vis[N];                 // 记录点的位置
int res;                    // 最大团的数目
bool dfs(int pos, int num) { // num 为已取的点数
  for (int i = pos + 1; i <= n; i++) {
    if (cnt[i] + num <= res) // 剪枝，若取 i 但 cnt[i]+ 已经取了的点数仍 <ans
      return false;

    if (G[pos][i]) { // 与当前团中元素比较，取 Non-N(i)
      int j;
      for (j = 0; j < num; j++)
        if (!G[i][vis[j]])
          break;
      if (j == num) { // 若为空，则皆与 i 相邻，则此时将 i 加入到最大团中
```

```
        vis[num] = i;
        if (dfs(i, num + 1))
          return true;
      }
    }
  }

  if (num > res) { // 每添加一个点最多使最大团数 +1, 后面的搜索就没有意义了
    res = num;                      // 最大团中点的数目
    for (int i = 1; i <= num; i++) // 最大团的元素
      group[i] = vis[i - 1];
    return true;
  }
  return false;
}
void maxClique() {
  res = -1;
  for (int i = n; i >= 1; i--) { // 枚举所有点
    vis[0] = i;
    dfs(i, 1);
    cnt[i] = res;
  }
}
int main() {
  while (scanf("%d", &n) != EOF && (n)) {
    for (int i = 1; i <= n; i++)
      for (int j = 1; j <= n; j++)
        scanf("%d", &G[i][j]);
    maxClique();

    printf("%d\n", res); // 最大团的个数
  }
  return 0;
}
```

第 5 章

字符串

> 字符串匹配、哈希、后缀结构和自动机。

5.1 Manacher

用途：在线性时间内求所有奇偶回文串半径。

```
vector<int> mach(string s) {
  string t = "#";
  for (int i = 1; i < s.size(); i++) {
    t += s[i];
    t += '#';
  }
  int n = t.size();
  vector<int> r(n + 1);
  for (int i = 0, j = 0; i < n; i++) {
    if (2 * j - i >= 0 && j + r[j] > i) {
      r[i] = min(r[2 * j - i], j + r[j] - i);
    }
    while (i - r[i] >= 0 && i + r[i] < n && t[i - r[i]] == t[i + r[i]]) {
      r[i] += 1;
    }
    if (i + r[i] > j + r[j]) {
      j = i;
    }
  }
  for (int i = n; i >= 1; i--) {
    r[i] = r[i - 1] - 1;
  }
  return r;
}
```

5.2 字符串哈希

用途：用随机或多项式哈希把字符串子串比较转成整数比较。

```
struct custom_hash {
  static uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
  }
  size_t operator()(uint64_t x) const {
    static const uint64_t FIXED_RANDOM =
        chrono::steady_clock::now().time_since_epoch().count();
    return splitmix64(x + FIXED_RANDOM);
  }
};
```

```
struct hash2 {
  using ull = unsigned long long;
  vector<ull> row, col;
  vector<vector<ull>> a;
  int n = 0, m = 0;
  ull rb = 131;
  ull cb = 1313;
  hash2(vector<vector<int>> tmp, int n, int m) {
    a.resize(n + 3, vector<ull>(m + 3));
    row.resize(max(n, m) + 3);
    col.resize(max(n, m) + 3);
    row[0] = 1;
    col[0] = 1;
    for (int i = 1; i <= max(n, m); i++) {
      row[i] = row[i - 1] * rb;
      col[i] = col[i - 1] * cb;
    }
    for (int i = 1; i <= n; i++) {
      for (int j = 1; j <= m; j++) {
        a[i][j] = tmp[i][j];
      }
    }
    for (int i = 1; i <= n; i++) {
      for (int j = 1; j <= m; j++) {
        a[i][j] += a[i][j - 1] * rb;
      }
    }
    for (int i = 1; i <= n; i++) {
      for (int j = 1; j <= m; j++) {
        a[i][j] += a[i - 1][j] * cb;
      }
    }
    this->n = n;
    this->m = m;
  }
  ull get(int x1, int y1, int x2, int y2) {
    ull res = a[x2][y2];
    res -= a[x1 - 1][y2] * col[x2 - x1 + 1];
    res -= a[x2][y1 - 1] * row[y2 - y1 + 1];
    res += a[x1 - 1][y1 - 1] * col[x2 - x1 + 1] * row[y2 - y1 + 1];
    return res;
  }
};
```

5.3　单哈希（自然溢出）

用途：利用无符号整数自然溢出实现低常数的单哈希。

```
struct Hash {
  using ull = unsigned long long;
  vector<ull> base;
  vector<ull> h;
  ull p = 131;
  Hash(string s) {
    int n = s.size() - 1;
    base.resize(n + 10);
    h.resize(n + 10);
    base[0] = 1;
    for (int i = 1; i <= n; i++) {
      base[i] = base[i - 1] * p;
      h[i] = h[i - 1] * p + s[i];
```

```
    }
  }
  ull get(int l, int r) {
    assert(l <= r);
    return h[r] - h[l - 1] * base[r - l + 1];
  }
};
```

5.4 单哈希（取模）

用途：在指定模数下维护前缀哈希和子串哈希。

```
struct Hashmod {
  using ll = long long;
  vector<ll> base;
  vector<ll> h;
  ll p = 131;
  const ll mod = 1e9 + 7;
  Hashmod(string s) {
    int n = s.size() - 1;
    base.resize(n + 10);
    h.resize(n + 10);
    base[0] = 1;
    for (int i = 1; i <= n; i++) {
      base[i] = base[i - 1] * p % mod;
      h[i] = (h[i - 1] * p + s[i]) % mod;
    }
  }
  ll get(int l, int r) {
    assert(l <= r);
    return ((h[r] - h[l - 1] * base[r - l + 1]) % mod + mod) % mod;
  }
};
```

5.5 字典序最大后缀（双指针）

用途：在线性时间内只求一个字符串的字典序最大后缀。若还需要所有后缀的排名、相邻后缀 LCP 或多次子串比较，应使用后面的后缀数组。

维护当前最优起点 `i` 和待比较起点 `j`。若两个后缀的最长公共前缀长度为 `k`，第一次不同的位置就能一次淘汰一整段不可能成为答案的起点，因此总比较次数为 $O(n)$。

```
string lastSubstring(const string &s) {
  int i = 0, j = 1, n = (int)s.size();
  while (j < n) {
    int k = 0;
    while (j + k < n && s[i + k] == s[j + k]) {
      ++k;
    }
    if (j + k < n && s[i + k] < s[j + k]) {
      int old_i = i;
      i = j;
      j = max(j + 1, old_i + k + 1);
    } else {
      j += k + 1;
    }
  }
  return s.substr(i);
}
```

时间复杂度为 $O(n)$，除返回的子串外只使用 $O(1)$ 额外空间。空串和单字符都可直接处理。代码按 `std::string` 的字节比较工作，适合 ASCII 或通常的竞赛字符集；UTF-8 多字节文本若要按 Unicode 码点排序，需要先解码。

5.6 后缀数组

用途：按字典序排列所有后缀，支持 LCP、子串排名和二分查找某个模式串。

构造函数要求字符串从下标 1 开始: 原串 `s` 应传成 `" " + s`, `s[0]` 只是哨兵且应与正文字符不同。构造过程先按首字符排序, 再用倍增法按长度 1, 2, 4, ... 排序; 随后 Kasai 算法求出 `lcp[k]=LCP(sa[k-1],sa[k])`。

成员和函数的含义:

- `sa[k]`：字典序第 `k` 个后缀在原串中的起点；`rank[i]`：原串位置 `i` 的后缀排名；
- `lcp[k]`：相邻两个 SA 后缀的最长公共前缀长度；
- `work()`：对 `lcp` 建 RMQ 稀疏表，必须在查询前调用；
- `getlcp(l,r)`：参数是两个 **SA 排名**，不是原串位置，返回对应后缀的 LCP。若要比较原串位置 `x,y`，应调用 `getlcp(rank[x], rank[y])`；
- `f` 和 `len` 是 `work()` 内部使用的稀疏表及对数表。

`getlcp` 的原理是：排名为 `l<r` 的两个后缀，其 LCP 等于 `min(lcp[l+1..r])`，所以 `work()` 后可以用 RMQ 在 $O(1)$ 取出这个最小值。

例如原串 banana（传入 `" banana"`）的后缀顺序为 `[6:"a", 4:"ana", 2:"anana", 1:"banana", 5:"na", 3:"nana"]`，所以 `rank[2]=3`、`rank[4]=2`。调用 `sa.work()` 后，`sa.getlcp(sa.rank[2], sa.rank[4])` 返回 3，对应公共前缀 `"ana"`。构造和 `work()` 的复杂度都是 $O(n \log n)$，查询是 $O(1)$，空间为 $O(n \log n)$。代码把 RMQ 层数固定为 21，若 `n` 更大需要把 `array<int,21>` 的层数一并扩大。

```
string raw = "banana";
sufarr sa(" " + raw);
sa.work();
int common = sa.getlcp(sa.rank[2], sa.rank[4]);
```

```
struct sufarr {
  vector<int> rank, sa, tp, cnt, lcp, len;
  vector<array<int, 21>> f;
  int n;
  sufarr(string s) {
    // 下标从 1 开始
    this->n = s.size() - 1;
    rank.resize(n + 2);
    sa.resize(n + 2);
    tp.resize(n + 2);
    cnt.resize(n + 2);
    lcp.resize(n + 2);
    for (int i = 1; i <= n; i++)
      sa[i] = i;
    sort(sa.begin() + 1, sa.begin() + 1 + n,
         [&](int x, int y) { return s[x] < s[y]; });
    for (int i = 1; i <= n; i++) {
      rank[sa[i]] = rank[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
    }
    auto sort = [&]() {
      for (int i = 1; i <= n; i++)
        cnt[i] = 0;
      for (int i = 1; i <= n; i++)
        cnt[rank[i]]++;
      for (int i = 1; i <= n; i++)
```

```
      cnt[i] += cnt[i - 1];
    for (int i = n; i >= 1; i--)
      sa[cnt[rank[tp[i]]]--] = tp[i];
  };
  for (int w = 1; w <= n; w *= 2) {
    int p = 0;
    for (int i = 1; i <= w; i++)
      tp[++p] = n - i + 1;
    for (int i = 1; i <= n; i++)
      if (sa[i] > w)
        tp[++p] = sa[i] - w;
    sort();
    swap(rank, tp);
    rank[sa[1]] = 1;
    for (int i = 2; i <= n; i++) {
      int x = sa[i - 1] + w;
      if (x > n)
        x = -1;
      else
        x = tp[sa[i - 1] + w];
      int y = sa[i] + w;
      if (y > n)
        y = -1;
      else
        y = tp[sa[i] + w];
      rank[sa[i]] = rank[sa[i - 1]] + !(tp[sa[i - 1]] == tp[sa[i]] && x == y);
    }
  }
  int k = 0;
  for (int i = 1; i <= n; i++) {
    if (k >= 1)
      k--;
    int j = sa[rank[i] - 1];
    while (i + k <= n && j + k <= n && s[i + k] == s[j + k])
      k++;
    lcp[rank[i]] = k;
  }
}
void work() {
  f.assign(n + 1, {});
  len.resize(n + 1);
  len[1] = 0;
  for (int i = 2; i <= n; i++)
    len[i] = len[i / 2] + 1;
  for (int i = 1; i <= n; i++)
    f[i][0] = lcp[i];
  const int lg = log2(n);
  for (int j = 1; j <= lg; j++) {
    for (int i = 1; i + (1 << (j)) - 1 <= n; i++) {
      f[i][j] = min(f[i][j - 1], f[i + (1 << (j - 1))][j - 1]);
    }
  }
}
int getlcp(int l, int r) {
  if (l > r)
    swap(l, r);
  if (l == r)
    return n - sa[l] + 1;
  l++;
  return min(f[l][len[r - l + 1]],
             f[r - (1 << len[r - l + 1]) + 1][len[r - l + 1]]);
}
};
```

5.7　Z 函数

用途：计算每个后缀与整个字符串的最长公共前缀。

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5 + 10;
int vis[300];
#define int long long
vector<int> ex(string s) {
  int n = s.size();
  s = " " + s;
  vector<int> z(n + 10);
  int l = 1;
  int r = 0;
  for (int i = 2; i <= n; i++) {
    if (i <= r)
      z[i] = min(z[i - l + 1], r - i + 1);
    while (i + z[i] <= n && s[i + z[i]] == s[1 + z[i]])
      z[i]++;
    if (i + z[i] - 1 > r)
      r = i + z[i] - 1, l = i;
  }
  z[1] = n;
  // z[i] 表示从 s 串 i 下标开始的后缀和 s 串能匹配的最长的前缀长度
  return z;
}
void slove() {
  string s, t;
  cin >> t >> s;
  int n = s.size();
  vector<int> tz = ex(s);
  int m = t.size();
  s = " " + s;
  t = " " + t;
  vector<int> z(m + 10);
  int l = 1;
  int r = 0;
  for (int i = 1; i <= m; i++) {
    if (i <= r)
      z[i] = min({tz[i - l + 1], r - i + 1});
    while (i + z[i] <= m && 1 + z[i] <= n && t[i + z[i]] == s[1 + z[i]])
      z[i]++;
    if (i + z[i] - 1 > r)
      l = i, r = i + z[i] - 1;
  }
  // z[i] 表示从 t 串 i 下标开始的后缀和 s 串能匹配的最长的前缀长度
}
```

5.8　KMP

用途：在线性时间内完成模式匹配，并维护前缀函数。

```
using i64 = long long;
mt19937_64 rnd(time(0));
auto kmp = [&](string s) {
  int n = s.size();
  s = " " + s;
  vector<int> nex(n + 10);
  int i = 1;
```

```
  int j = 0;
  while (i <= n) {
    while (j && s[i] != s[j])
      j = nex[j];
    nex[++i] = ++j;
  }
  return nex;
};
// 示例: s = ababac, nex = 0 1 1 2 3 4。
int p = k + 1;
// t 是待匹配串，charnow 是当前枚举到的字符。
while (p != 0 && t[p] != charnow)
  p = nex[p];
// 匹配成功时，p 等于 t 的长度。
void getborder(string a) {
  auto kmp = [&](string s) {
    int n = s.size() - 1;
    vector<int> nex(n + 10);
    int i = 1;
    int j = 0;
    while (i <= n) {
      while (j && s[i] != s[j])
        j = nex[j];
      nex[++i] = ++j;
    }
    return nex;
  };
  // 所有的 border 可以组成 Logn 个等差序列
  //  pos 为每个等差序列的右端点，num 为这个等差序列的数量,d 为差值
  vector<int> nex = kmp(a);
  int n = a.size() - 1;
  vector<int> d(n + 10), pos(n + 10);
  vector<int> num(n + 10);
  int p = n + 1;
  int cnt = 0;
  while (1) {
    if (nex[p] >= 2) {
      ++cnt;
      if (2 * (nex[p] - 1) >= p - 1) {
        d[cnt] = p - 1 - (nex[p] - 1);
        pos[cnt] = (nex[p] - 1);
        int f = (p - 1) % d[cnt] + d[cnt];
        num[cnt] = (nex[p] - 1 - f) / d[cnt] + 1;
        assert((nex[p] - 1 - f) % d[cnt] == 0);
        p = f + 1;
      } else {
        d[cnt] = (nex[p] - 1);
        pos[cnt] = (nex[p] - 1);
        num[cnt] = 1;
        p = nex[p];
      }
    } else
      break;
  }
}
```

5.9 AC 自动机

用途：把多个模式串组织成 Trie 和失配边，在一个文本中同时匹配它们。

状态 1 是 Trie 根，状态 0 是虚拟根；`Node::len` 是 Trie 深度，`link` 是最长真后缀对应的状态，`next` 在 `work()` 后会被补成完整 DFA 转移。各函数的用法是：

- `init()`：清空并建立虚拟根和根；构造函数会自动调用它，重新使用对象时手动调用；
- `newNode()`：新建一个 Trie 节点并返回编号，通常不需要在外部直接调用；
- `add(pattern)`：插入一个小写模式串，返回它的终点状态，应保存这个编号来标记模式；
- `work()`：所有模式串插入完后调用一次，用 BFS 求失配边并补全转移；之后才能可靠使用 `next/link`；
- `next(p,c)`：从状态 `p` 读入 0-based 字符 `c` 后到达的状态；`link(p)`、`len(p)`、`size()` 分别查询失配边、深度和状态总数（包含 0、1）。

例：插入 `he`、`she`、`hers`，保存三个 `add` 返回值并调用 `work()`。扫描文本 `ushers` 时令 `p=1`，逐字符执行 `p=ac.next(p,c-'a')`：读到 `e` 时状态对应 `she`，沿失配链还能到达 `he`，因此同一位置同时匹配两个模式；末尾状态对应 `hers`（其失配链还会落到模式 `s` 的节点，如果插入了该模式）。失配边的作用是把当前最长后缀接到已有 Trie 状态，不需要回退文本指针。当前模板没有 `term`/输出链字段，只能得到状态转移；若要统计模式出现次数，需自行在终点记录编号，再沿 `link` 链检查，或建立失配树做一次 DFS 汇总。

最小使用代码如下（`end[i]` 是第 `i` 个模式串的终点）：

```
AhoCorasick ac;
vector<int> end;
for (string p : {"he", "she", "hers"})
  end.push_back(ac.add(p));
ac.work();
int state = 1;
for (char c : string("ushers"))
  state = ac.next(state, c - 'a');
```

插入总长度为 L、字母表大小为 26 时，`add`、`work` 和扫描文本的复杂度分别为 $O(L)$、$O(26L) = O(L)$ 和 $O(|text|)$，空间为 $O(26L)$。必须在所有 `add` 完成后再 `work`；`work` 后继续 `add` 会破坏已经补全的转移。

```
// 记得先 work 才能使用 link
struct AhoCorasick {
  static constexpr int ALPHABET = 26;
  struct Node {
    int len;
    int link;
    std::array<int, ALPHABET> next;
    Node() : len{0}, link{0}, next{} {}
  };
  std::vector<Node> t;
  AhoCorasick() { init(); }
  void init() {
    t.assign(2, Node());
    t[0].next.fill(1);
    t[0].len = -1;
  }
  int newNode() {
    t.emplace_back();
    return t.size() - 1;
  }
  int add(const std::string &a) {
    int p = 1;
    for (auto c : a) {
      int x = c - 'a';
      if (t[p].next[x] == 0) {
        t[p].next[x] = newNode();
        t[t[p].next[x]].len = t[p].len + 1;
      }
      p = t[p].next[x];
    }
    return p;
  }
  void work() {
```

```
    queue<int> q;
    q.push(1);
    while (!q.empty()) {
      int x = q.front();
      q.pop();
      for (int i = 0; i < ALPHABET; i++) {
        if (t[x].next[i] == 0) {
          t[x].next[i] = t[t[x].link].next[i];
        } else {
          t[t[x].next[i]].link = t[t[x].link].next[i];
          q.push(t[x].next[i]);
        }
      }
    }
  }
  int next(int p, int x) { return t[p].next[x]; }
  int link(int p) { return t[p].link; }
  int len(int p) { return t[p].len; }
  int size() { return t.size(); }
};
```

5.10 后缀自动机

用途：在线性时间构造一个表示所有子串的有向自动机，支持子串存在性、不同子串个数和出现次数统计。下面给出了数组转移版和 map 转移版，二者功能相同，实际代码中只能二选一，不能同时定义两个同名 SAM。

状态 1 是真正的根，状态 0 是构造用的虚拟状态。对每个状态 v：

- `len(v)` 是该状态所代表子串的最大长度；
- `link(v)` 指向最长的真后缀状态；该状态代表的长度区间是 `(len(link(v)), len(v)]`；
- `next(v,c)` 是在末尾追加字符 c 的转移。

函数用法：`init()` 清空并建立根；`newNode()` 新建状态；`extend(p,c)` 表示从当前前缀状态 p 追加 0-based 字符 c，返回新前缀对应的状态，建串时必须更新 p：

```
SAM sam;
int p = 1;
for (char c : string("abab"))
  p = sam.extend(p, c - 'a');
int v = 1;
for (char c : string("bab"))
  v = v ? sam.next(v, c - 'a') : 0;
bool exists = (v != 0);
```

`next/link/len/size` 分别读取转移、后缀链接、最大长度和状态总数。查询一个子串时从状态 1 逐字符走 `next`，中途没有边即不存在。数组版固定 26 个字符，单次转移 $O(1)$；map 版支持任意整数字符，但查边通常为 $O(\log \sigma)$。

`extend` 的三个分支正对应构造细节：若 p 没有 c 边，就新建 cur，沿后缀链接补上所有缺失的 c 边，再递归得到 `cur.link`；若已有边 `p-c->q` 且 `len[q]=len[p]+1`，直接复用 q；否则复制 q 得到 clone，复制其转移和后缀链接，并把所有指向 q 的相关边改指向 clone，这样才能保持后缀链接树的长度约束。

以 `"abab"` 为例：初始只有根；加入 a 建状态 2（`len=1,link=1`），加入 b 建状态 3（`len=2,link=1`），加入第二个 a 建状态 4（`len=3,link=2`），加入第二个 b 建状态 5（`len=4,link=3`）。状态 3 代表长度区间 1..2 的 b、ab，状态 4 代表 ba、aba，状态 5 代表 bab、abab。不同子串总数可用 `\sum_{v>1}(len[v]-len[link[v]])` 得到 7；若要统计出现次数，先对每个前缀终点 p 加一，再按 `len` 从大到小把计数累加到 `link[p]`。

实际状态（根及克隆）至多 2n-1，连同虚拟状态 0 至多 2n；数组版构造时间和空间分别为 $O(n)$、$O(26n)$；map 版时间约为 $O(n \log \sigma)$。克隆状态是内部等价类拆分，不应当成一个新的输入字符前缀单独计数。

```cpp
struct SAM {
  static constexpr int ALPHABET_SIZE = 26;
  struct Node {
    int len;
    int link;
    std::array<int, ALPHABET_SIZE> next;
    Node() : len{}, link{}, next{} {}
  };
  std::vector<Node> t;
  SAM() { init(); }
  void init() {
    t.assign(2, Node());
    t[0].next.fill(1);
    t[0].len = -1;
  }
  int newNode() {
    t.emplace_back();
    return t.size() - 1;
  }
  int extend(int p, int c) {
    if (t[p].next[c]) {
      int q = t[p].next[c];
      if (t[q].len == t[p].len + 1) {
        return q;
      }
      int r = newNode();
      t[r].len = t[p].len + 1;
      t[r].link = t[q].link;
      t[r].next = t[q].next;
      t[q].link = r;
      while (t[p].next[c] == q) {
        t[p].next[c] = r;
        p = t[p].link;
      }
      return r;
    }
    int cur = newNode();
    t[cur].len = t[p].len + 1;
    while (!t[p].next[c]) {
      t[p].next[c] = cur;
      p = t[p].link;
    }
    t[cur].link = extend(p, c);
    return cur;
  }
  int next(int p, int x) { return t[p].next[x]; }
  int link(int p) { return t[p].link; }
  int len(int p) { return t[p].len; }
  int size() { return t.size(); }
};

// 下面是 map 转移版本；与上面的 struct SAM 二选一，不能同时编译。
struct SAM {
  struct Node {
    int len;
    int link;
    std::map<int, int> next;
    Node() : len{}, link{} {}
  };
  std::vector<Node> t;
  SAM() { init(); }
  void init() {
    t.assign(2, Node());
```

```
    t[0].next.clear();
    t[0].len = -1;
  }
  int newNode() {
    t.emplace_back();
    return t.size() - 1;
  }
  int extend(int p, int c) {
    if (t[p].next.count(c)) {
      int q = t[p].next[c];
      if (t[q].len == t[p].len + 1) {
        return q;
      }
      int r = newNode();
      t[r].len = t[p].len + 1;
      t[r].link = t[q].link;
      t[r].next = t[q].next;
      t[q].link = r;
      while (p != 0 && (t[p].next.count(c) && t[p].next[c] == q)) {
        t[p].next[c] = r;
        p = t[p].link;
      }
      return r;
    }
    int cur = newNode();
    t[cur].len = t[p].len + 1;
    while (p != 0 && !t[p].next.count(c)) {
      t[p].next[c] = cur;
      p = t[p].link;
    }
    if (p == 0) {
      t[cur].link = 1;
    } else {
      t[cur].link = extend(p, c);
    }
    return cur;
  }
  int next(int p, int x) {
    if (p == 0) {
      return 1;
    }
    return t[p].next.count(x) ? t[p].next[x] : 0;
  }
  int link(int p) { return t[p].link; }
  int len(int p) { return t[p].len; }
  int size() { return t.size(); }
};

int p = 1;
vector<int> f(5e6);
for (auto it : s) {
  p = sam.extend(p, it - 'a');
  f[p]++;
}
long long ans = 0;
vector<vector<int>> g(sam.size() + 2);
for (int i = 2; i < sam.size(); i++) {
  g[sam.link(i)].push_back(i);
}
```

5.11　广义后缀自动机求多个串不同字串模板

```
template<int S = 26> struct GSAM {
    struct Node {
        int len, link;
        array<int, S> go;
        Node(int l = 0, int f = -1) : len(l), link(f) { go.fill(-1); }
    };
    vector<Node> trie{Node{}}, sam; int last;
    void add(string s) {
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (trie[u].go[c] < 0) trie[u].go[c] = trie.size(), trie.emplace_back();
            u = trie[u].go[c];
        }
    }
    void extend(int c) {
        int cur = sam.size(), p = last;
        sam.emplace_back(sam[last].len + 1);
        while (p >= 0 && sam[p].go[c] < 0) sam[p].go[c] = cur, p = sam[p].link;
        if (p < 0) sam[cur].link = 0;
        else {
            int q = sam[p].go[c];
            if (sam[p].len + 1 == sam[q].len) sam[cur].link = q;
            else {
                int clone = sam.size(); sam.push_back(sam[q]);
                sam[clone].len = sam[p].len + 1;
                while (p >= 0 && sam[p].go[c] == q) sam[p].go[c] = clone, p = sam[p].link;
                sam[q].link = sam[cur].link = clone;
            }
        }
        last = cur;
    }
    void dfs(int u) {
        int save = last;
        for (int c = 0; c < S; ++c) if (trie[u].go[c] >= 0) {
            extend(c); dfs(trie[u].go[c]); last = save;
        }
    }
    int work() {
        sam.clear(); sam.emplace_back(); last = 0; dfs(0);
        int ans = 0;
        for (int i = 1; i < (int)sam.size(); ++i)
            ans += sam[i].len - sam[sam[i].link].len;
        return ans;
    }
};
```

第 6 章

离线算法

> 通过排序、分块或时间维度降低查询复杂度。

6.1 莫队算法

用途：通过重新排序区间端点，把可增删维护的区间查询降到近似 O((N+Q)√N)。

```
const int N = 1e6;
struct node {
  int l, r;
} arr[N];
int be[N];
bool cmp(struct node a, struct node b) {
  if (be[a.l] != be[b.l])
    return a.l < b.l;
  if (be[a.l] % 2 == 1)
    return a.r < b.r;
  return a.r > b.r;
  // 奇偶优化
}
signed main() {
  int n, m;
  cin >> n >> m;
  int bk = sqrt(n);
  for (int i = 1; i <= n; i++) {
    be[i] = (i - 1) / bk + 1;
  }
  sort(arr + 1, arr + 1 + m, cmp);
  auto del = [&](int x) {};
  auto add = [&](int x) {};
  int l = 1;
  int r = 0;
  // 注意先 add 再 del 先扩展，再收缩，防止 RE。
  for (int i = 1; i <= m; i++) {
    while (arr[i].r > r)
      add(++r);
    while (arr[i].l < l)
      add(--l);
    while (arr[i].r < r)
      del(r--);
    while (arr[i].l > l)
      del(l++);
  };
}
```

6.2　回滚莫队（只增）

用途：只允许右端点扩展、左端点回滚的莫队变体，适合不可删除的维护信息。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
const int N = 2e5;
int be[N];
struct node {
  int l, r, id;
  bool operator<(struct node a) {
    if (be[l] != be[a.l])
      return be[l] < be[a.l];
    return r < a.r;
  }
} arr[N];
void slove() {
  int n, q;
  cin >> n >> q;
  vector<int> a(n + 10);
  for (int i = 1; i <= n; i++) {
    cin >> a[i];
  }
  for (int i = 1; i <= q; i++) {
    cin >> arr[i].l >> arr[i].r;
    arr[i].id = i;
  }
  int bk = sqrt(n);
  for (int i = 1; i <= n; i++) {
    be[i] = (i + bk - 1) / bk;
  }
  sort(arr + 1, arr + 1 + q);
  array<int, 3> ans = {0, 0, (int)1e9};
  auto temp = ans;
  int l = bk + 1;
  int r = bk;
  vector<int> cnt(n + 10);
  auto add = [&](int i) {
    int x = a[i];
    ans[0] -= (cnt[x] * (cnt[x] - 1)) / 2 * x;
    ++cnt[x];
    ans[0] += (cnt[x] * (cnt[x] - 1)) / 2 * x;
    int now = (cnt[x] * (cnt[x] - 1)) / 2 * x;
    if (now > ans[1]) {
      ans[1] = now;
      ans[2] = x;
    } else if (now == ans[1]) {
      ans[2] = min(ans[2], x);
    }
  };
  auto del = [&](int i) { --cnt[a[i]]; };
  vector<array<int, 2>> sum(q + 10);
  for (int i = 1; i <= q; i++) {

    if (be[arr[i].l] >= be[l]) {
      ans = temp;
      for (int j = l; j <= r; j++)
        del(j);
      r = be[arr[i].l] * bk;
      l = r + 1;
    }
```

```
    if (be[arr[i].l] == be[arr[i].r]) {
      for (int j = arr[i].l; j <= arr[i].r; j++) {
        add(j);
      }
      sum[arr[i].id] = {ans[0], ans[2]};
      for (int j = arr[i].l; j <= arr[i].r; j++) {
        del(j);
      }
      ans = temp;
      continue;
    }
    while (r < arr[i].r) {
      add(++r);
    }
    auto t = ans;
    for (int j = arr[i].l; j < l; j++)
      add(j);
    sum[arr[i].id] = {ans[0], ans[2]};
    for (int j = arr[i].l; j < l; j++)
      del(j);
    ans = t;
  }
  for (int i = 1; i <= q; i++) {
    cout << sum[i][0] << " " << sum[i][1] << "\n";
  }
}
signed main() {
  std::ios::sync_with_stdio(0);
  std::cin.tie(0);
  mt19937_64 rng(time(0));
  std::cout.tie(0);
  slove();
  return 0;
}
```

6.3 带修莫队

用途：在莫队排序中加入时间维度，处理区间查询与单点修改混合的问题。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
long long asksize;
const int N = 1e6 + 10;
int pos[N];
int pre[N];
int a[N];
struct askuery {
  int id, t, l, r;
  bool operator<(const askuery &b) const {
    return (pos[l] < pos[b.l]) || (pos[l] == pos[b.l] && pos[r] < pos[b.r]) ||
           (pos[l] == pos[b.l] && pos[r] == pos[b.r] && t < b.t);
  }
} ask[N];
struct operation {
  int p, v, t;
} mo[N];
int vis[N];
signed main() {
```

```
  cin.tie(nullptr);
  ios::sync_with_stdio(false);
  int n, q;
  cin >> n >> q;
  int cnt1 = 0;
  int cnt2 = 0;
  asksize = pow(n, 0.67);
  for (int i = 1; i <= n; i++)
    cin >> a[i], pre[i] = a[i];
  for (int i = 1; i <= N; i++) {
    pos[i] = (i - 1) / asksize;
  }
  for (int i = 1; i <= q; i++) {
    char op;
    cin >> op;
    if (op == 'Q') {
      int l, r;
      cin >> l >> r;
      ++cnt1;
      ask[cnt1] = {cnt1, cnt2, l, r};
    } else {
      int x, v;
      cin >> x >> v;
      ++cnt2;
      mo[cnt2].p = x;
      mo[cnt2].v = pre[x];
      pre[x] = mo[cnt2].t = v;
    }
  }
  sort(ask + 1, ask + cnt1 + 1);
  int sum = 0;
  auto add = [&](int i) {
    if (++vis[a[i]] == 1)
      sum++;
  };
  auto del = [&](int i) {
    if (--vis[a[i]] == 0)
      sum--;
  };
  auto modify = [&](int i, int v, int l, int r) {
    if (i >= l && i <= r) {
      del(i);
      a[i] = v;
      add(i);
    } else
      a[i] = v;
  };
  vector<int> ans(q + 10);

  int l = 1;
  int r = 0;
  int last = 0;
  for (int i = 1; i <= cnt1; i++) {
    while (ask[i].r > r)
      add(++r);
    while (ask[i].l < l)
      add(--l);
    while (ask[i].r < r)
      del(r--);
    while (ask[i].l > l)
      del(l++);
    while (last + 1 <= ask[i].t) {
      last++;
      modify(mo[last].p, mo[last].t, ask[i].l, ask[i].r);
```

```
    }
    while (last > ask[i].t) {
      modify(mo[last].p, mo[last].v, ask[i].l, ask[i].r);
      last--;
    }
    ans[ask[i].id] = sum;
  }
  for (int i = 1; i <= cnt1; i++) {
    cout << ans[i] << "\n";
  }
}
```

6.4 树上莫队

用途：把树上路径转化为 Euler 序区间，离线处理路径统计；本模板还带有“时间”维度，可以和单点修改混合。

路径为什么能变成区间： DFS 时每个点入栈一次、回溯一次，记入栈位置为 `st[u]`、回溯位置为 `ed[u]`，并令 `now[k]` 为 Euler 序第 k 个点。因此每个顶点在 now 中出现两次。对一个区间 `[L,R]`，把其中每个点出现的次数对 2 取模，恰好出现奇数次的点就是要维护的集合。

设 `st[x] <= st[y]`、`z=lca(x,y)`：

- 若 `z==x`，使用区间 `[st[x], st[y]]`；
- 否则使用 `[ed[x], st[y]]`，这个区间会得到路径上除 z 外的点，再把 z 临时加入一次。

例如树边为 `1-2,1-3,3-4,3-5`，DFS 序为 `now=[1,2,2,3,4,4,5,5,3,1]`。查询路径 `2->4` 时，`z=1`，区间是 `[ed[2],st[4]]=[3,5]`，其中 2,3,4 各出现一次，再临时加入 1，得到路径 `{2,1,3,4}`；查询 `3->5` 时 `z=3`，区间 `[st[3],st[5]]=[4,7]` 中 4 出现两次抵消，剩下 `{3,5}`。

DFS 中 `f[u][j]` 是 u 的 2^j 级祖先，`de[u]` 是深度；`lca(x,y)` 先把较深节点提升到同一深度，再从大到小同时跳祖先。`st/ed/now` 负责 Euler 转化，`pos` 只用于把 Euler 位置分块并排序查询。

代码中的维护： `add(i)` 实际是“切换”节点 `i` 的选中状态，扩展和收缩区间都调用它，所以最后 `vis[i]==1` 的正是当前路径节点。`cnt[x]` 是当前路径上颜色（或编号）x 的出现次数，`ans` 按题目给定的 `w[cnt[x]]*val[x]` 更新。`mo` 保存每次修改的旧值和新值，`last` 表示当前处理到的修改时间；向前调用 `modify(p,new)`，向后则调用 `modify(p,old)` 回滚。修改不在当前路径时只改 `a[p]`，在路径中时还要先从计数中删除旧值、再加入新值。

这里 `ans` 维护的是每种颜色的前缀贡献 `(w[1]+...+w[cnt[x]])*val[x]`：某颜色第 1 个节点加入时加 `w[1]*val[x]`，第 2 个再加 `w[2]*val[x]`，删除时按相反顺序减回。`q[i].ad` 非零时，主循环先把 LCA 切换进来、记录答案，再切换一次将它移除，因此 LCA 不会污染下一条查询。

查询按 Euler 左端点块、右端点、时间排序，减少区间端点和时间的移动。代码中的 `qsize=2500` 是针对特定数据范围的经验值；常见理论取块长约 $n^{2/3}$。预处理 LCA 为 $O(n\log n)$，每次 `add/modify` 为 $O(1)$，三维莫队的典型总复杂度约为 $O((n+q)n^{2/3})$，空间为 $O(n\log n+q)$。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
long long qsize;
const int N = 2e5 + 10;
int pos[N];
struct query {
  int now, t, l, r;
  int ad = 0;
  bool operator<(const query &b) const {
    return (pos[l] < pos[b.l]) || (pos[l] == pos[b.l] && pos[r] < pos[b.r]) ||
           (pos[l] == pos[b.l] && pos[r] == pos[b.r] && t < b.t);
  }
```

```
} q[N];
struct operation {
  int p, v, t;
} mo[N];
int val[N];
int w[N];
int a[N];
int f[N][21];
int de[N];
int pre[N];
int now[N];
int st[N];
int ed[N];
int sum[N];
int cnt[N];
int vis[N];
signed main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  cout.tie(0);
  int n, m;
  cin >> n >> m;
  int qq;
  cin >> qq;
  int cnt1 = 0;
  int cnt2 = 0;
  qsize = 2500;
  for (int i = 1; i <= m; i++)
    cin >> val[i];
  for (int i = 1; i <= n; i++)
    cin >> w[i];
  vector<vector<int>> g(n + 1);
  for (int i = 1; i < n; i++) {
    int x, y;
    cin >> x >> y;
    g[x].push_back(y);
    g[y].push_back(x);
  }
  for (int i = 1; i <= n; i++)
    cin >> a[i], pre[i] = a[i];
  int idx = 0;
  auto dfs = [&](auto dfs, int u, int fa) -> void {
    f[u][0] = fa;
    for (int i = 1; i <= 20; i++)
      f[u][i] = f[f[u][i - 1]][i - 1];
    de[u] = de[fa] + 1;
    st[u] = ++idx;
    now[idx] = u;
    for (auto it : g[u]) {
      if (it == fa)
        continue;
      dfs(dfs, it, u);
    }
    ed[u] = ++idx;
    now[idx] = u;
  };
  dfs(dfs, 1, 0);
  for (int i = 1; i <= idx; i++) {
    pos[i] = (i - 1) / qsize;
  }
  auto lca = [&](int x, int y) {
    if (de[x] < de[y])
      swap(x, y);
    for (int i = 20; i >= 0; i--) {
```

```
      if (de[f[x][i]] >= de[y])
        x = f[x][i];
    }
    if (x == y)
      return x;
    for (int i = 20; i >= 0; i--) {
      if (f[x][i] != f[y][i]) {
        x = f[x][i];
        y = f[y][i];
      }
    }
    return f[x][0];
  };
  for (int i = 1; i <= qq; i++) {
    int op, x, y;
    cin >> op >> x >> y;
    if (op == 1) {
      ++cnt1;
      if (st[x] > st[y])
        swap(x, y);
      int lc = lca(x, y);
      if (lc == x)
        q[cnt1] = {cnt1, cnt2, st[x], st[y], 0};
      else
        q[cnt1] = {cnt1, cnt2, ed[x], st[y], lc};
    } else {
      ++cnt2;
      mo[cnt2].p = x;
      mo[cnt2].v = pre[x];
      pre[x] = mo[cnt2].t = y;
    }
  }
  sort(q + 1, q + cnt1 + 1);
  int ans = 0;
  auto add = [&](int i) {
    int x = a[i];
    if (vis[i] == 0) {
      ++cnt[x];
      ans += w[cnt[x]] * val[x];
    } else {
      ans -= w[cnt[x]] * val[x];
      --cnt[x];
    }
    vis[i] ^= 1;
  };
  auto modify = [&](int i, int v) {
    if (vis[i]) { // 如果节点 i 为 1 那么说明访问过 所以 他会增加一个 cnt 的值
      ans -= w[cnt[a[i]]] * val[a[i]];
      cnt[a[i]]--;
      a[i] = v;
      cnt[a[i]]++;
      ans += w[cnt[a[i]]] * val[a[i]];
    } else
      a[i] = v; // 否则我们不需要考虑节点 i 在不在区间内部
  };
  int l = 1, r = 0, last = 0;
  for (int i = 1; i <= cnt1; i++) {

    while (q[i].r > r)
      add(now[++r]);
    while (q[i].l < l)
      add(now[--l]);
    while (q[i].r < r)
      add(now[r--]);
```

```
    while (q[i].l > l)
      add(now[l++]);
    while (last + 1 <= q[i].t) {
      last++;
      modify(mo[last].p, mo[last].t);
    }
    while (last > q[i].t) {
      modify(mo[last].p, mo[last].v);
      last--;
    }
    int lc = q[i].ad;
    if (lc) {
      add(lc);
      sum[q[i].now] = ans;
      add(lc);
    } else
      sum[q[i].now] = ans;
  }
  for (int i = 1; i <= cnt1; i++) {
    cout << sum[i] << "\n";
  }
}
```

第 7 章

计算几何与扫描线

> 坐标变换、几何基础和扫描线类算法。

7.1 矩形并面积（整数网格）

用途： 用扫描线和离散化维护多个闭区间矩形的并面积。`add({x1,y1,x2,y2})` 将端点视为整数网格边界，内部自动转成半开区间；`work()` 返回并集面积。

```
struct UnionArea {
  vector<array<int, 4>> a;
  vector<int> y, c, s;

  void add(array<int, 4> q) {
    if (q[0] > q[2])
      swap(q[0], q[2]);
    if (q[1] > q[3])
      swap(q[1], q[3]);
    if (q[0] > q[2] || q[1] > q[3])
      return;

    q[2]++;
    q[3]++;
    a.push_back({q[0], q[1], q[3], 1});
    a.push_back({q[2], q[1], q[3], -1});
  }

  int work() {
    if (a.empty())
      return 0;

    y.clear();
    for (auto q : a)
      y.insert(y.end(), {q[1], q[2]});
    sort(y.begin(), y.end());
    y.erase(unique(y.begin(), y.end()), y.end());
    sort(a.begin(), a.end());

    c = s = vector<int>(4 * y.size());
    auto update = [&](auto &&update, int p, int L, int R, int l, int r,
                      int v) -> void {
      if (l <= L && R <= r)
        c[p] += v;
      else {
        int M = (L + R) / 2;
        if (l < M)
          update(update, p * 2, L, M, l, r, v);
        if (r > M)
          update(update, p * 2 + 1, M, R, l, r, v);
      }

      s[p] = c[p] ? y[R] - y[L]
```

```
                : (R - L < 2 ? 0 : s[p * 2] + s[p * 2 + 1]);
    };

    int ans = 0, last = a[0][0];
    for (auto q : a) {
      ans += (q[0] - last) * s[1];
      int l = lower_bound(y.begin(), y.end(), q[1]) - y.begin();
      int r = lower_bound(y.begin(), y.end(), q[2]) - y.begin();
      update(update, 1, 0, y.size() - 1, l, r, q[3]);
      last = q[0];
    }
    return ans;
  }
};
```

7.2　二维矩形加与矩形查询

用途：离线处理二维矩形加法和矩形求和；`work()` 支持加法、查询交错出现，`work2()` 适用于所有加法先出现、查询后出现的情况。

操作数组 `A` 的格式为 `{type,x1,y1,x2,y2,v}`：`type == 1` 表示给矩形加上 `v`，其他值表示查询矩形和；查询项的 `v` 会被忽略。

```
struct Rect {
  using A = array<int, 6>;
  struct U {
    int x, y, v, Y, p;
  };
  struct P {
    int x, y, s, id, Y;
  };
  struct T {
    vector<U> u;
    vector<P> q;
  };
  struct B {
    vector<int> t;
    B() {}
    B(int n) : t(n + 1) {}
    void add(int p, int v) {
      for (; p < (int)t.size(); p += p & -p)
        t[p] += v;
    }
    int sum(int p) {
      int r = 0;
      for (; p; p -= p & -p)
        r += t[p];
      return r;
    }
  } b[4], bp;

  vector<A> a;
  vector<T> t;
  vector<int> ys, ans;

  void add(A z) { a.push_back(z); }

  int prep() {
    t.assign(a.size(), {});
    ys.clear();
    int n = 0;
```

```
  for (int i = 0; i < (int)a.size(); ++i) {
    auto z = a[i];
    int x1 = z[1], y1 = z[2], x2 = z[3], y2 = z[4];
    if (x1 > x2)
      swap(x1, x2);
    if (y1 > y2)
      swap(y1, y2);
    if (z[0] == 1) {
      int v = z[5];
      if (x1 == x2 && y1 == y2)
        t[i].u = {{x1, y1, v, 0, 1}};
      else
        t[i].u = {{x1, y1, v, 0, 0},
                  {x1, y2 + 1, -v, 0, 0},
                  {x2 + 1, y1, -v, 0, 0},
                  {x2 + 1, y2 + 1, v, 0, 0}};
      for (auto e : t[i].u)
        ys.push_back(e.y);
    } else {
      auto f = [&](int x, int y, int s) {
        t[i].q.push_back({x, y, s, n, 0});
      };
      f(x2, y2, 1);
      f(x1 - 1, y2, -1);
      f(x2, y1 - 1, -1);
      f(x1 - 1, y1 - 1, 1);
      ++n;
    }
  }

  sort(ys.begin(), ys.end());
  ys.erase(unique(ys.begin(), ys.end()), ys.end());
  for (auto &z : t) {
    for (auto &e : z.u)
      e.Y = lower_bound(ys.begin(), ys.end(), e.y) - ys.begin() + 1;
    for (auto &e : z.q)
      e.Y = upper_bound(ys.begin(), ys.end(), e.y) - ys.begin();
  }
  for (auto &z : b)
    z = B(ys.size());
  bp = B(ys.size());
  ans.assign(n, 0);
  return n;
}

void put(U e, int sign = 1) {
  int v = e.v * sign;
  if (e.p) {
    bp.add(e.Y, v);
    return;
  }
  b[0].add(e.Y, v);
  b[1].add(e.Y, v * e.x);
  b[2].add(e.Y, v * e.y);
  b[3].add(e.Y, v * e.x * e.y);
}

void get(P e) {
  if (!e.Y)
    return;
  int X = e.x, Y = e.y;
  int v = bp.sum(e.Y) + (X + 1) * (Y + 1) * b[0].sum(e.Y) -
          (Y + 1) * b[1].sum(e.Y) - (X + 1) * b[2].sum(e.Y) +
          b[3].sum(e.Y);
```

```
    ans[e.id] += e.s * v;
  }

  vector<int> work() { // 操作交错
    prep();
    auto dc = [&](auto &&dc, int l, int r) -> void {
      if (l == r)
        return;
      int m = (l + r) >> 1;
      dc(dc, l, m);
      dc(dc, m + 1, r);
      vector<U> u;
      vector<P> q;
      for (int i = l; i <= m; ++i)
        u.insert(u.end(), t[i].u.begin(), t[i].u.end());
      for (int i = m + 1; i <= r; ++i)
        q.insert(q.end(), t[i].q.begin(), t[i].q.end());
      sort(u.begin(), u.end(), [](U a, U b) { return a.x < b.x; });
      sort(q.begin(), q.end(), [](P a, P b) { return a.x < b.x; });
      int j = 0;
      for (auto e : q) {
        while (j < (int)u.size() && u[j].x <= e.x)
          put(u[j++]);
        get(e);
      }
      for (int i = 0; i < j; ++i)
        put(u[i], -1);
    };
    if (!a.empty())
      dc(dc, 0, (int)a.size() - 1);
    return ans;
  }

  vector<int> work2() { // 所有加法在所有查询之前
    prep();
    vector<U> u;
    vector<P> q;
    for (auto &z : t) {
      u.insert(u.end(), z.u.begin(), z.u.end());
      q.insert(q.end(), z.q.begin(), z.q.end());
    }
    sort(u.begin(), u.end(), [](U a, U b) { return a.x < b.x; });
    sort(q.begin(), q.end(), [](P a, P b) { return a.x < b.x; });
    int j = 0;
    for (auto e : q) {
      while (j < (int)u.size() && u[j].x <= e.x)
        put(u[j++]);
      get(e);
    }
    return ans;
  }
};
```

7.3　计算几何基础

用途：二维点、向量、叉积、直线和几何关系的基础模板。

```
template <class T> struct Point {
  T x;
  T y;
  Point(const T &x_ = 0, const T &y_ = 0) : x(x_), y(y_) {}
```

```
  template <class U> operator Point<U>() { return Point<U>(U(x), U(y)); }
  Point &operator+=(const Point &p) & {
    x += p.x;
    y += p.y;
    return *this;
  }
  Point &operator-=(const Point &p) & {
    x -= p.x;
    y -= p.y;
    return *this;
  }
  Point &operator*=(const T &v) & {
    x *= v;
    y *= v;
    return *this;
  }
  Point &operator/=(const T &v) & {
    x /= v;
    y /= v;
    return *this;
  }
  Point operator-() const { return Point(-x, -y); }
  friend Point operator+(Point a, const Point &b) { return a += b; }
  friend Point operator-(Point a, const Point &b) { return a -= b; }
  friend Point operator*(Point a, const T &b) { return a *= b; }
  friend Point operator/(Point a, const T &b) { return a /= b; }
  friend Point operator*(const T &a, Point b) { return b *= a; }
  friend bool operator==(const Point &a, const Point &b) {
    return a.x == b.x && a.y == b.y;
  }
  friend std::istream &operator>>(std::istream &is, Point &p) {
    return is >> p.x >> p.y;
  }
  friend std::ostream &operator<<(std::ostream &os, const Point &p) {
    return os << "(" << p.x << ", " << p.y << ")";
  }
};
template <class T>
long double
utov(Point<T> u,
     Point<T> v) { // 顺时针 u -> v 转过的角度 如果 u 和 v 重合的话这里默认为 0
  long double pi = acosl(0) * 2;
  long double ans1 = atan2(u.y, u.x);
  if (ans1 < 0)
    ans1 += 2 * pi;
  long double ans2 = atan2(v.y, v.x);
  if (ans2 < 0)
    ans2 += 2 * pi;
  ans1 = ans1 - ans2;
  if (ans1 < 0)
    ans1 += 2 * pi;
  return ans1;
}
template <class T> struct Line {
  Point<T> a;
  Point<T> b;
  Line(const Point<T> &a_ = Point<T>(), const Point<T> &b_ = Point<T>())
      : a(a_), b(b_) {}
};
template <class T> T dot(const Point<T> &a, const Point<T> &b) {
  return a.x * b.x + a.y * b.y;
}
template <class T> T cross(const Point<T> &a, const Point<T> &b) {
```

```
  return a.x * b.y - a.y * b.x;
}
template <class T> T square(const Point<T> &p) { return dot(p, p); }

template <class T> double length(const Point<T> &p) {
  return std::sqrt(square(p));
}

template <class T> double length(const Line<T> &l) { return length(l.a - l.b); }

template <class T> Point<T> normalize(const Point<T> &p) {
  return p / length(p);
}

template <class T> bool parallel(const Line<T> &l1, const Line<T> &l2) {
  return cross(l1.b - l1.a, l2.b - l2.a) == 0;
}
template <class T> double distance(const Point<T> &a, const Point<T> &b) {
  return length(a - b);
}
template <class T> double distancePL(const Point<T> &p, const Line<T> &l) {
  return std::abs(cross(l.a - l.b, l.a - p)) / length(l);
}
template <class T> double distancePS(const Point<T> &p, const Line<T> &l) {
  if (dot(p - l.a, l.b - l.a) < 0) {
    return distance(p, l.a);
  }
  if (dot(p - l.b, l.a - l.b) < 0) {
    return distance(p, l.b);
  }
  return distancePL(p, l);
}
template <class T> double distanceSS(const Line<T> &l1, const Line<T> &l2) {
  if (std::get<0>(segmentIntersection(l1, l2)) != 0) {
    return 0.0;
  }
  return std::min({distancePS(l1.a, l2), distancePS(l1.b, l2),
                   distancePS(l2.a, l1), distancePS(l2.b, l1)});
}
template <class T> Point<T> rotate(const Point<T> &a) {
  return Point(-a.y, a.x);
}
template <class T> int sgn(const Point<T> &a) {
  return a.y > 0 || (a.y == 0 && a.x > 0) ? 1 : -1;
}
template <class T> bool pointOnLineLeft(const Point<T> &p, const Line<T> &l) {
  return cross(l.b - l.a, p - l.a) > 0;
}
template <class T>
Point<T> lineIntersection(const Line<T> &l1, const Line<T> &l2) {
  return l1.a + (l1.b - l1.a) * (cross(l2.b - l2.a, l1.a - l2.a) /
                                 cross(l2.b - l2.a, l1.a - l1.b));
}

template <class T> bool pointOnSegment(const Point<T> &p, const Line<T> &l) {
  return cross(p - l.a, l.b - l.a) == 0 && std::min(l.a.x, l.b.x) <= p.x &&
         p.x <= std::max(l.a.x, l.b.x) && std::min(l.a.y, l.b.y) <= p.y &&
         p.y <= std::max(l.a.y, l.b.y);
}

template <class T>
bool pointInPolygon(const Point<T> &a, const std::vector<Point<T>> &p) {
  int n = p.size();
  for (int i = 0; i < n; i++) {
```

```
    if (pointOnSegment(a, Line(p[i], p[(i + 1) % n]))) {
      return true;
    }
  }
  int t = 0;
  for (int i = 0; i < n; i++) {
    auto u = p[i];
    auto v = p[(i + 1) % n];
    if (u.x < a.x && v.x >= a.x && pointOnLineLeft(a, Line(v, u))) {
      t ^= 1;
    }
    if (u.x >= a.x && v.x < a.x && pointOnLineLeft(a, Line(u, v))) {
      t ^= 1;
    }
  }

  return t == 1;
}
// 0 : not intersect
// 1 : strictly intersect
// 2 : overlap
// 3 : intersect at endpoint
template <class T>
std::tuple<int, Point<T>, Point<T>> segmentIntersection(const Line<T> &l1,
                                                        const Line<T> &l2) {
  if (std::max(l1.a.x, l1.b.x) < std::min(l2.a.x, l2.b.x)) {
    return {0, Point<T>(), Point<T>()};
  }
  if (std::min(l1.a.x, l1.b.x) > std::max(l2.a.x, l2.b.x)) {
    return {0, Point<T>(), Point<T>()};
  }
  if (std::max(l1.a.y, l1.b.y) < std::min(l2.a.y, l2.b.y)) {
    return {0, Point<T>(), Point<T>()};
  }
  if (std::min(l1.a.y, l1.b.y) > std::max(l2.a.y, l2.b.y)) {
    return {0, Point<T>(), Point<T>()};
  }
  if (cross(l1.b - l1.a, l2.b - l2.a) == 0) {
    if (cross(l1.b - l1.a, l2.a - l1.a) != 0) {
      return {0, Point<T>(), Point<T>()};
    } else {
      auto maxx1 = std::max(l1.a.x, l1.b.x);
      auto minx1 = std::min(l1.a.x, l1.b.x);
      auto maxy1 = std::max(l1.a.y, l1.b.y);
      auto miny1 = std::min(l1.a.y, l1.b.y);
      auto maxx2 = std::max(l2.a.x, l2.b.x);
      auto minx2 = std::min(l2.a.x, l2.b.x);
      auto maxy2 = std::max(l2.a.y, l2.b.y);
      auto miny2 = std::min(l2.a.y, l2.b.y);
      Point<T> p1(std::max(minx1, minx2), std::max(miny1, miny2));
      Point<T> p2(std::min(maxx1, maxx2), std::min(maxy1, maxy2));
      if (!pointOnSegment(p1, l1)) {
        std::swap(p1.y, p2.y);
      }
      if (p1 == p2) {
        return {3, p1, p2};
      } else {
        return {2, p1, p2};
      }
    }
  }
  auto cp1 = cross(l2.a - l1.a, l2.b - l1.a);
  auto cp2 = cross(l2.a - l1.b, l2.b - l1.b);
  auto cp3 = cross(l1.a - l2.a, l1.b - l2.a);
```

```
  auto cp4 = cross(l1.a - l2.b, l1.b - l2.b);
  if ((cp1 > 0 && cp2 > 0) || (cp1 < 0 && cp2 < 0) || (cp3 > 0 && cp4 > 0) ||
      (cp3 < 0 && cp4 < 0)) {
    return {0, Point<T>(), Point<T>()};
  }
  Point p = lineIntersection(l1, l2);
  if (cp1 != 0 && cp2 != 0 && cp3 != 0 && cp4 != 0) {
    return {1, p, p};
  } else {
    return {3, p, p};
  }
}
template <class T> std::vector<Point<T>> getHull(std::vector<Point<T>> p) {
  std::vector<Point<T>> h, l;
  std::sort(p.begin(), p.end(), [&](auto a, auto b) {
    if (a.x != b.x) {
      return a.x < b.x;
    } else {
      return a.y < b.y;
    }
  });
  p.erase(std::unique(p.begin(), p.end()), p.end());
  if (p.size() <= 1) {
    return p;
  }
  for (auto a : p) {
    while (h.size() > 1 && cross(a - h.back(), a - h[h.size() - 2]) <= 0) {
      h.pop_back();
    }
    while (l.size() > 1 && cross(a - l.back(), a - l[l.size() - 2]) >= 0) {
      l.pop_back();
    }
    l.push_back(a);
    h.push_back(a);
  }
  l.pop_back();
  std::reverse(h.begin(), h.end());
  h.pop_back();
  l.insert(l.end(), h.begin(), h.end());
  return l;
}
template <class T> double hullDiameter(vector<Point<T>> b) {
  std::vector<Point<T>> d;
  int m = b.size();
  d.reserve(2 * m);
  for (int i = 0; i < m; i++) {
    int j = i == m - 1 ? 0 : i + 1;
    d.push_back(b[j] - b[i]);
    d.push_back(b[i] - b[j]);
  }
  // 闵可斯夫基求凸包直径
  sort(d.begin(), d.end(), [&](const auto &a, const auto &b) {
    if (sgn(a) != sgn(b)) {
      return sgn(a) == 1;
    }
    return cross(a, b) > 0;
  });
  b.resize(2 * m);
  Point<T> u{};
  for (int i = 0; i < 2 * m; i++) {
    b[i] = u;
    u += d[i];
  }
  double ans = 0;
```

```
  for (int i = 0; i < m; i++) {
    ans = max(ans, distance(b[i], b[i + m]));
  }
  return ans / 2;
}
template <class T> double getAreaPolygon(vector<Point<T>> b) {
  int n = b.size();
  double ans = 0;
  for (int i = 0; i < n; i++) {
    int j = (i + 1) % n;
    ans += cross(b[i], b[j]);
  }
  ans /= 2;
  return ans;
}
template <class T>
std::vector<Point<T>>
hp(std::vector<Line<T>> lines) { // 左侧是逆时针 lines 要保证是逆时针加的
  // 半平面交
  std::sort(lines.begin(), lines.end(), [&](auto l1, auto l2) {
    auto d1 = l1.b - l1.a;
    auto d2 = l2.b - l2.a;
    if (sgn(d1) != sgn(d2)) {
      return sgn(d1) == 1;
    }
    return cross(d1, d2) > 0;
  });
  std::deque<Line<T>> ls;
  std::deque<Point<T>> ps;
  for (auto l : lines) {
    if (ls.empty()) {
      ls.push_back(l);
      continue;
    }
    while (!ps.empty() && !pointOnLineLeft(ps.back(), l)) {
      ps.pop_back();
      ls.pop_back();
    }
    while (!ps.empty() && !pointOnLineLeft(ps[0], l)) {
      ps.pop_front();
      ls.pop_front();
    }
    if (cross(l.b - l.a, ls.back().b - ls.back().a) == 0) {
      if (dot(l.b - l.a, ls.back().b - ls.back().a) > 0) {

        if (!pointOnLineLeft(ls.back().a, l)) {
          assert(ls.size() == 1);
          ls[0] = l;
        }
        continue;
      }
      return {};
    }
    ps.push_back(lineIntersection(ls.back(), l));
    ls.push_back(l);
  }

  while (!ps.empty() && !pointOnLineLeft(ps.back(), ls[0])) {
    ps.pop_back();
    ls.pop_back();
  }
  if (ls.size() <= 2) {
    return {};
  }
```

```
  ps.push_back(lineIntersection(ls[0], ls.back()));
  return std::vector<Point<T>>(ps.begin(), ps.end());
}
template <class T>
int pointInPolygon1(const Point<T> &a, const std::vector<Point<T>> &p) {
  int n = p.size();
  int l = 0;
  int r = n - 1;
  while (l < r) {
    int mid = (l + r) / 2;
    int c1 = cross(p[mid] - p[0], a - p[0]);
    int c2 = cross(p[(mid + 1) % n] - p[0], a - p[0]);
    int c3 = cross(p[(mid + 1) % n] - p[mid], a - p[mid]);
    if (c1 >= 0 && c2 <= 0 && c3 >= 0) {
      if (!c3 || (mid == 1 && !c1) || (mid == n - 2 && !c2))
        return 0;
      return 1;
    }
    if (c1 >= 0)
      l = mid + 1;
    else
      r = mid;
  }
  //-1 不在
  // 0 在边上
  // 1 在内部
  return -1;
}
```

第 8 章

动态规划与优化

> 状态转移、背包和相关优化技巧。

8.1 背包与方案恢复

用途：给出目标和可达性及方案恢复的背包实现。**复杂度：**令 $n = |a|$、$V = \max_i a_i$。状态窗口大小为 $2V$，空间为 $O(nV)$；在该实现的摊还分析下时间为 $O(nV)$，因此复杂度取决于 a 的最大值域，而不是只取决于目标 s；它实际只保留 s-V+1 到 s+V-1 附近的和。使用前还应确认题目满足模板所需的非负权值和边界条件。

```
vector<bool> subset_sum(const vector<int> &a, int s) {
  if (s == 0)
    return vector<bool>(a.size(), false);
  int n = a.size();
  int mx = *max_element(a.begin(), a.end());
  int b = 0, sb = 0;
  while (b < n && a[b] + sb <= s)
    sb += a[b++];
  if (b == n && sb < s)
    return {};
  int offset = s - mx + 1;
  vector<int> dp(mx * 2, -1);
  vector<vector<int>> pre(n, vector<int>(mx * 2, -1));
  dp[sb - offset] = b;
  for (int i = b; i < n; i++) {
    vector<int> ndp(dp);
    vector<int> &prei = pre[i];
    for (int j = mx - 1; j >= 0; j--) {
      if (ndp[j + a[i]] < ndp[j]) {
        prei[j + a[i]] = -2;
        ndp[j + a[i]] = ndp[j];
      }
    }
    for (int j = mx * 2 - 1; j >= mx; j--) {
      for (int k = ndp[j] - 1; k >= max(dp[j], 0); k--) {
        if (ndp[j - a[k]] < k) {
          prei[j - a[k]] = k;
          ndp[j - a[k]] = k;
        }
      }
    }
    swap(dp, ndp);
  }
  if (dp[mx - 1] == -1)
    return {};
  vector<bool> res(n, false);
  int i = n - 1, j = mx - 1;
  while (i >= b) {
    int p = pre[i][j];
    if (p == -2) {
      res[i] = !res[i];
      j -= a[i];
```

```
      i--;
    } else if (p == -1)
      i--;
    else {
      res[p] = !res[p];
      j += a[p];
    }
  }
  while (i >= 0) {
    res[i] = !res[i];
    i--;
  }
  return res;
}
```